PHP 8.x — BÀI TẬP TỰ HỌC

PHP cho người học Laravel

Bộ bài tập thực hành từ cơ bản đến nâng cao, kèm đáp án và giải thích chi tiết ở phần sau

Gồm 77 bài tập thuộc 12 chương

Mọi đáp án đã được chạy thật bằng PHP 8.5 — phần "Kết quả khi chạy" là output thực tế, không phải gõ tay

Thư mục thực hành: laravel-crash-course/php-warmup

Mục lục

0	Cách dùng tài liệu này	ĐỌC TRƯỚC
1	Nền tảng ngôn ngữ: chuỗi, so sánh, hằng, kiểu	5 bài
2	Array cơ bản: indexed, associative, mảng lồng nhau	4 bài
3	Truy cập mảng an toàn: ??, isset, array_key_exists	3 bài
4	Hàm xử lý array: map, filter, reduce, column	9 bài
5	Sắp xếp: <=>, usort và họ sort	5 bài
6	Toán tử PHP 8: ??, ?:, ??=, ?->, match	6 bài
7	Class và OOP cơ bản	6 bài
8	File, CSV, JSON và xử lý lỗi	7 bài
9	Luyện logic: sắp xếp, tìm kiếm, tìm cặp, nhóm	12 bài
10	Closure và arrow function	7 bài
11	Tổng hợp: mini project đơn hàng	2 bài
12	SQL nền tảng với SQLite (chạy bằng PDO)	11 bài
PL	Phụ lục: bảng hàm cần nhớ và lỗi hay gặp	TRA CỨU

> **Cấu trúc:** Phần 1 là toàn bộ đề bài (kèm kết quả mong đợi cho bài viết code). Phần 2 ở nửa sau là đáp án và giải thích. Bấm vào liên kết "Xem đáp án" để nhảy đến đúng bài, và "Về đề bài" để quay lại.

CHƯƠNG 0

Cách dùng tài liệu này

Mục tiêu

Bạn tự code 100% các bài tập, đối chiếu với đáp án và giải thích, để nắm chắc PHP trước khi vào Laravel. Tài liệu này **bổ sung** cho lộ trình Laravel (file `laravel-crash-course-plan.pdf`), không thay thế nó: lộ trình dạy lý thuyết Laravel, tài liệu này cho bạn chỗ luyện tay PHP thuần.

Quy trình làm một bài (làm đúng thứ tự)

1. Đọc đề, **dự đoán** hoặc tự phác thảo cách làm trên giấy.
2. Gõ code vào một file `.php` trong `php-warmup`, chạy bằng lệnh `php ten-file.php`.
3. So sánh với mục "Kết quả khi chạy" trong đề (nếu có). Khác thì tự debug, nhìn lỗi PHP báo ở dòng nào.
4. Kẹt hơn **15 phút** mới mở đáp án. Đọc đáp án rồi **đóng lại** và viết lại từ trí nhớ.
5. Làm thí nghiệm "phá code": đổi một dấu, một điều kiện, xoá một `use`, xem lỗi gì. Hiểu lỗi nhớ lâu hơn hiểu đúng.

Chạy file và đọc lỗi

- Chạy: `php ten-file.php`. Kiểm tra phiên bản: `php -v` (tài liệu chạy kiểm tra trên PHP 8.5, yêu cầu tối thiểu 8.3).
- `Warning` và `Notice`: chương trình vẫn chạy nhưng có vấn đề (thường là đọc key không tồn tại). Đừng bỏ qua.
- `Fatal error: Uncaught ...`: ném exception mà không ai bắt. Đọc dòng đầu: loại lỗi và message; dòng `#0` cho biết hàm nào, file nào, dòng nào.
- Dùng `print_r($x)` hoặc `var_dump($x)` để xem nội dung biến, `json_encode($x)` để in gọn một dòng.

Độ khó

Mỗi bài có ★ (dễ), ★★ (vừa), ★★★ (khó, cần suy nghĩ). Bài **dự đoán output** được đánh dấu rõ: chỉ có code, không có kết quả mong đợi, bạn phải tự đoán trước khi chạy.

Bảng chương và file thực hành tương ứng

Chương	Nội dung	File bạn đã làm
1	Chuỗi, so sánh, hằng, kiểu	`index.php`
2	Array cơ bản	`array-basics.php`, `array.php`
3	Truy cập mảng an toàn	`array-safe.php`
4	map, filter, reduce, column	`array-function.php`, `array-keys.php`, `array-column.php`, `array-excires.php`
5	Sắp xếp	`array-sort.php`
6	Toán tử PHP 8	`operators.php`
7	Class và OOP	`php-class/*.php`
8	File, CSV, JSON, exception	`file.php`

9	Logic và thuật toán	`logic.php`
10	Closure	`closure/closure.php`, `closure/excirse.php`
11	Mini project tổng hợp	tự tạo `project/`
12	SQL nền tảng (SQLite, PDO)	tự tạo `sql/`

Quy ước về đáp án

Có nhiều cách đúng cho cùng một bài. Đáp án trong tài liệu là **một** cách rõ ràng và an toàn, không phải cách duy nhất. Nếu code của bạn cho cùng kết quả (kể cả các trường hợp biên), nó vẫn đúng. Hãy so sánh hai cách, hiểu vì sao đáp án chọn như vậy.

Phần chưa có trong tài liệu này

Các chủ đề PHP sau nằm trong lộ trình Laravel nhưng chưa có bài tập ở đây: `interface`, `abstract class`, `trait`, `enum`, `static`, namespace + Composer autoload (PSR-4), attribute, và dependency injection. Khi bạn học đến đó sẽ có thể bổ sung chương mới.

Checklist tự đánh giá cuối khoá

Đánh dấu khi bạn **tự làm được không cần xem**:

- Giải thích được khác nhau của `==` / `===`, `??` / `?:`, `isset` / `array_key_exists`.
- Biết `array_filter` giữ key và biết khi nào dùng `array_values`.
- Phân biệt `sort` / `asort` / `usort` và biết hàm nào giữ key.
- Viết được `usort` nhiều tiêu chí với `<=>`.
- Viết được class có constructor promotion, `private`, `readonly`, ném và bắt exception.
- Đọc/ghi được file, CSV, JSON; xử lý được file không tồn tại.
- Phân biệt `use ($x)`, `use (&$x)` và `fn`; viết được hàm trả về closure.
- Giải thích được vì sao tìm nhị phân nhanh hơn và vì sao dùng hash map (`isset`) nhanh hơn `in_array`.

PHẦN 1

ĐỀ BÀI

Tự làm trước khi xem đáp án.

CHƯƠNG 1

Nền tảng ngôn ngữ: chuỗi, so sánh, hằng, kiểu

Chương này ôn những điều căn bản nhất của PHP mà người từ JavaScript/TypeScript sang hay vấp: nháy đơn khác nháy kép, nối chuỗi bằng dấu chấm, `==` khác `===`, và vì sao nên bật `declare(strict_types=1)`.

Ứng với file thực hành: `php-warmup/index.php`.

Bài 1.1 Nháy đơn và nháy kép

★☆☆

Đọc đoạn code bên dưới và **dự đoán chính xác** 5 dòng output. Chú ý dòng nào PHP thay biến bằng giá trị, dòng nào giữ nguyên chữ.

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

$name = 'Bao';
$age = 20;

echo 'Xin chào $name', "\n";
echo "Xin chào $name", "\n";
echo "Năm sau {$name} sẽ " . ($age + 1) . " tuổi", "\n";
echo 'Dòng 1\nDòng 2', "\n";
echo "Dòng 1\nDòng 2", "\n";
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 1.1 →

Bài 1.2 Ba cách ghép chuỗi

★☆☆

Với `$name = 'Bao'` và `$age = 20`, in ra đúng câu `Tôi là Bao, 20 tuổi, học PHP.` **ba lần**, mỗi lần bằng một cách khác nhau:

1. Nối chuỗi bằng toán tử `.`
2. Nội suy biến trong nháy kép
3. Hàm `sprintf` với `%s` và `%d`

Mỗi câu phải nằm trên một dòng riêng.

KẾT QUẢ KHI CHẠY

```
Tôi là Bao, 20 tuổi, học PHP.
Tôi là Bao, 20 tuổi, học PHP.
Tôi là Bao, 20 tuổi, học PHP.
```

Xem đáp án bài 1.2 →

Bài 1.3 So sánh == và ===

★★☆

Dự đoán kết quả `var_dump` (chỉ `true` hoặc `false`) của 9 dòng sau. Sau khi dự đoán, hãy chạy và đếm xem bạn đoán đúng mấy dòng.

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

var_dump(0 == 'abc');
var_dump('1e3' == '1000');
var_dump('1e3' === '1000');
var_dump(null == false);
var_dump('0' == false);
var_dump([] == false);
var_dump('abc' == 0);
var_dump(100 == '100');
var_dump(100 === '100');
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 1.3 →

Bài 1.4 Hằng số: const và define

★☆☆

Khai báo hai hằng:

- `TAX_RATE = 0.1` bằng `const`
- `SHOP_NAME = 'Cửa hàng PHP'` bằng `define()`

Viết hàm `priceWithTax(int $price): float` trả giá đã cộng thuế. In tên cửa hàng, rồi in giá sau thuế của `200000` theo định dạng kiểu Việt Nam (dấu chấm ngăn cách hàng nghìn): `220.000`.

KẾT QUẢ KHI CHẠY

```
Cửa hàng PHP
220.000
```

Xem đáp án bài 1.4 →

Bài 1.5 declare(strict_types=1) và TypeError

★★☆

Viết hàm `add(int $a, int $b): int`. Gọi `add(2, 3)` và in kết quả. Sau đó gọi `add('5', 3)` (truyền chuỗi `'5'` thay vì số) bên trong `try/catch` bắt `TypeError`, và in ra **tên class** của lỗi (dùng `get_class($e)`).

KẾT QUẢ KHI CHẠY

```
5
TypeError
```

Xem đáp án bài 1.5 →

CHƯƠNG 2

Array cơ bản: indexed, associative, mảng lồng nhau

PHP chỉ có **một kiểu array** nhưng nó gánh cả hai vai: danh sách (như `Array` của JS) và từ điển key-value (như `Object` / `Map` của JS). Laravel dùng array ở khắp nơi: config, request data, kết quả query, dữ liệu validate.

Ứng với file thực hành: `php-warmup/array-basics.php`, `array.php`.

Bài 2.1 Indexed array

★☆☆

Tạo mảng `$tags = ['php', 'laravel']`. Thực hiện lần lượt và in kết quả mỗi bước trên một dòng:

1. Thêm `'composer'` vào cuối mảng.
2. In phần tử đầu tiên.
3. In số phần tử.
4. In phần tử cuối cùng (không được gõ cứng chỉ số `2`).
5. In `yes` hoặc `no` cho câu hỏi "mảng có chứa `'laravel'` không?" (dùng `in_array` với tham số strict).
6. In toàn bộ mảng thành một chuỗi, các phần tử cách nhau bằng dấu phẩy và khoảng trắng.

KẾT QUẢ KHI CHẠY

```
php
3
composer
yes
php, laravel, composer
```

Xem đáp án bài 2.1 →

Bài 2.2 Associative array

★☆☆

Tạo `$post = ['title' => 'Hello', 'views' => 10]`. Lần lượt:

1. In `title`.
2. Đổi `views` thành `11`.
3. Thêm key `author` có giá trị `'Bao'`.
4. In `author`.
5. Xoá key `author` bằng `unset`.
6. In cả mảng ra dạng JSON (dùng `json_encode`) trên một dòng.

KẾT QUẢ KHI CHẠY

```
Hello
Bao
{"title":"Hello","views":11}
```

Xem đáp án bài 2.2 →

Bài 2.3 foreach với key và value

★☆☆

Cho mảng bài viết:

```php
$posts = [
    ['title' => 'A', 'views' => 5],
    ['title' => 'B', 'views' => 20],
    ['title' => 'C', 'views' => 11],
];
```

Dùng `foreach` in từng dòng theo mẫu `STT 1: A (5 views)`. Số thứ tự bắt đầu từ 1, dùng `printf` với `%d` và `%s`.

KẾT QUẢ KHI CHẠY

```
STT 1: A (5 views)
STT 2: B (20 views)
STT 3: C (11 views)
```

Xem đáp án bài 2.3 →

Bài 2.4 Mảng hai chiều

★★☆

Với `$posts` ở bài 2.3, hãy in lần lượt:

1. Title của bài thứ **ba** bằng cách truy cập trực tiếp (không dùng vòng lặp).
2. Tổng `views` của tất cả bài, tính bằng `foreach`.
3. Title của bài có `views` lớn nhất (tìm bằng `foreach` và một biến theo dõi, không dùng `usort`).

KẾT QUẢ KHI CHẠY

```
C
36
B
```

Xem đáp án bài 2.4 →

CHƯƠNG 3

Truy cập mảng an toàn: ??, isset, array_key_exists

Đọc một key không tồn tại trong PHP cho ra **cảnh báo** `Undefined array key` (và giá trị `null`). Trong ứng dụng thật, dữ liệu từ request hay API luôn có thể thiếu key, nên phải đọc an toàn.

Ứng với file thực hành: `php-warmup/array-safe.php`.

Bài 3.1 Toán tử ?? trên mảng

★☆☆

Dự đoán output của đoạn code sau (4 dòng). Chú ý dòng 3: key `views` **tồn tại** nhưng giá trị là `null`.

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

$post = ['title' => 'Hello', 'views' => null];

echo $post['title'] ?? 'N/A', "\n";
echo $post['xxx'] ?? 'N/A', "\n";
echo $post['views'] ?? 'N/A', "\n";

$posts = [['title' => 'A'], ['title' => 'B']];
echo $posts[1]['title'], "\n";
echo $posts[5]['title'] ?? 'không có', "\n";
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 3.1 →

Bài 3.2 isset và array_key_exists

★★☆

Dự đoán kết quả của ba lệnh `var_dump` với cùng một mảng. Đây là điểm khác biệt hay bị hỏi.

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

$post = ['title' => 'Hello', 'views' => null];

var_dump(isset($post['title']));
var_dump(isset($post['views']));
var_dump(array_key_exists('views', $post));
var_dump(isset($post['xxx']));
var_dump(array_key_exists('xxx', $post));
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 3.2 →

Bài 3.3 Hàm đọc an toàn

★★☆

Viết hàm `safeTitle(array $posts, int $index): string` trả về title của bài ở vị trí `$index`, hoặc chuỗi `'không có'` nếu vị trí đó không tồn tại hoặc bài đó thiếu key `title`.

Kiểm tra với `$posts = [['title' => 'A'], ['views' => 3], ['title' => 'C']]` và các chỉ số `0`, `1`, `2`, `9`.

KẾT QUẢ KHI CHẠY

```
0: A
1: không có
2: C
9: không có
```

Xem đáp án bài 3.3 →

CHƯƠNG 4

Hàm xử lý array: map, filter, reduce, column

Ba hàm `array_map`, `array_filter`, `array_reduce` là bộ ba quan trọng nhất khi xử lý dữ liệu. Chúng nhận một **closure** (hàm không tên) làm tham số. Trong Laravel bạn sẽ gặp lại chúng dưới dạng `Collection` (`collect($x)->map()->filter()`), nhưng phải hiểu mảng thuần trước.

Điều cần nhớ: `array_filter` **giữ nguyên key cũ**, nên kết quả có thể bị "khuyết" chỉ số và biến thành object khi `json_encode`.

Ứng với file thực hành: `array-function.php`, `array-keys.php`, `array-column.php`, `array-excires.php`.

Bài 4.1 array_map: biến đổi từng phần tử

★☆☆

Cho `$numbers = [1, 2, 3, 4, 5, 6]`. Dùng `array_map` với arrow function để tạo mảng mới gồm mỗi số **nhân đôi**, rồi in mảng đó bằng `json_encode`.

KẾT QUẢ KHI CHẠY

```
[2,4,6,8,10,12]
```

Xem đáp án bài 4.1 →

Bài 4.2 array_filter giữ nguyên key

★★☆

Dự đoán output (hai dòng). Dòng 2 là chỗ nhiều người ngạc nhiên.

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

$numbers = [1, 2, 3, 4, 5, 6];

$evens = array_filter($numbers, fn (int $n): bool => $n % 2 === 0);
echo json_encode($evens), "\n";
var_dump(array_is_list($evens));
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 4.2 →

Bài 4.3 array_reduce: gộp thành một giá trị

★★☆

Với `$numbers = [1, 2, 3, 4, 5, 6]`, dùng `array_reduce` để tính hai giá trị:

1. Tổng các số.
2. Tích các số (`1 × 2 × 3 × 4 × 5 × 6`).

In hai kết quả trên hai dòng.

KẾT QUẢ KHI CHẠY

```
21
720
```

Xem đáp án bài 4.3 →

Bài 4.4 Sửa lỗi JSON thành object: array_values

★★☆

Đoạn code dưới đây `json_encode` ra **object** thay vì mảng. Hãy sửa để kết quả là mảng JSON `[20,40,60]`, và in thêm `true` hoặc `false` cho `array_is_list` của kết quả.

```
$numbers = [1, 2, 3, 4, 5, 6];
$evens = array_filter($numbers, fn (int $n): bool => $n % 2 === 0);
$scaled = array_map(fn (int $n): int => $n * 10, $evens);
echo json_encode($scaled), "\n";
```

KẾT QUẢ KHI CHẠY

```
{"1":20,"3":40,"5":60}
[20,40,60]
bool(false)
bool(true)
```

Xem đáp án bài 4.4 →

Bài 4.5 array_column

★★☆

Với `$posts` bên dưới, in ba dòng JSON:

1. Danh sách title.
2. Mảng dạng `title => views` (key là title, giá trị là views).
3. Danh sách title của những bài có `views > 10`.

```
$posts = [
    ['title' => 'A', 'views' => 5],
    ['title' => 'B', 'views' => 20],
    ['title' => 'C', 'views' => 11],
];
```

KẾT QUẢ KHI CHẠY

```
["A","B","C"]
{"A":5,"B":20,"C":11}
["B","C"]
```

Xem đáp án bài 4.5 →

Bài 4.6 Chuỗi hàm: lẻ, bình phương, tổng

★★☆

Với `$numbers = [1, 2, 3, 4, 5, 6]` : lấy các số **lẻ**, **bình phương** từng số, rồi **cộng tất cả lại**. Viết bằng ba hàm `array_filter` , `array_map` , `array_reduce` nối nhau, in kết quả.

Kiểm tra bằng tay trước: `1² + 3² + 5² = 1 + 9 + 25 = 35` .

KẾT QUẢ KHI CHẠY

```
35
```

Xem đáp án bài 4.6 →

Bài 4.7 Lọc rồi tính tổng cột

★★☆

Với 5 bài viết bên dưới, tính **tổng views của các bài có views > 10**. Dùng `array_filter`, `array_column`, `array_sum`. Kết quả mong đợi: `20 + 11 + 40 = 71`.

```
$posts: A=5, B=20, C=11, D=3, E=40  (title, views)
```

KẾT QUẢ KHI CHẠY

```
71
```

Xem đáp án bài 4.7 →

Bài 4.8 Top 3 bài nhiều views

★★☆

Từ 5 bài viết A..E ở bài 4.7, in ra **title của 3 bài có nhiều views nhất**, cách nhau bằng dấu phẩy, theo thứ tự giảm dần: `E, B, C`.

Dùng `usort` (sao chép mảng trước để không làm đổi `$posts` gốc) và `array_slice`.

KẾT QUẢ KHI CHẠY

```
E, B, C
A, B, C, D, E
```

Xem đáp án bài 4.8 →

Bài 4.9 Bộ hàm tiện ích hay dùng

★★☆

Cho `$nums = [3, 1, 4, 1, 5, 9, 2, 6, 5]`. In ra mỗi dòng một giá trị theo nhãn:

- `sum`: tổng
- `max`, `min`: lớn nhất, nhỏ nhất
- `count`: số phần tử
- `unique`: các giá trị không trùng (JSON, là **mảng** chứ không phải object)
- `has4`: `yes` / `no` nếu có số 4
- `index9`: vị trí của số 9 (dùng `array_search`)
- `reverse`: mảng đảo ngược (JSON)
- `first3`: 3 phần tử đầu (JSON)
- `merged`: nối với `[7, 8]` (JSON)

KẾT QUẢ KHI CHẠY

```
sum: 36
max: 9
min: 1
count: 9
unique: [3,1,4,5,9,2,6]
has4: yes
index9: 5
reverse: [5,6,2,9,5,1,4,1,3]
first3: [3,1,4]
merged: [3,1,4,1,5,9,2,6,5,7,8]
```

Xem đáp án bài 4.9 →

CHƯƠNG 5

Sắp xếp: <=>, usort và họ sort

PHP có sẵn nhiều hàm sắp xếp. Chọn đúng hàm phụ thuộc vào hai câu hỏi: **có cần giữ key không** và **sắp theo giá trị hay theo key**.

Hàm so sánh tuỳ chỉnh của `usort` dùng toán tử **spaceship** `<=>`, trả về `-1`, `0` hoặc `1`.

Ứng với file thực hành: `array-sort.php`, `array-excires.php`, `operators.php` (bài 1.5).

Bài 5.1 Toán tử spaceship <=>

★☆☆

Dự đoán kết quả `var_dump` của các lệnh sau (mỗi dòng là một số nguyên).

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

var_dump(1 <=> 2);
var_dump(2 <=> 2);
var_dump(3 <=> 2);
var_dump('a' <=> 'b');
var_dump('b' <=> 'a');
var_dump([1, 2] <=> [1, 3]);
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 5.1 →

Bài 5.2 usort giảm dần mà không đụng mảng gốc

★★☆

Với 5 bài viết A..E (views 5, 20, 11, 3, 40), sắp xếp theo `views` **giảm dần** vào một biến mới (không đổi `$posts` gốc). In hai dòng title: sau khi sort, và mảng gốc.

KẾT QUẢ KHI CHẠY

```
E, B, C, A, D
A, B, C, D, E
```

Xem đáp án bài 5.2 →

Bài 5.3 Sort nhiều tiêu chí

★★★

Với dữ liệu dưới đây, sắp xếp theo `views` **giảm dần**; nếu `views` bằng nhau thì theo `title` **tăng dần** (A đến Z). In ra từng dòng `title - views`.

```
Vue 50, Laravel 100, Docker 100, PHP 50, Redis 100
```

Gợi ý: nếu hai bài bằng views thì `<=>` thứ nhất trả `0`, mà `0` là falsy, hãy tận dụng toán tử `?:`.

KẾT QUẢ KHI CHẠY

```
Docker - 100
Laravel - 100
Redis - 100
```

```
PHP - 50
Vue - 50
```

Xem đáp án bài 5.3 →

Bài 5.4 Tự viết bubble sort cho mảng bài viết ★★★

Không dùng `usort`. Viết hàm `handleSort(array $arr): array` sắp xếp các bài viết theo `views` giảm dần bằng bubble sort: lặp đi lặp lại so sánh hai phần tử cạnh nhau và đổi chỗ nếu sai thứ tự. Dùng hai vòng `for` lồng nhau. In danh sách title theo thứ tự đã sắp, và chứng minh `$posts` gốc không bị đổi.

KẾT QUẢ KHI CHẠY

```
E, B, C, A, D
A, B, C, D, E
```

Xem đáp án bài 5.4 →

Bài 5.5 Họ hàm sort: giữ key hay không ★★★

Cho `$scores = ['an' => 70, 'binh' => 90, 'chi' => 80]`. Dự đoán 7 dòng JSON được in ra, mỗi dòng bắt đầu từ bản sao mới của `$scores`:

1. `asort`
2. `arsort`
3. `ksort`
4. `krsort`
5. `sort`
6. `rsort`
7. `array_values` rồi `sort`

Đặc biệt chú ý dòng nào mất tên (key) và đổi thành danh sách.

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

$scores = ['an' => 70, 'binh' => 90, 'chi' => 80];

$a = $scores;
asort($a);
echo 'asort:  ', json_encode($a), "\n";

$a = $scores;
arsort($a);
echo 'arsort: ', json_encode($a), "\n";

$a = $scores;
ksort($a);
echo 'ksort:  ', json_encode($a), "\n";

$a = $scores;
krsort($a);
echo 'krsort: ', json_encode($a), "\n";
```

```php
$a = $scores;
sort($a);
echo 'sort:   ', json_encode($a), "\n";

$a = $scores;
rsort($a);
echo 'rsort:  ', json_encode($a), "\n";

$a = array_values($scores);
sort($a);
echo 'values: ', json_encode($a), "\n";
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 5.5 →

CHƯƠNG 6

Toán tử PHP 8: ??, ?:, ??=, ?->, match

Các toán tử hiện đại của PHP 8 xuất hiện liên tục trong code Laravel. Chương này luyện từng cái, kèm chỗ dễ nhầm.

Ứng với file thực hành: `php-warmup/operators.php`.

Bài 6.1 ?? khác ?:

★★☆

Cho `$settings = ['theme' => 'dark', 'perPage' => 0, 'lang' => null]`. In ra 6 dòng, mỗi dòng một giá trị:

1. `theme` với mặc định `'light'` (dùng `??`)
2. `perPage` với mặc định `15` dùng `??`
3. `perPage` với mặc định `15` dùng `?:`
4. `lang` với mặc định `'vi'` dùng `??`
5. `timezone` (không có trong mảng) mặc định `'UTC'` dùng `??`
6. `perPage` dùng `isset` rồi in `có` hoặc `không`

Hãy tự dự đoán dòng 2 và dòng 3 trước khi chạy.

KẾT QUẢ KHI CHẠY

```
dark
0
15
vi
UTC
có
```

Xem đáp án bài 6.1 →

Bài 6.2 ??= gán nếu chưa có

★★☆

Cho `$config = ['limit' => 0]`. Dùng `??=`:

- gán `limit` mặc định `15`,
- gán `sort` mặc định `'desc'`.

In `$config` bằng `json_encode`. Sau đó dùng `??=` để **đếm** số lần xuất hiện của từng chữ trong `['a', 'b', 'a', 'c', 'a']` (kết quả `{"a":3,"b":1,"c":1}`).

KẾT QUẢ KHI CHẠY

```
{"limit":0,"sort":"desc"}
{"a":3,"b":1,"c":1}
```

Xem đáp án bài 6.2 →

Bài 6.3 Nullsafe ?->

★★☆

Tạo ba class bằng constructor promotion:

- `Address` có `string $city`

- `Profile` có `?Address $address = null`
- `User` có `?Profile $profile = null`

Tạo ba user: (1) đủ dữ liệu với city `'Hà Nội'`, (2) có profile nhưng chưa có address, (3) không có profile. In `city` của từng user bằng `?->` kèm mặc định `'Chưa rõ'`.

KẾT QUẢ KHI CHẠY

```
Hà Nội
Chưa rõ
Chưa rõ
```

Xem đáp án bài 6.3 →

Bài 6.4 match thay switch

★★☆

Viết hàm `httpLabel(int $code): string` dùng `match` trả về: 200 và 201 là `'Thành công'`, 301 và 302 là `'Chuyển hướng'`, 404 là `'Không tìm thấy'`, 500 là `'Lỗi server'`. **Không** viết nhánh `default`.

In nhãn cho 200, 404, 301. Sau đó gọi với 418 bên trong `try/catch` bắt `UnhandledMatchError` và in ra `Không hỗ trợ mã này` thay vì để chương trình dừng.

KẾT QUẢ KHI CHẠY

```
Thành công
Không tìm thấy
Chuyển hướng
Không hỗ trợ mã này
```

Xem đáp án bài 6.4 →

Bài 6.5 match(true) cho điều kiện khoảng

★★☆

Phí vận chuyển theo tổng tiền đơn hàng:

- từ 500000 trở lên: miễn phí (0)
- từ 200000 đến dưới 500000: 15000
- dưới 200000: 30000

Viết hàm `shippingFee(int $total): int` dùng `match (true)`. In phí cho các tổng: 100000, 200000, 450000, 500000.

KẾT QUẢ KHI CHẠY

```
100000 => 30000
200000 => 15000
450000 => 15000
500000 => 0
```

Xem đáp án bài 6.5 →

Bài 6.6 match so sánh nghiêm ngặt, switch so sánh lỏng

★★★

Dự đoán output của đoạn code sau. Chú ý biến `$v` là chuỗi `'1'` nhưng `case` và nhánh `match` là số nguyên `1`.

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

$v = '1';

switch ($v) {
    case 1:
        echo 'switch khớp', "\n";
        break;
    default:
        echo 'switch không khớp', "\n";
}

echo match ($v) {
    1 => 'match khớp',
    default => 'match không khớp',
}, "\n";
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 6.6 →

CHƯƠNG 7

Class và OOP cơ bản

Laravel là framework hướng đối tượng: controller, model, service, job đều là class. Chương này luyện phần cốt lõi: **constructor property promotion**, thuộc tính `public` / `private`, `readonly`, `final`, so sánh object và ném exception từ method.

Quy ước PHP: dùng `$this->` để truy cập thành viên của chính object, `->` để gọi method/property từ bên ngoài, `new` để tạo object.

Ứng với file thực hành: `php-class/class-basics.php`, `class-readonly.php`, `product.php`.

Bài 7.1 Class đầu tiên: Post

★☆☆

Viết class `Post` với:

- constructor nhận `public string $title` và `public int $views = 0` (dùng constructor promotion),
- method `addView(): void` tăng `views` thêm 1,
- method `summary(): string` trả về chuỗi dạng `Hello (2 views)`.

Tạo hai post `$a = new Post('Hello')` và `$b = new Post('PHP', 10)`. Gọi `addView()` hai lần cho `$a`, một lần cho `$b`, rồi in `summary()` của cả hai và `title` của `$a`.

KẾT QUẢ KHI CHẠY

```
Hello (2 views)
PHP (11 views)
Hello
```

Xem đáp án bài 7.1 →

Bài 7.2 private và encapsulation

★★☆

Viết class `Product` có `public string $name` và `private int $price` (cả hai qua constructor promotion), method `priceAfterDiscount(int $percent): float` và `label(): string` trả `"Áo - 200000"`.

Tạo `new Product('Áo', 200000)`, in giá sau giảm 10% và nhãn. Sau đó thử đọc `$p->price` **từ bên ngoài** bên trong `try/catch (Error $e)` và in `getMessage()`.

KẾT QUẢ KHI CHẠY

```
180000
Áo - 200000
Cannot access private property Product::$price
```

Xem đáp án bài 7.2 →

Bài 7.3 readonly, final và immutable object

★★★

Viết `final class Money` với `public readonly int $amount` và `public readonly string $currency = 'VND'` (constructor promotion). Thêm method `add(Money $other): Money` trả về **một object Money mới** có amount là tổng.

Tạo `$a = new Money(100)`, `$b = $a->add(new Money(50))`. In `amount` của `$a` và `$b`. Cuối cùng thử gán `$a->amount = 999` trong `try/catch (Error $e)` và in `getMessage()`.

KẾT QUẢ KHI CHẠY

```
100
150
Cannot modify readonly property Money::$amount
```

Xem đáp án bài 7.3 →

Bài 7.4 So sánh object: ===, == và cùng instance

★★★

Dùng class `Money` ở bài 7.3 (hãy chép lại). Dự đoán 4 dòng output:

- `$x = new Money(1); $y = $x; $z = new Money(1);`
- `$x === $y`
- `$x === $z`
- `$x == $z`
- `$x == new Money(2)`

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

final class Money
{
    public function __construct(
        public readonly int $amount,
        public readonly string $currency = 'VND',
    ) {}
}

$x = new Money(1);
$y = $x;
$z = new Money(1);

var_dump($x === $y);
var_dump($x === $z);
var_dump($x == $z);
var_dump($x == new Money(2));
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 7.4 →

Bài 7.5 BankAccount: ném và bắt exception

★★★

Viết class `BankAccount` với `private int $balance = 0`. Thêm:

- `deposit(int $amount): void`: nếu `$amount <= 0` thì ném `InvalidArgumentException('Số tiền phải lớn hơn 0')`, ngược lại cộng vào số dư.
- `withdraw(int $amount): void`: nếu rút nhiều hơn số dư thì ném `RuntimeException('Số dư không đủ')`; nếu `$amount <= 0` cũng ném `InvalidArgumentException`.
- `balance(): int` trả về số dư.

Kịch bản: gửi 1000, rút 300, in số dư. Rồi thử rút 5000 và gửi `-5`, mỗi lần bắt đúng loại exception và in message. Thêm khối `finally` in `Xong`.

KẾT QUẢ KHI CHẠY

```
700
Lỗi nghiệp vụ: Số dư không đủ
Xong
Đầu vào sai: Số tiền phải lớn hơn 0
700
```

Xem đáp án bài 7.5 →

Bài 7.6 **Cart: kết hợp class và array** ★★★

Viết class `Item` (final, readonly): `string $name`, `int $price`, `int $qty`. Viết class `Cart` giữ một mảng `private array $items = []` với:

- `add(Item $item): void`
- `total(): int` tính tổng `price * qty` (dùng `array_reduce`)
- `names(): array` trả danh sách tên, **sắp theo thứ tự chữ cái** (dùng `array_map` rồi `sort`).

Thêm các món: Áo 200000 x2, Quần 350000 x1, Mũ 50000 x3. In tổng tiền và danh sách tên (JSON, `JSON_UNESCAPED_UNICODE`).

KẾT QUẢ KHI CHẠY

```
900000
["Mũ","Quần","Áo"]
```

Xem đáp án bài 7.6 →

CHƯƠNG 8

File, CSV, JSON và xử lý lỗi

Chương này làm việc với dữ liệu ngoài chương trình: đọc/ghi file văn bản, CSV, JSON. Đây là nền cho `Storage` (upload file), import/export và log của Laravel.

Mọi bài đều dùng `__DIR__` (thư mục chứa file đang chạy) để đường dẫn đúng bất kể bạn chạy lệnh `php` từ thư mục nào.

Ứng với file thực hành: `php-warmup/file.php`.

Bài 8.1 Ghi đè và ghi nối: file_put_contents

★☆☆

Dùng `file_put_contents` và `file_get_contents` với file `note.txt` nằm cùng thư mục:

1. Ghi `Dòng 1` vào file (xuống dòng ở cuối).
2. Ghi nối thêm `Dòng 2`.
3. Đọc cả file và in ra.
4. Ghi **không có cờ nối** chuỗi `Làm lại\n`, rồi đọc và in lại để thấy điều gì xảy ra.

Cuối cùng xoá file bằng `unlink`.

KẾT QUẢ KHI CHẠY

```
Dòng 1
Dòng 2
---
Làm lại
```

Xem đáp án bài 8.1 →

Bài 8.2 Đọc từng dòng: fopen và fgets

★★☆

Tạo file `lines.txt` gồm 5 dòng: `một`, `hai`, `ba`, một dòng **trống**, rồi `năm`. Rồi đọc bằng hai cách và in:

- **Cách 1:** `fopen` + vòng `while` + `fgets`, in từng dòng kèm số thứ tự (bỏ ký tự xuống dòng bằng `rtrim`).
- **Cách 2:** `file()` với cờ `FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES`, in số dòng đọc được và JSON của mảng.

KẾT QUẢ KHI CHẠY

```
1: một
2: hai
3: ba
4:
5: năm
---
4
["một","hai","ba","năm"]
```

Xem đáp án bài 8.2 →

Bài 8.3 Ghi CSV bằng fputcsv

★★☆

Viết hàm `addExpense(string $path, string $date, string $category, int $amount, string $note): void` ghi **nối** một dòng vào file CSV. Nếu file chưa tồn tại thì ghi dòng tiêu đề `date,category,amount,note`

trước. Gọi 3 lần, ít nhất một `note` chứa dấu phẩy (ví dụ `ăn trưa, cà phê`). In nội dung file bằng `file_get_contents`.

KẾT QUẢ KHI CHẠY

```
date,category,amount,note
2026-10-01,food,45000,"ăn trưa, cà phê"
2026-10-02,transport,30000,xăng
2026-10-03,rent,3000000,"tiền phòng"
```

Xem đáp án bài 8.3 →

Bài 8.4 Đọc CSV thành mảng kết hợp

★★★

Cho sẵn hàm `seedCsv($path)` tạo file mẫu (xem đáp án để chép). Viết hàm `loadExpenses(string $path): array` đọc CSV thành **mảng các mảng kết hợp**, key lấy từ dòng tiêu đề, `amount` ép về `int`. Nếu file không tồn tại thì ném `RuntimeException`. In ra số dòng dữ liệu và một dòng đầu tiên dạng JSON.

KẾT QUẢ KHI CHẠY

```
6
{"date":"2026-10-01","category":"food","amount":45000,"note":"ăn trưa, cà phê"}
int(45000)
```

Xem đáp án bài 8.4 →

Bài 8.5 Báo cáo chi tiêu

★★★

Dựa vào dữ liệu đọc từ CSV ở bài 8.4, viết hàm `report(array $rows): array` trả về mảng gồm:

- `total`: tổng chi (dùng `array_reduce`)
- `byCategory`: tổng chi theo từng danh mục, **sắp giảm dần** (dùng `??=` rồi `arsort`)
- `top`: tên danh mục chi nhiều nhất (dùng `array_key_first`)

In kết quả bằng `json_encode`. Kết quả với dữ liệu mẫu: tổng `3190000`, top là `rent`.

KẾT QUẢ KHI CHẠY

```
{"total":3190000,"byCategory":{"rent":3000000,"food":140000,"transport":50000},"top":"rent"}
```

Xem đáp án bài 8.5 →

Bài 8.6 JSON: ghi ra rồi đọc lại

★★☆

Từ báo cáo ở bài 8.5, ghi ra file `report.json` **định dạng đẹp** (xuống dòng thụt lề) và **giữ nguyên tiếng Việt** (không bị `\u...`). Sau đó đọc file ngược lại bằng `json_decode` thành **mảng** và in `total` cùng `top`. In thêm nội dung file `report.json`.

Để thấy rõ tiếng Việt, thêm key `'ghiChu' => 'Tổng kết tháng 10'` vào mảng báo cáo.

KẾT QUẢ KHI CHẠY

```
{
    "total": 3190000,
    "byCategory": {
        "rent": 3000000,
        "food": 140000,
        "transport": 50000
```

```
    },
    "top": "rent",
    "ghiChu": "Tổng kết tháng 10"
}
3190000
rent
```

Xem đáp án bài 8.6 →

Bài 8.7 **Exception: try, catch, finally** ★★☆

Viết hàm `readOrFail(string $path): string` ném `RuntimeException('Không tìm thấy file: ...')` (dùng `basename`) nếu file không tồn tại, ngược lại trả nội dung file.

Gọi hàm với file không tồn tại bên trong `try/catch (RuntimeException $e)`, in `Lỗi: ...`. Thêm `finally` in `Đã thử đọc file`. Gọi tiếp với một file có thật (tạo tạm bằng `file_put_contents`) và in nội dung.

KẾT QUẢ KHI CHẠY

```
Lỗi: Không tìm thấy file: khong-ton-tai.csv
Đã thử đọc file
xin chào
Đã thử đọc file
```

Xem đáp án bài 8.7 →

CHƯƠNG 9

Luyện logic: sắp xếp, tìm kiếm, tìm cặp, nhóm

Các bài này luyện tư duy vòng lặp và cấu trúc dữ liệu. Chúng không phải là thứ bạn tự viết hằng ngày khi làm Laravel (đã có hàm sẵn), nhưng làm được chúng nghĩa là bạn **hiểu** các hàm sẵn có đang làm gì, và gặp phỏng vấn cũng không ngợp.

Quy tắc luyện: với mỗi bài, (1) tự thử với mảng rỗng và mảng 1 phần tử, (2) lấy giấy bút trace bằng tay một ví dụ nhỏ, (3) chỉ xem đáp án sau 15 phút.

Ứng với file thực hành: `php-warmup/logic.php`.

Bài 9.1 Bubble sort có tham số hướng

★★☆

Viết `bubbleSort(array $arr, bool $desc = false): array`: sắp xếp mảng số tăng dần (mặc định) hoặc giảm dần (khi `$desc = true`), **không dùng hàm sort có sẵn**. Chỉ dùng **một** chỗ so sánh cho cả hai hướng. Thử với `[5, 2, 9, 1, 7]`, mảng rỗng và mảng `[3]`.

KẾT QUẢ KHI CHẠY

```
1, 2, 5, 7, 9
9, 7, 5, 2, 1
|
3
```

Xem đáp án bài 9.1 →

Bài 9.2 Selection sort

★★☆

Viết `selectionSort(array $arr): array` sắp xếp tăng dần. Ý tưởng: với mỗi vị trí `i`, tìm phần tử **nhỏ nhất** ở đoạn từ `i` đến cuối, đổi nó về vị trí `i`. Thử với `[64, 25, 12, 22, 11]` và mảng rỗng.

KẾT QUẢ KHI CHẠY

```
11, 12, 22, 25, 64
|
```

Xem đáp án bài 9.2 →

Bài 9.3 Max, min và lớn thứ hai trong một lượt duyệt

★★★

Viết `stats(array $arr): array` trả `['max' => ..., 'min' => ..., 'second' => ...]`, tìm bằng **một lần duyệt**, không sort. `second` là số lớn thứ hai **khác** `max`; nếu không có thì là `null`. Mảng rỗng thì cả ba là `null`. Thử `[5, 9, 2, 9, 7]`, `[5, 5, 5]`, `[]`.

KẾT QUẢ KHI CHẠY

```
{"max":9,"min":2,"second":7}
{"max":5,"min":5,"second":null}
{"max":null,"min":null,"second":null}
```

Xem đáp án bài 9.3 →

Bài 9.4 Tìm kiếm nhị phân và so sánh với tìm tuần tự ★★★

Viết `binarySearch(array $sorted, int $target, int &$steps = 0): int` trả **vị trí** của `$target` trong mảng **đã sắp tăng dần**, hoặc `-1` nếu không có. Tham số `&$steps` (tham chiếu) trả về số bước lặp. Viết thêm `linearSearch` có cùng chữ ký (duyệt từ đầu). Với mảng `range(1, 1000)` tìm số `777` bằng cả hai và so sánh số bước. Thử cả số `5000` (không có).

KẾT QUẢ KHI CHẠY

```
binary: vị trí 776, 8 bước
linear: vị trí 776, 777 bước
không có: -1
```

Xem đáp án bài 9.4 →

Bài 9.5 Tìm cặp có tổng bằng K (hai cách) ★★★

Cho mảng `[1, 5, 3, 7, 4, 6, 2]` và `K = 8`. Tìm tất cả cặp phần tử có tổng bằng `K` bằng **hai cách**:

- `pairsNested`: hai vòng `for` lồng nhau.
- `pairsWithSum`: duyệt **một lần**, dùng mảng kết hợp làm "sổ" những số đã gặp. Với mỗi số `x`, hỏi sổ xem `K - x` đã có chưa.

In kết quả cả hai cách và so sánh. Giải thích vì sao cách 2 nhanh hơn.

KẾT QUẢ KHI CHẠY

```
nested: [[1,7],[5,3],[6,2]]
hash:   [[5,3],[1,7],[6,2]]
```

Xem đáp án bài 9.5 →

Bài 9.6 Phần tử trùng và số lần xuất hiện ★★☆

Viết `duplicates(array $arr): array` trả về những giá trị xuất hiện **nhiều hơn một lần**, kèm số lần (dạng `giá_trị => số_lần`). Thử `[1, 2, 2, 3, 3, 3, 4]` và `['a', 'b', 'a']`.

KẾT QUẢ KHI CHẠY

```
{"2":2,"3":3}
{"a":2}
[]
```

Xem đáp án bài 9.6 →

Bài 9.7 Nhóm các từ đảo chữ (anagram) ★★★

Cho `['listen', 'silent', 'enlist', 'google', 'gogole', 'cat', 'act']`. Viết `groupAnagrams(array $words): array` gom các từ là anagram của nhau vào cùng một nhóm. In kết quả JSON.

Gợi ý: hai từ là anagram khi **sắp xếp các chữ cái** của chúng cho ra cùng một chuỗi, và chuỗi đó dùng làm key của nhóm.

KẾT QUẢ KHI CHẠY

```
[["listen","silent","enlist"],["google","gogole"],["cat","act"]]
```

Xem đáp án bài 9.7 →

Bài 9.8 Kiểm tra ngoặc hợp lệ bằng ngăn xếp ★★★

Viết `isValidBrackets(string $s): bool` với chuỗi chỉ gồm `()[]{}`. Trả `true` nếu ngoặc đóng/mở khớp đúng thứ tự. Thử: `'([]{})'`, `'([)]'`, `'(('`, chuỗi rỗng, `')('`.

Gợi ý: dùng mảng như **ngăn xếp (stack)**: gặp ngoặc mở thì đẩy vào, gặp ngoặc đóng thì lấy phần tử trên cùng ra và phải khớp.

KẾT QUẢ KHI CHẠY

```
"([]{})" => true
"([)]" => false
"((" => false
"" => true
")(" => false
```

Xem đáp án bài 9.8 →

Bài 9.9 FizzBuzz ★☆☆

In các số từ 1 đến 20, mỗi số một dòng. Nhưng: chia hết cho 3 in `Fizz`, chia hết cho 5 in `Buzz`, chia hết cho cả 3 và 5 in `FizzBuzz`. Dùng `match (true)`.

KẾT QUẢ KHI CHẠY

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
```

Xem đáp án bài 9.9 →

Bài 9.10 Chuỗi đối xứng (palindrome) ★★☆

Viết `isPalindrome(string $s): bool` kiểm tra chuỗi đọc xuôi ngược giống nhau, **bỏ qua hoa thường và bỏ qua mọi ký tự không phải chữ/số** (khoảng trắng, dấu câu). Thử `'Madam'`, `'A man, a plan, a canal: Panama'`, `'hello'`.

KẾT QUẢ KHI CHẠY

```
Madam => true
A man, a plan, a canal: Panama => true
hello => false
```

Xem đáp án bài 9.10 →

Bài 9.11 Số nguyên tố

★★☆

Viết `isPrime(int $n): bool`. Số nguyên tố là số lớn hơn 1 chỉ chia hết cho 1 và chính nó. Chỉ thử chia đến **căn bậc hai** của `n`. In tất cả số nguyên tố nhỏ hơn 50 trên một dòng, cách nhau bằng dấu phẩy.

KẾT QUẢ KHI CHẠY

```
2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47
```

Xem đáp án bài 9.11 →

Bài 9.12 Đếm tần suất từ trong câu

★★☆

Cho câu `'php laravel php vue laravel php'`. Đếm số lần xuất hiện của từng từ, sắp **giảm dần theo số lần**, in từng dòng `từ: số lần`. Dùng `explode`, mẫu bộ đếm `??=` và `arsort`.

KẾT QUẢ KHI CHẠY

```
php: 3
laravel: 2
vue: 1
```

Xem đáp án bài 9.12 →

CHƯƠNG 10

Closure và arrow function

Closure là hàm không tên, lưu vào biến và truyền đi như một giá trị. Laravel dùng closure khắp nơi: route (`Route::get('/', function () {...})`), `DB::transaction(function () {...})` , Collection (`->map(fn ...)`), đăng ký service vào container (`bind('x', fn () => new X())`).

Điểm khác JS quan trọng nhất: closure PHP **không tự thấy biến bên ngoài**, phải khai báo bằng `use` . Arrow function `fn` thì tự bắt biến (theo giá trị).

	`function () use (...)`	`fn () => ...`
Bắt biến ngoài	Phải `use`	Tự động, theo giá trị
Nhiều câu lệnh	Được	Chỉ một biểu thức
Sửa biến ngoài	Được (với `use (&$x)`)	Không

Ứng với file thực hành: `php-warmup/closure/closure.php` và `closure/excirse.php` .

Bài 10.1 Closure bắt biến thế nào? Thí nghiệm A, B, C ★★★

Chạy trong đầu ba đoạn code, đều bắt đầu bằng `$x = 10` , tạo closure, **sau đó đổi** `$x = 99` , rồi gọi closure. Dự đoán ba số in ra.

- **A:** `function () use ($x) { return $x; }`
- **B:** `function () use (&$x) { return $x; }`
- **C:** `fn () => $x`

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

$x = 10;
$a = function () use ($x) {
    return $x;
};
$x = 99;
echo 'A: ', $a(), "\n";

$x = 10;
$b = function () use (&$x) {
    return $x;
};
$x = 99;
echo 'B: ', $b(), "\n";

$x = 10;
$c = fn () => $x;
$x = 99;
echo 'C: ', $c(), "\n";
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 10.1 →

Bài 10.2 array_map: fn và function use ★☆☆

Cho `$prices = [100, 250, 80, 400]` và `$tax = 0.1`. Tạo mảng giá sau thuế (mỗi giá nhân `1 + $tax`) bằng `array_map` **hai cách**: (1) arrow function `fn`, (2) closure `function ... use ($tax)`. In cả hai bằng `implode(', ', ...)` và xác nhận giống nhau.

KẾT QUẢ KHI CHẠY

```
110, 275, 88, 440
110, 275, 88, 440
bool(true)
```

Xem đáp án bài 10.2 →

Bài 10.3 makeCounter: closure nhớ trạng thái ★★★

Viết `makeCounter(): Closure` trả về một closure; mỗi lần gọi closure đó thì tăng bộ đếm thêm 1 và trả về số hiện tại. Tạo hai bộ đếm độc lập `$a` và `$b`, gọi theo thứ tự `$a, $a, $b, $a` và in kết quả. Mong đợi `1, 2, 1, 3`.

KẾT QUẢ KHI CHẠY

```
1
2
1
3
```

Xem đáp án bài 10.3 →

Bài 10.4 Hàm trả về hàm: multiplier ★★☆

Viết `multiplier(int $factor): Closure` trả về closure nhân một số với `$factor`. Tạo `$double = multiplier(2)` và `$triple = multiplier(3)`, in `$double(5)` và `$triple(5)`. Rồi viết `repeatTimes(int $n, callable $action): void` gọi `$action($i)` cho `$i` từ 1 đến `$n`, và dùng nó in `lần 1`, `lần 2`, `lần 3`.

KẾT QUẢ KHI CHẠY

```
10
15
lần 1
lần 2
lần 3
```

Xem đáp án bài 10.4 →

Bài 10.5 compose: ghép hai hàm ★★★

Viết `compose(Closure $f, Closure $g): Closure` trả về closure mới nhận `$x`, chạy `$f` trước rồi đưa kết quả cho `$g`. Với `$f = fn ($x) => $x + 1` và `$g = fn ($x) => $x * 2`: in kết quả `compose($f, $g)(5)` và `compose($g, $f)(5)`.

KẾT QUẢ KHI CHẠY

```
12
11
```

Xem đáp án bài 10.5 →

Bài 10.6 **pipeline: chuỗi nhiều hàm** ★★★

Viết `pipeline(array $fns, mixed $value): mixed` chạy lần lượt các closure trong `$fns`, kết quả của closure trước làm đầu vào cho closure sau, bằng `array_reduce`. Thử với `[+1, *2, -3]` và giá trị đầu `5` (mong đợi `9`), và với mảng hàm **rỗng** (trả lại đúng giá trị đầu).

KẾT QUẢ KHI CHẠY

```
9
5
```

Xem đáp án bài 10.6 →

Bài 10.7 **filterBy: tự cài lại array_filter** ★★★

Viết `filterBy(array $items, Closure $predicate): array` làm giống `array_filter` nhưng bằng `foreach`, **giữ nguyên key gốc**. Dùng nó lọc các bài viết có `views > 50` từ danh sách `Vue 50, Laravel 100, Docker 100, PHP 50, Redis 100`. In JSON kết quả của `filterBy` và so sánh với `array_filter`.

KẾT QUẢ KHI CHẠY

```
["Laravel","Docker","Redis"]
[1,2,4]
bool(true)
```

Xem đáp án bài 10.7 →

CHƯƠNG 11

Tổng hợp: mini project đơn hàng

Chương cuối kết hợp mọi thứ đã học: class `readonly`, mảng, `array_reduce`, `match (true)`, closure, exception, file JSON. Làm xong chương này nghĩa là bạn đã đủ nền PHP để bước sang Laravel.

Cách làm: tự thiết kế trước (vẽ ra giấy các class và hàm), tự code, chạy so với kết quả mong đợi, rồi mới xem đáp án để so sánh cách tổ chức.

Bài 11.1 Tính tiền đơn hàng với mã giảm giá ★★★

Xây hệ thống tính tiền nhỏ:

- `Product` (final, readonly): `name`, `price`.
- `Line` (final, readonly): `Product $product`, `int $qty`. Ném `InvalidArgumentException` nếu `qty <= 0`. Có method `subtotal(): int`.
- Hàm `shippingFee(int $total): int` (dùng `match (true)`): từ 500000 trở lên là 0, từ 200000 là 15000, còn lại 30000.
- Mảng `$discounts` chứa **closure** theo mã: `none` giảm 0; `vip10` giảm 10% (`intdiv($t * 10, 100)`); `minus50k` giảm 50000 nhưng không quá tổng tiền hàng.
- Hàm `checkout(array $lines, string $code, array $discounts): array` trả `subtotal`, `discount`, `shipping`, `total` (= subtotal − discount + shipping). Mã không tồn tại thì ném `InvalidArgumentException`.

Đơn mẫu: Áo 200000 × 2, Mũ 50000 × 1. Chạy với mã `none`, `vip10`, `minus50k`, rồi mã sai `abc`, và tạo `Line` với `qty = 0`. Bắt các lỗi và in message.

KẾT QUẢ KHI CHẠY

```
none: {"subtotal":450000,"discount":0,"shipping":15000,"total":465000}
vip10: {"subtotal":450000,"discount":45000,"shipping":15000,"total":420000}
minus50k: {"subtotal":450000,"discount":50000,"shipping":15000,"total":415000}
Mã giảm giá không tồn tại: abc
Số lượng phải lớn hơn 0
```

Xem đáp án bài 11.1 →

Bài 11.2 Báo cáo đơn hàng: sắp, lưu JSON, đọc lại ★★★

Dùng lại `Product`, `Line` và đơn mẫu ở bài 11.1 (Áo 200000 × 2, Mũ 50000 × 1, thêm Quần 350000 × 1). Hãy:

1. Tạo mảng báo cáo từng dòng dạng `['name' => ..., 'qty' => ..., 'subtotal' => ...]`.
2. Sắp xếp theo `subtotal` **giảm dần** (dùng `usort` với `<=>`).
3. Ghi ra file `order.json` (đẹp, giữ tiếng Việt) cùng key `topProduct` là tên dòng đầu tiên.
4. Đọc lại file bằng `json_decode` thành mảng và in `topProduct` cùng số dòng.
5. Gọi đọc một file không tồn tại và bắt `RuntimeException`.

KẾT QUẢ KHI CHẠY

```
Áo
3
Không tìm thấy file: khong-co.json
```

Xem đáp án bài 11.2 →

CHƯƠNG 12

SQL nền tảng với SQLite (chạy bằng PDO)

Eloquent và Query Builder chỉ là lớp bọc quanh SQL. Không hiểu SQL thì N+1, index, `withCount`, `groupBy` chỉ là học thuộc. Chương này luyện SQL thuần trên **SQLite** (cũng là database mặc định của Laravel 13) và chạy bằng `PDO` của PHP, nên bạn không cần cài thêm gì, chỉ cần extension `pdo_sqlite` (kiểm tra: `php -m | grep -i pdo_sqlite`).

Dữ liệu mẫu bám theo dự án blog xuyên suốt của lộ trình Laravel:

Bảng	Cột	Ghi chú
`users`	`id`, `name`, `email`	3 người: An, Bình, Chi (Chi chưa có bài nào)
`posts`	`id`, `user_id`, `title`, `views`, `published_at`	5 bài, một bài nháp có `published_at` là NULL
`comments`	`id`, `post_id`, `body`	6 bình luận, bài 3 và bài 5 chưa có bình luận nào

Quan hệ: một `user` có nhiều `posts`, một `post` có nhiều `comments`. Cột `user_id` và `post_id` là **khoá ngoại** (foreign key).

Hai hàm tiện ích dùng xuyên suốt chương: `makeDb()` (tạo database trong RAM với dữ liệu mẫu) và `show()` (chạy câu SQL và in từng dòng kết quả). Bài 12.1 cho bạn viết hai hàm này; các bài sau chạy kèm chúng.

> **Lưu ý:** Cú pháp khác biệt nhỏ giữa các hệ quản trị: SQLite dùng `AUTOINCREMENT`, MySQL dùng `AUTO_INCREMENT`; lệnh xem kế hoạch là `EXPLAIN QUERY PLAN` ở SQLite, còn `EXPLAIN` ở MySQL/PostgreSQL. Phần SELECT, JOIN, GROUP BY gần như giống hệt nhau.

Ứng với Part SQL trong tài liệu lộ trình Laravel.

Bài 12.1 Dựng database mẫu bằng PDO

★★☆

Viết hàm `makeDb(): PDO` tạo database SQLite **trong RAM** (`'sqlite::memory:'`) với ba bảng `users`, `posts`, `comments` và dữ liệu mẫu như mô tả ở đầu chương. Yêu cầu:

- Bật chế độ ném exception khi lỗi SQL: `ATTR_ERRMODE` là `ERRMODE_EXCEPTION`.
- Kết quả mặc định ở dạng mảng kết hợp: `ATTR_DEFAULT_FETCH_MODE` là `FETCH_ASSOC`.
- Bật khoá ngoại: `PRAGMA foreign_keys = ON` (SQLite **tắt** kiểm tra khoá ngoại theo mặc định).

Viết thêm hàm `show(PDO $pdo, string $sql, array $params = []): void` chạy câu SQL (dùng `prepare` + `execute`) và in mỗi dòng kết quả trên một dòng, các cột cách nhau bằng `|`, giá trị `NULL` in là chữ `NULL`.

Dùng chúng để in số dòng của mỗi bảng.

KẾT QUẢ KHI CHẠY

```
3
5
6
1 | Học PHP | 120 | 2026-01-05
2 | Học Laravel | 300 | 2026-02-10
3 | Học SQL | 80 | NULL
4 | Docker cơ bản | 150 | 2026-03-01
5 | Redis là gì | 45 | 2026-03-15
```

Xem đáp án bài 12.1 →

Bài 12.2 SELECT, WHERE, ORDER BY, LIMIT

★☆☆

Dùng `makeDb()` và `show()`. Viết các câu SQL để in:

1. Tiêu đề và lượt xem của các bài có `views > 100`, **sắp giảm dần** theo `views`.
2. Hai bài có nhiều lượt xem nhất (`LIMIT`).
3. Hai bài có nhiều lượt xem tiếp theo, tức bỏ qua hai bài đầu (`LIMIT` kèm `OFFSET`), đây chính là cơ chế phân trang.
4. Các bài có tiêu đề bắt đầu bằng chữ `Học` (`LIKE`).
5. Các bài có `views` từ 80 đến 150 (`BETWEEN`) **hoặc** thuộc `user_id` 2, chỉ in `id`, sắp theo `id` (`IN` / `OR`).

Mỗi nhóm cách nhau bằng một dòng `---`.

KẾT QUẢ KHI CHẠY

```
Học Laravel | 300
Docker cơ bản | 150
Học PHP | 120
---
Học Laravel | 300
Docker cơ bản | 150
---
Học PHP | 120
Học SQL | 80
---
Học PHP
Học Laravel
Học SQL
---
1
3
4
5
```

Xem đáp án bài 12.2 →

Bài 12.3 NULL và COUNT

★★★

NULL là chỗ khó nhất của SQL: nó nghĩa là "không biết giá trị", không phải 0 hay chuỗi rỗng. Dự đoán output của 6 câu truy vấn sau. Với mỗi câu, hãy nghĩ trước kết quả là bao nhiêu dòng.

CODE ĐỀ BÀI

```php
<?php

declare(strict_types=1);

$pdo = makeDb();

echo "1: ";
show($pdo, 'SELECT COUNT(*) FROM posts WHERE published_at = NULL');
echo "2: ";
show($pdo, 'SELECT COUNT(*) FROM posts WHERE published_at IS NULL');
echo "3: ";
show($pdo, 'SELECT COUNT(*) FROM posts WHERE published_at IS NOT NULL');
echo "4: ";
```

```
show($pdo, 'SELECT COUNT(*), COUNT(published_at) FROM posts');
echo "5: ";
show($pdo, 'SELECT COUNT(*) FROM posts WHERE published_at != \'2026-01-05\'');
echo "6: ";
show($pdo, 'SELECT title FROM posts WHERE published_at IS NULL OR published_at < \'2026-03-01\'
ORDER BY id');
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, sau đó tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 12.3 →

Bài 12.4 UPDATE, DELETE và ràng buộc khoá ngoại

★★☆

1. Tăng `views` của bài `id = 1` lên 1 bằng **một câu UPDATE** (không đọc ra rồi ghi lại bằng PHP). In `views` mới.
2. Thử xoá user `id = 1` (người đã có bài viết) trong `try/catch (PDOException $e)` và in thông báo lỗi.
3. Xoá user `id = 3` (Chi, chưa có bài) và in số dòng bị xoá bằng `rowCount()`. In số user còn lại.

KẾT QUẢ KHI CHẠY

```
121
SQLSTATE[23000]: Integrity constraint violation: 19 FOREIGN KEY constraint failed
1
2
```

Xem đáp án bài 12.4 →

Bài 12.5 GROUP BY, hàm gộp và HAVING

★★☆

Viết SQL in ra:

1. Với mỗi `user_id`: số bài và tổng lượt xem, cột đặt tên `bai` và `tong_view`, sắp theo `user_id`.
2. Chỉ những user có **tổng lượt xem lớn hơn 200** (`HAVING`).
3. Trung bình lượt xem của **các bài đã xuất bản** (`AVG`, làm tròn 1 chữ số thập phân bằng `ROUND`), lượt xem cao nhất, thấp nhất.
4. Dùng `WHERE` và `HAVING` **cùng lúc**: chỉ tính các bài đã xuất bản, nhóm theo user, giữ lại nhóm có ít nhất 2 bài.

KẾT QUẢ KHI CHẠY

```
1 | 3 | 500
2 | 2 | 195
---
1 | 500
---
153.8 | 300 | 45
---
1 | 2
2 | 2
```

Xem đáp án bài 12.5 →

Bài 12.6 INNER JOIN: ghép bảng theo khoá ngoại

★★☆

Viết SQL dùng `JOIN` để in:

1. Mỗi bài viết kèm **tên tác giả**: `title | name`, sắp theo `posts.id`.

2. Mỗi bình luận kèm **tiêu đề bài** và **tên tác giả bài** (ghép **ba** bảng): `body | title | name`, sắp theo `comments.id`.
3. Số bình luận của mỗi bài **chỉ tính các bài có ít nhất một bình luận** (dùng `JOIN` + `GROUP BY`), sắp giảm dần theo số bình luận.

KẾT QUẢ KHI CHẠY

```
Học PHP | An
Học Laravel | An
Học SQL | An
Docker cơ bản | Bình
Redis là gì | Bình
---
Hay quá | Học PHP | An
Cảm ơn bạn | Học PHP | An
Rất chi tiết | Học Laravel | An
Có ví dụ không? | Học Laravel | An
Đã thử, chạy tốt | Học Laravel | An
Cần thêm phần Sail | Docker cơ bản | Bình
---
Học Laravel | 3
Học PHP | 2
Docker cơ bản | 1
```

Xem đáp án bài 12.6 →

Bài 12.7 LEFT JOIN: giữ lại dòng không có bên kia ★★★

Viết SQL dùng `LEFT JOIN` để in:

1. **Mọi user** kèm số bài viết của họ (kể cả user có 0 bài), dạng `name | so_bai`, sắp theo `users.id`.
2. Những **bài chưa có bình luận nào** (chỉ in `title`): `LEFT JOIN comments` rồi lọc dòng không ghép được.
3. Mọi bài kèm số bình luận (kể cả 0), sắp theo số bình luận giảm dần rồi theo `id`.

KẾT QUẢ KHI CHẠY

```
An | 3
Bình | 2
Chi | 0
---
Học SQL
Redis là gì
---
Học Laravel | 3
Học PHP | 2
Docker cơ bản | 1
Học SQL | 0
Redis là gì | 0
```

Xem đáp án bài 12.7 →

Bài 12.8 Vấn đề N+1 ở tầng SQL ★★★

Cần in danh sách bài viết kèm tên tác giả. Cài đặt **ba cách** và đếm xem mỗi cách chạy bao nhiêu câu SQL (dùng closure có `use (&$queries)` để đếm, ôn lại bài closure):

- **Cách A (N+1):** lấy tất cả bài, rồi với **từng bài** chạy một câu `SELECT name FROM users WHERE id = ?`.

- **Cách B (eager loading):** lấy tất cả bài; lấy các `user_id` khác nhau; chạy **một** câu `WHERE id IN (?, ?, ...)` để lấy tên; ghép trong PHP.
- **Cách C (JOIN):** một câu `JOIN` duy nhất.

In số câu truy vấn của mỗi cách.

KẾT QUẢ KHI CHẠY

```
A (N+1): 6 câu
B (IN): 2 câu
C (JOIN): 1 câu
```

Xem đáp án bài 12.8 →

Bài 12.9 Index và EXPLAIN QUERY PLAN ★★★

Tạo bảng `logs(id, user_id, action)` với **20000 dòng** (chèn trong **một transaction** cho nhanh; `user_id` lấy `$i % 1000`). Rồi:

1. In kế hoạch thực thi (`EXPLAIN QUERY PLAN`) của `SELECT * FROM logs WHERE user_id = 42` **khi chưa có index**.
2. Tạo index `idx_logs_user_id` trên `user_id`, in kế hoạch của **cùng** câu đó.
3. In kế hoạch của `SELECT * FROM logs WHERE action = 'login'` (cột chưa có index).
4. Tạo **index hợp** `(user_id, action)`, in kế hoạch của `WHERE user_id = 42 AND action = 'login'`.

Chỉ in cột `detail` của kế hoạch.

KẾT QUẢ KHI CHẠY

```
1. Chưa có index:
SCAN logs
2. Đã có index user_id:
SEARCH logs USING INDEX idx_logs_user_id (user_id=?)
3. Lọc theo cột chưa có index:
SCAN logs
4. Index hợp (user_id, action):
SEARCH logs USING COVERING INDEX idx_logs_user_action (user_id=? AND action=?)
```

Xem đáp án bài 12.9 →

Bài 12.10 Transaction: tất cả hoặc không gì cả ★★★

Bảng `accounts(id, name, balance)` có An 1000 và Bình 500. Viết hàm `transfer(PDO $pdo, int $from, int $to, int $amount): void` chuyển tiền: trừ ở tài khoản gửi, cộng ở tài khoản nhận, **trong một transaction**. Nếu số dư không đủ thì ném `RuntimeException` và **huỷ** (`rollBack`) mọi thay đổi.

Chuyển 300 từ An sang Bình, in số dư. Rồi thử chuyển 5000 (không đủ tiền) và chứng minh số dư **không đổi** sau lỗi.

KẾT QUẢ KHI CHẠY

```
Ban đâu: An=1000, Bình=500
Sau khi chuyển 300: An=700, Bình=800
Lỗi: Sô´dư không đủ
Sau lần lỗi: An=700, Bình=800
```

Xem đáp án bài 12.10 →

Bài 12.11 SQL injection: ghép chuỗi và tham số hoá ★★★

Với `makeDb()`, một hàm đăng nhập giả nhận `$email` từ người dùng. Chạy cùng đầu vào độc hại `$input = "' OR '1'='1"` bằng **hai cách**:

- **Cách nguy hiểm:** ghép thẳng `$input` vào câu SQL: `"SELECT name FROM users WHERE email = '$input'"`.
- **Cách an toàn:** `prepare` với dấu `?` và `execute([$input])`.

In kết quả cả hai. Sau đó viết hàm `safeSortColumn(string $input): string` chỉ chấp nhận cột trong **danh sách cho phép** (`id`, `title`, `views`), còn lại trả về `id`, và thử với `'views'` và với `'views; DROP TABLE posts'`.

KẾT QUẢ KHI CHẠY

```
Ghép chuôĩ:
An
Bình
Chi
Tham sô´hoá:
views
id
```

Xem đáp án bài 12.11 →

PHẦN 2

ĐÁP ÁN VÀ GIẢI THÍCH CHI TIẾT

Chỉ mở phần này sau khi đã thử ít nhất 15 phút cho mỗi bài.

ĐÁP ÁN — CHƯƠNG 1

Nền tảng ngôn ngữ: chuỗi, so sánh, hằng, kiểu

Đáp án 1.1 Nháy đơn và nháy kép

KẾT QUẢ KHI CHẠY

```
Xin chào $name
Xin chào Bao
Năm sau Bao sẽ 21 tuổi
Dòng 1\nDòng 2
Dòng 1
Dòng 2
```

Giải thích từng dòng

- Dòng 1 dùng **nháy đơn**: PHP giữ nguyên mọi ký tự, kể cả `$name`. Không có nội suy biến.
- Dòng 2 dùng **nháy kép**: PHP thay `$name` bằng `Bao`.
- Dòng 3: `{$name}` là cú pháp nội suy tường minh, cần khi biểu thức phức tạp (ví dụ `{$post['title']}`). Phần `($age + 1)` phải nằm **ngoài** chuỗi và nối bằng dấu `.`, vì trong chuỗi PHP không tự tính phép toán.
- Dòng 4: nháy đơn không hiểu `\n` nên in nguyên hai ký tự `\` và `n`.
- Dòng 5: nháy kép hiểu `\n` là xuống dòng thật, nên chữ "Dòng 2" nằm dòng mới.

Quy tắc nhớ nhanh

Nháy đơn = chữ nguyên văn (nhanh, an toàn). Nháy kép = có nội suy biến và ký tự escape như `\n`, `\t`. Cần xuống dòng hay chèn biến thì dùng nháy kép.

> **Lưu ý:** Người từ JS thường nhầm: JS dùng backtick và `${name}`, PHP dùng nháy kép và `$name` hoặc `{$name}`. Không có template literal.

← Về đề bài 1.1

Đáp án 1.2 Ba cách ghép chuỗi

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$name = 'Bao';
$age = 20;

echo 'Tôi là ' . $name . ', ' . $age . ' tuổi, học PHP.' . "\n";
echo "Tôi là $name, $age tuổi, học PHP.\n";
echo sprintf('Tôi là %s, %d tuổi, học PHP.', $name, $age), "\n";
```

KẾT QUẢ KHI CHẠY

```
Tôi là Bao, 20 tuổi, học PHP.
Tôi là Bao, 20 tuổi, học PHP.
Tôi là Bao, 20 tuổi, học PHP.
```

Giải thích

- **Cách 1** (`.`): PHP không dùng `+` để nối chuỗi. Viết `'a' + 'b'` sẽ báo lỗi `Unsupported operand types` hoặc `TypeError` ở PHP 8.
- **Cách 2** (nội suy): ngắn gọn nhất. Với biến đơn thì `$name` là đủ. Lưu ý dấu `,` ngay sau `$name` : PHP hiểu tên biến kết thúc ở ký tự không phải chữ/số/gạch dưới, nên không bị nhầm.
- **Cách 3** (`sprintf`): định dạng theo mẫu. `%s` là chuỗi, `%d` là số nguyên, `%.2f` là số thực 2 chữ số thập phân. Rất hữu ích khi cần căn lề hoặc làm tròn.

Khi nào dùng cách nào

Chuỗi ngắn có biến: nội suy. Chuỗi cần định dạng số: `sprintf` hoặc `printf` (in thẳng). Chuỗi ghép từ nhiều phần phức tạp: nối bằng `.` .

← Về đề bài 1.2

Đáp án 1.3 So sánh == và ===

KẾT QUẢ KHI CHẠY

```
bool(false)
bool(true)
bool(false)
bool(true)
bool(true)
bool(true)
bool(false)
bool(true)
bool(false)
```

Giải thích

- `==` **ép kiểu** rồi so sánh; `===` so sánh **cả giá trị lẫn kiểu**.
- `0 == 'abc'` và `'abc' == 0` là `false` kể từ PHP 8 (trước PHP 8 là `true` , một bug kinh điển). Từ PHP 8, so sánh số với chuỗi không phải số sẽ so sánh theo kiểu chuỗi.
- `'1e3' == '1000'` là `true` vì cả hai chuỗi đều trông giống số: `1e3` là 1000 theo ký hiệu khoa học. Nhưng `===` thì `false` vì hai chuỗi khác nhau.
- `null == false` , `'0' == false` , `[] == false` đều `true` : đây là các giá trị "falsy" (rỗng) và `==` coi chúng giống `false` .
- `100 == '100'` là `true` (ép chuỗi thành số), nhưng `100 === '100'` là `false` (một bên `int` , một bên `string`).

> **Lưu ý:** Quy tắc vàng của PHP hiện đại: **luôn dùng** `===` **và** `!==` , chỉ dùng `==` khi bạn chủ động muốn ép kiểu. Tài liệu Laravel của bạn cũng nhấn mạnh điều này.

← Về đề bài 1.3

Đáp án 1.4 Hằng số: const và define

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);
```

```php
const TAX_RATE = 0.1;
define('SHOP_NAME', 'Cửa hàng PHP');

function priceWithTax(int $price): float
{
    return $price * (1 + TAX_RATE);
}

echo SHOP_NAME, "\n";
echo number_format(priceWithTax(200000), 0, ',', '.'), "\n";
```

KẾT QUẢ KHI CHẠY

```
Cửa hàng PHP
220.000
```

Giải thích

- Hằng không có dấu `$` khi dùng. Viết `TAX_RATE`, không viết `$TAX_RATE`.
- `const` được xử lý lúc **biên dịch** nên chỉ dùng ở phạm vi ngoài cùng của file (hoặc trong class). `define()` chạy lúc **thực thi** nên dùng được trong `if` hay hàm và tên hằng có thể là biến.
- Trong hàm, hằng **tự thấy** được, không cần `global` hay `use` như biến.
- `number_format($số, số_chữ_số_thập_phân, dấu_thập_phân, dấu_ngăn_nghìn)`: ở đây `0, ',', '.'` cho ra `220.000`. Hàm tự làm tròn nên `220000.00000000003` (sai số của số thực) thành `220.000`.
- Khai báo kiểu trả về `: float` giúp PHP kiểm tra, vì `$price * (1 + TAX_RATE)` là phép nhân số nguyên với số thực nên kết quả là `float`.

Lưu ý

Không thể gán lại hằng sau khi khai báo. Trong Laravel, giá trị cấu hình thay đổi theo môi trường nằm trong `.env` và `config/*.php`, không phải hằng.

← Về đề bài 1.4

Đáp án 1.5 declare(strict_types=1) và TypeError

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function add(int $a, int $b): int
{
    return $a + $b;
}

echo add(2, 3), "\n";

try {
    echo add('5', 3), "\n";
} catch (TypeError $e) {
    echo get_class($e), "\n";
}
```

KẾT QUẢ KHI CHẠY

```
5
TypeError
```

Giải thích

- Dòng `declare(strict_types=1);` phải nằm **ngay đầu file**, sau `<?php`. Nó bật chế độ nghiêm ngặt về kiểu cho các lời gọi hàm **trong file đó**.
- Khi bật: truyền `'5'` (string) vào tham số `int` bị từ chối và ném `TypeError`. Khi **tắt**: PHP tự ép `'5'` thành `5` và `add('5', 3)` trả về `8` mà không báo gì. Đó là lý do bug dễ lọt qua.
- `TypeError` là một `Error` (không phải `Exception`) nên bạn bắt bằng `catch (TypeError $e)` hoặc `catch (Throwable $e)`.
- Ngoại lệ duy nhất vẫn cho phép: truyền `int` vào tham số `float` (PHP tự nâng kiểu).

> **Lưu ý:** Mọi file code mẫu trong tài liệu này bắt đầu bằng `declare(strict_types=1);` và Laravel cũng làm như vậy. Hãy tạo thói quen gõ nó ở dòng đầu.

← Về đề bài 1.5

ĐÁP ÁN — CHƯƠNG 2

Array cơ bản: indexed, associative, mảng lồng nhau

Đáp án 2.1 Indexed array

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$tags = ['php', 'laravel'];
$tags[] = 'composer';

echo $tags[0], "\n";
echo count($tags), "\n";
echo $tags[array_key_last($tags)], "\n";
echo in_array('laravel', $tags, true) ? 'yes' : 'no', "\n";
echo implode(', ', $tags), "\n";
```

KẾT QUẢ KHI CHẠY

```
php
3
composer
yes
php, laravel, composer
```

Giải thích

- `$tags[] = 'x'` thêm vào cuối mảng, tương đương `push` của JS. Chỉ số mới là chỉ số lớn nhất hiện có cộng 1.
- Chỉ số bắt đầu từ `0`. `count($tags)` trả số phần tử.
- Phần tử cuối: `$tags[count($tags) - 1]` cũng được, nhưng `array_key_last()` an toàn hơn vì nó đúng cả khi chỉ số không liên tục. (Còn có `end($tags)` nhưng hàm này **di chuyển con trỏ nội bộ** của mảng nên ít dùng.)
- `in_array($giá_trị, $mảng, true)`: tham số thứ 3 `true` bật so sánh `===`. Quên nó, PHP dùng `==` và có thể cho kết quả sai bất ngờ, ví dụ `in_array('abc', [0])` từng là `true` ở PHP 7.
- `implode(', ', $tags)` nối mảng thành chuỗi (giống `join` trong JS). Tham số đầu là dấu nối.

> **Lưu ý:** Lỗi hay gặp: viết `echo $tags;` để in cả mảng. PHP sẽ in chữ `Array` kèm cảnh báo. Muốn xem cấu trúc dùng `print_r($tags)` hoặc `var_dump($tags)`.

← Về đề bài 2.1

Đáp án 2.2 Associative array

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);
```

```php
$post = ['title' => 'Hello', 'views' => 10];

echo $post['title'], "\n";

$post['views'] = 11;
$post['author'] = 'Bao';
echo $post['author'], "\n";

unset($post['author']);
echo json_encode($post), "\n";
```

KẾT QUẢ KHI CHẠY

```
Hello
Bao
{"title":"Hello","views":11}
```

Giải thích

- Truy cập phần tử bằng `$post['title']` (dùng `[]`), **không** dùng dấu chấm như object JS.
- Gán vào key chưa tồn tại thì PHP **tạo mới** key đó, vì vậy thêm và sửa dùng cùng một cú pháp.
- `unset($post['author'])` xoá hẳn key khỏi mảng. Gán `null` thì key vẫn còn (xem bài 3.2).
- `json_encode` biến mảng kết hợp thành object JSON `{"title":"Hello","views":11}`. Nếu mảng là danh sách liên tục `[0,1,2...]` thì ra mảng JSON `[...]`.

Ghi nhớ

Associative array trong PHP thay cho cả `Object` lẫn `Map` của JS. Khi làm Laravel, request body, config và row từ database đều ở dạng này.

← Về đề bài 2.2

Đáp án 2.3 foreach với key và value

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$posts = [
    ['title' => 'A', 'views' => 5],
    ['title' => 'B', 'views' => 20],
    ['title' => 'C', 'views' => 11],
];

foreach ($posts as $i => $post) {
    printf("STT %d: %s (%d views)\n", $i + 1, $post['title'], $post['views']);
}
```

KẾT QUẢ KHI CHẠY

```
STT 1: A (5 views)
STT 2: B (20 views)
STT 3: C (11 views)
```

Giải thích

- `foreach ($mảng as $key => $value)` lấy cả chỉ số lẫn giá trị. Chỉ cần giá trị thì bỏ phần `$key =>`.
- Chỉ số của mảng danh sách bắt đầu từ `0`, nên số thứ tự hiển thị là `$i + 1`.
- `printf` in thẳng theo mẫu; `sprintf` thì trả về chuỗi thay vì in. Nhớ thêm `\n` ở cuối mẫu vì `printf` không tự xuống dòng.
- Biến `$post` trong vòng lặp là một **bản sao** của phần tử. Sửa nó không đổi mảng gốc (muốn sửa tại chỗ phải dùng `foreach ($posts as &$post)`, nhưng nên tránh).

Lỗi thường gặp

Dùng `%d` cho chuỗi hoặc `%s` cho số sai ý định: `%d` ép giá trị thành số nguyên nên `printf('%d', 'abc')` in `0`.

← Về đề bài 2.3

Đáp án 2.4 ## Mảng hai chiều

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$posts = [
    ['title' => 'A', 'views' => 5],
    ['title' => 'B', 'views' => 20],
    ['title' => 'C', 'views' => 11],
];

echo $posts[2]['title'], "\n";

$total = 0;
foreach ($posts as $post) {
    $total += $post['views'];
}
echo $total, "\n";

$best = $posts[0];
foreach ($posts as $post) {
    if ($post['views'] > $best['views']) {
        $best = $post;
    }
}
echo $best['title'], "\n";
```

KẾT QUẢ KHI CHẠY

```
C
36
B
```

Giải thích

- Mảng 2 chiều là mảng có phần tử cũng là mảng. Truy cập nối hai cặp ngoặc: `$posts[2]['title']`. Chỉ số `2` vì đếm từ 0.
- Tổng: khởi tạo `$total = 0` **trước** vòng lặp, rồi cộng dồn. Quên khởi tạo sẽ gây warning "Undefined variable".

- Tìm lớn nhất: giả sử phần tử đầu là lớn nhất (`$best = $posts[0]`), rồi duyệt và thay thế khi gặp phần tử lớn hơn. Đây là khuôn mẫu của mọi bài "tìm max/min" trong lập trình.

Mở rộng

Cách ngắn hơn khi đã học `array_column` : `array_sum(array_column($posts, 'views'))` , xem chương 4. Nhưng hãy chắc bạn viết được vòng lặp thuần trước.

← Về đề bài 2.4

ĐÁP ÁN — CHƯƠNG 3

Truy cập mảng an toàn: ??, isset, array_key_exists

Đáp án 3.1 Toán tử ?? trên mảng

KẾT QUẢ KHI CHẠY

```
Hello
N/A
N/A
B
không có
```

Giải thích

- `a ?? b` trả về `a` nếu `a` **tồn tại và khác** `null`, ngược lại trả về `b`. Nó **không sinh warning** khi key không tồn tại.
- Dòng 1: `title` có và khác null nên in `Hello`.
- Dòng 2: key `xxx` không có nên in `N/A`.
- Dòng 3: `views` có nhưng bằng `null`, `??` coi `null` như "không có" nên in `N/A`.
- Dòng 4: truy cập bình thường, in `B`.
- Dòng 5: `$posts[5]` không tồn tại nên cả chuỗi `$posts[5]['title']` rơi vào nhánh mặc định, in `không có`. Không có lỗi dù vừa đọc phần tử không tồn tại rồi đọc tiếp key bên trong, `??` bao hết.

> **Lưu ý:** Nếu bỏ `?? 'không có'` ở dòng cuối, PHP sẽ báo `Warning: Undefined array key 5` và `Trying to access array offset on value of type null`.

← Về đề bài 3.1

Đáp án 3.2 isset và array_key_exists

KẾT QUẢ KHI CHẠY

```
bool(true)
bool(false)
bool(true)
bool(false)
bool(false)
```

Giải thích

- `isset($post['title'])` là `true`: key có và khác `null`.
- `isset($post['views'])` là `false`: key **có** nhưng giá trị `null`, mà `isset` coi `null` là "chưa đặt".
- `array_key_exists('views', $post)` là `true`: nó chỉ kiểm tra **key có mặt hay không**, bỏ qua giá trị.
- Hai dòng cuối với key `xxx` không tồn tại: cả hai đều `false`.

Chọn cái nào

- Muốn biết "có giá trị dùng được không" (khác null): dùng `isset` hoặc `??`.
- Muốn biết "key này có trong mảng không, dù giá trị là null": dùng `array_key_exists`.

Ví dụ thực tế: form gửi `{"phone": null}` (người dùng chủ động xoá số điện thoại). `isset` sẽ coi như họ không gửi `phone`, còn `array_key_exists` biết họ có gửi. Laravel dùng sự khác biệt này khi xử lý PATCH.

← Về đề bài 3.2

Đáp án 3.3 Hàm đọc an toàn

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function safeTitle(array $posts, int $index): string
{
    return $posts[$index]['title'] ?? 'không có';
}

$posts = [['title' => 'A'], ['views' => 3], ['title' => 'C']];

foreach ([0, 1, 2, 9] as $i) {
    echo $i, ': ', safeTitle($posts, $i), "\n";
}
```

KẾT QUẢ KHI CHẠY

```
0: A
1: không có
2: C
9: không có
```

Giải thích

- Một biểu thức `??` xử lý được cả hai trường hợp: `$posts[1]` có nhưng **thiếu** key `title`, và `$posts[9]` **không có** phần tử.
- Vì hàm khai báo kiểu trả về `string`, giá trị trả về phải là chuỗi. Nếu `title` lại là `null` hoặc số, `??` cho ra giá trị khác chuỗi và dưới `strict_types` sẽ có `TypeError` hoặc lỗi kiểu, một lý do để luôn khai báo kiểu rõ.
- Vòng `foreach ([0, 1, 2, 9] as $i)` duyệt trực tiếp một mảng literal, tiện khi cần thử vài giá trị.

Mở rộng suy nghĩ

Trong Laravel, `data_get($array, 'a.b.c', 'mặc định')` làm đúng việc này cho đường dẫn lồng sâu, và `Arr::get()` tương tự. Hiểu `??` rồi thì các helper đó chỉ là bọc lại.

← Về đề bài 3.3

ĐÁP ÁN — CHƯƠNG 4

Hàm xử lý array: map, filter, reduce, column

Đáp án 4.1 array_map: biến đổi từng phần tử

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$numbers = [1, 2, 3, 4, 5, 6];

$doubled = array_map(fn (int $n): int => $n * 2, $numbers);
echo json_encode($doubled), "\n";
```

KẾT QUẢ KHI CHẠY

```
[2,4,6,8,10,12]
```

Giải thích

- `array_map(callback, mảng)` : **callback đứng trước, mảng đứng sau**. Đây là chỗ khác `array_filter` (xem 4.2) và hay nhầm.
- Callback nhận từng phần tử và trả về giá trị mới. Mảng kết quả có **cùng số phần tử** với mảng gốc.
- Mảng gốc `$numbers` không bị thay đổi: PHP hàm kiểu này trả về mảng mới.
- `fn (int $n): int => $n * 2` là arrow function: khai báo kiểu tham số và kiểu trả về giống hàm thường.

So sánh với JS

`numbers.map(n => n * 2)` trong JS tương ứng `array_map(fn ($n) => $n * 2, $numbers)`. Thứ tự tham số đảo nhau.

← Về đề bài 4.1

Đáp án 4.2 array_filter giữ nguyên key

KẾT QUẢ KHI CHẠY

```
{"1":2,"3":4,"5":6}
bool(false)
```

Giải thích

- `array_filter` giữ lại phần tử mà callback trả `true`, nhưng **giữ nguyên key gốc**. Số chẵn `2, 4, 6` nằm ở các chỉ số `1, 3, 5`, nên mảng kết quả là `[1 => 2, 3 => 4, 5 => 6]`.
- `json_encode` chỉ xuất mảng JSON `[...]` khi key là `0, 1, 2, ...` liên tục. Không liên tục thì xuất **object** `{"1":2,"3":4,"5":6}`.
- `array_is_list` (có từ PHP 8.1) kiểm tra đúng điều đó: key có bắt đầu từ 0 và liên tục không. Ở đây là `false`.

> **Lưu ý:** Đây là bug kinh điển: API của bạn tưởng trả mảng nhưng client nhận object. Cách sửa ở bài 4.4.

← Về đề bài 4.2

Đáp án 4.3 array_reduce: gộp thành một giá trị

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$numbers = [1, 2, 3, 4, 5, 6];

$sum = array_reduce($numbers, fn (int $carry, int $n): int => $carry + $n, 0);
$product = array_reduce($numbers, fn (int $carry, int $n): int => $carry * $n, 1);

echo $sum, "\n";
echo $product, "\n";
```

KẾT QUẢ KHI CHẠY

```
21
720
```

Giải thích

- `array_reduce(mảng, callback, giá_trị_khởi_đầu)`. Callback nhận `$carry` (kết quả tích luỹ đến hiện tại) và phần tử hiện tại, rồi **trả về `$carry` mới**.
- Khởi đầu với tổng là `0` (phần tử trung hòa của phép cộng). Khởi đầu với tích là `1` (phần tử trung hòa của phép nhân). Khởi đầu `0` cho tích sẽ ra `0`.
- Quá trình tính tổng: `0+1=1`, `1+2=3`, `3+3=6`, `6+4=10`, `10+5=15`, `15+6=21`.
- Quá trình tính tích: `1*1`, `1*2`, `2*3`, `6*4`, `24*5`, `120*6 = 720`.

Khi nào dùng

Bất cứ khi nào cần "nén" cả mảng thành một giá trị: tổng, tích, đếm có điều kiện, gộp thành mảng khác (ví dụ nhóm). Trong bài closure, bạn sẽ dùng `array_reduce` để viết `pipeline`.

← Về đề bài 4.3

Đáp án 4.4 Sửa lỗi JSON thành object: array_values

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$numbers = [1, 2, 3, 4, 5, 6];

$evens = array_filter($numbers, fn (int $n): bool => $n % 2 === 0);
$scaled = array_map(fn (int $n): int => $n * 10, $evens);
$list = array_values($scaled);

echo json_encode($scaled), "\n";
echo json_encode($list), "\n";
var_dump(array_is_list($scaled));
var_dump(array_is_list($list));
```

KẾT QUẢ KHI CHẠY

```
{"1":20,"3":40,"5":60}
[20,40,60]
bool(false)
bool(true)
```

Giải thích

- `array_map` **giữ key** của mảng đầu vào khi nhận đúng một mảng. Vì `$evens` có key `1, 3, 5`, nên `$scaled` cũng có key `1, 3, 5` và `json_encode` ra object.
- `array_values($mảng)` bỏ key cũ, đánh lại `0, 1, 2, ...`. Sau đó `json_encode` cho mảng JSON `[20,40,60]`.
- Quy tắc thực hành: **sau `array_filter`, nếu kết quả sẽ trả ra JSON hoặc cần truy cập `[0]`, hãy bọc `array_values()`**.

Lưu ý thêm

Truy cập `$evens[0]` sẽ không có gì (key 0 đã bị lọc mất), còn `$evens[1]` lại là `2`. Đây là lý do `$x[0] ?? 'không có'` ở file thực hành của bạn cho ra "Không có key 0".

> **Gợi ý:** Laravel Collection có sẵn `->values()` làm đúng việc này và `->filter()->values()` là cặp bạn sẽ viết rất nhiều.

← Về đề bài 4.4

Đáp án 4.5 array_column

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$posts = [
    ['title' => 'A', 'views' => 5],
    ['title' => 'B', 'views' => 20],
    ['title' => 'C', 'views' => 11],
];

echo json_encode(array_column($posts, 'title')), "\n";
echo json_encode(array_column($posts, 'views', 'title')), "\n";

$popular = array_filter($posts, fn (array $p): bool => $p['views'] > 10);
echo json_encode(array_column($popular, 'title')), "\n";
```

KẾT QUẢ KHI CHẠY

```
["A","B","C"]
{"A":5,"B":20,"C":11}
["B","C"]
```

Giải thích

- `array_column($mảng_2_chiều, 'cột')` lấy một "cột" từ mảng các mảng, giống `posts.map(p => p.title)` trong JS.

- Tham số thứ 3 `array_column($posts, 'views', 'title')` cho biết dùng cột `title` làm **key** của kết quả: `{"A":5,"B":20,"C":11}`.
- Dòng 3: lọc trước bằng `array_filter`, rồi lấy cột title. May mắn là `array_column` **đánh lại chỉ số từ 0**, nên kết quả `["B","C"]` là mảng JSON thật, không cần `array_values`. Hãy nhớ đây là điểm khác với `array_map`.

Bẫy

Nếu một phần tử thiếu key cột, `array_column` **bỏ qua** phần tử đó thay vì báo lỗi. Có lợi nhưng cũng dễ che giấu dữ liệu sai.

← Về đề bài 4.5

Đáp án 4.6 Chuỗi hàm: lẻ, bình phương, tổng

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$numbers = [1, 2, 3, 4, 5, 6];

$odds = array_filter($numbers, fn (int $n): bool => $n % 2 !== 0);
$squares = array_map(fn (int $n): int => $n * $n, $odds);
$result = array_reduce($squares, fn (int $carry, int $n): int => $carry + $n, 0);

echo $result, "\n";
```

KẾT QUẢ KHI CHẠY

```
35
```

Giải thích

- Bước 1: `array_filter` giữ `1, 3, 5` (key `0, 2, 4`).
- Bước 2: `array_map` bình phương thành `1, 9, 25`, vẫn giữ key. Việc key khuyết **không ảnh hưởng** kết quả vì `array_reduce` chỉ duyệt giá trị.
- Bước 3: `array_reduce` cộng dồn ra `35`.
- `$n % 2 !== 0` là phép chia lấy dư. Số lẻ chia 2 dư 1. Nếu viết `$n % 2 === 1` cũng đúng cho số dương, nhưng số âm lẻ như `-3 % 2` ra `-1` nên dùng `!== 0` an toàn hơn.

Mở rộng

Mỗi bước tạo một mảng trung gian. Khi mảng cực lớn thì tốn bộ nhớ; Laravel Collection có `lazy()` để xử lý dần. Với dữ liệu nhỏ thì cách này hoàn toàn ổn và dễ đọc.

← Về đề bài 4.6

Đáp án 4.7 Lọc rồi tính tổng cột

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);
```

```php
$posts = [
    ['title' => 'A', 'views' => 5],
    ['title' => 'B', 'views' => 20],
    ['title' => 'C', 'views' => 11],
    ['title' => 'D', 'views' => 3],
    ['title' => 'E', 'views' => 40],
];

$popular = array_filter($posts, fn (array $p): bool => $p['views'] > 10);
$total = array_sum(array_column($popular, 'views'));

echo $total, "\n";
```

KẾT QUẢ KHI CHẠY

```
71
```

Giải thích

- `array_filter` giữ B, C, E.
- `array_column($popular, 'views')` lấy ra `[20, 11, 40]`.
- `array_sum` cộng thành `71`. `array_sum([])` của mảng rỗng trả `0`, nên không cần xử lý riêng trường hợp không có bài nào đạt.

Cách viết khác

Có thể dùng `array_reduce` một lần: `array_reduce($posts, fn ($c, $p) => $p['views'] > 10 ? $c + $p['views'] : $c, 0)`. Cách kết hợp `filter + column + sum` dễ đọc hơn, nên ưu tiên khi hiệu năng không phải vấn đề.

← Về đề bài 4.7

Đáp án 4.8 Top 3 bài nhiều views

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$posts = [
    ['title' => 'A', 'views' => 5],
    ['title' => 'B', 'views' => 20],
    ['title' => 'C', 'views' => 11],
    ['title' => 'D', 'views' => 3],
    ['title' => 'E', 'views' => 40],
];

$sorted = $posts;
usort($sorted, fn (array $a, array $b): int => $b['views'] <=> $a['views']);

$top3 = array_slice($sorted, 0, 3);
echo implode(', ', array_column($top3, 'title')), "\n";
echo implode(', ', array_column($posts, 'title')), "\n";
```

KẾT QUẢ KHI CHẠY

```
E, B, C
A, B, C, D, E
```

Giải thích

- `usort($mảng, callback)` sắp xếp **tại chỗ** (sửa trực tiếp mảng, trả về `true`). Vì vậy ta chép `$posts` sang `$sorted` trước. Dòng in cuối chứng minh `$posts` vẫn theo thứ tự cũ `A, B, C, D, E`.
- Callback trả số âm / 0 / dương. `$b['views'] <=> $a['views']` (đảo b và a) cho **giảm dần**.
- `array_slice($mảng, offset, length)` lấy `length` phần tử bắt đầu từ `offset`. Với offset 0 và length 3 là 3 phần tử đầu. Nó đánh lại key số từ 0.
- Cuối cùng `array_column` lấy title, `implode` nối.

Chú ý

`array_slice` **không sửa** mảng gốc. Còn `array_splice` thì sửa. Hai hàm tên gần giống nhau nhưng hành vi khác nhau.

← Về đề bài 4.8

Đáp án 4.9 Bộ hàm tiện ích hay dùng

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$nums = [3, 1, 4, 1, 5, 9, 2, 6, 5];

echo 'sum: ', array_sum($nums), "\n";
echo 'max: ', max($nums), "\n";
echo 'min: ', min($nums), "\n";
echo 'count: ', count($nums), "\n";
echo 'unique: ', json_encode(array_values(array_unique($nums))), "\n";
echo 'has4: ', in_array(4, $nums, true) ? 'yes' : 'no', "\n";
echo 'index9: ', array_search(9, $nums, true), "\n";
echo 'reverse: ', json_encode(array_reverse($nums)), "\n";
echo 'first3: ', json_encode(array_slice($nums, 0, 3)), "\n";
echo 'merged: ', json_encode(array_merge($nums, [7, 8])), "\n";
```

KẾT QUẢ KHI CHẠY

```
sum: 36
max: 9
min: 1
count: 9
unique: [3,1,4,5,9,2,6]
has4: yes
index9: 5
reverse: [5,6,2,9,5,1,4,1,3]
first3: [3,1,4]
merged: [3,1,4,1,5,9,2,6,5,7,8]
```

Giải thích

- `array_sum`, `max`, `min`, `count` đều nhận cả mảng.

- `array_unique` loại trùng nhưng **giữ key của phần tử đầu tiên**, nên có thể khuyết chỉ số: phải bọc `array_values` để JSON ra mảng thật. (Cùng lỗi đã gặp ở 4.4.)
- `array_search($giá_trị, $mảng, true)` trả về **key** của phần tử khớp đầu tiên, hoặc `false` nếu không có. Nhớ dùng `true` (strict) và so sánh kết quả bằng `=== false`, vì vị trí hợp lệ `0` cũng là falsy.
- `array_reverse` trả mảng đảo ngược. Với mảng danh sách, key được đánh lại.
- `array_merge` nối hai mảng. Với mảng có **key chuỗi** trùng nhau, giá trị sau ghi đè giá trị trước; với key số thì nối tiếp. Toán tử `+` giữ giá trị của mảng bên trái, khác `array_merge`.

Bảng nhanh

Xem Phụ lục cuối tài liệu để tra các hàm array thường dùng.

← Về đề bài 4.9

ĐÁP ÁN — CHƯƠNG 5

Sắp xếp: <=>, usort và họ sort

Đáp án 5.1 Toán tử spaceship <=>

KẾT QUẢ KHI CHẠY

```
int(-1)
int(0)
int(1)
int(-1)
int(1)
int(-1)
```

Giải thích

- `a <=> b` trả `-1` nếu `a < b`, `0` nếu bằng, `1` nếu `a > b`.
- Với chuỗi, so sánh theo thứ tự ký tự: `'a'` đứng trước `'b'` nên `'a' <=> 'b'` là `-1`.
- Với mảng, so sánh **từng phần tử từ trái sang phải** và dừng ở chỗ khác nhau đầu tiên: `[1, 2]` và `[1, 3]` giống ở phần tử đầu, phần tử thứ hai `2 < 3` nên kết quả `-1`. Tính chất này dùng để sort theo nhiều tiêu chí (bài 5.3).

Vì sao quan trọng

`usort` cần một hàm trả số âm / 0 / dương, và `<=>` trả đúng ba giá trị đó. Trước PHP 7 người ta phải viết `if ($a < $b) return -1; ...` dài dòng.

← Về đề bài 5.1

Đáp án 5.2 usort giảm dần mà không đụng mảng gốc

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$posts = [
    ['title' => 'A', 'views' => 5],
    ['title' => 'B', 'views' => 20],
    ['title' => 'C', 'views' => 11],
    ['title' => 'D', 'views' => 3],
    ['title' => 'E', 'views' => 40],
];

$sorted = $posts;
usort($sorted, fn (array $a, array $b): int => $b['views'] <=> $a['views']);

echo implode(', ', array_column($sorted, 'title')), "\n";
echo implode(', ', array_column($posts, 'title')), "\n";
```

KẾT QUẢ KHI CHẠY

```
E, B, C, A, D
A, B, C, D, E
```

Giải thích

- `usort` thay đổi trực tiếp mảng truyền vào (tham số tham chiếu) và trả về `true`. Vì vậy phải copy `$posts` sang `$sorted` **trước**: PHP copy mảng theo giá trị, nên hai biến độc lập.
- `$b['views'] <=> $a['views']` đảo vai trò hai phần tử để có giảm dần. Muốn tăng dần thì `$a['views'] <=> $b['views']`.
- Kết quả `E, B, C, A, D` ứng với views `40, 20, 11, 5, 3`.

> **Lưu ý:** Lỗi hay gặp: viết `$sorted = usort($posts, ...)`. Khi đó `$sorted` chỉ là `true`, không phải mảng đã sắp.

Các hàm sort chỉ dùng khi sắp xếp giá trị đơn giản

`sort`, `rsort` không cần callback. `usort` dùng khi cần so sánh tuỳ chỉnh (theo một field của phần tử).

← Về đề bài 5.2

Đáp án 5.3 Sort nhiều tiêu chí

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$posts = [
    ['title' => 'Vue', 'views' => 50],
    ['title' => 'Laravel', 'views' => 100],
    ['title' => 'Docker', 'views' => 100],
    ['title' => 'PHP', 'views' => 50],
    ['title' => 'Redis', 'views' => 100],
];

usort(
    $posts,
    fn (array $a, array $b): int => ($b['views'] <=> $a['views']) ?: ($a['title'] <=>
$b['title'])
);

foreach ($posts as $post) {
    echo $post['title'], ' - ', $post['views'], "\n";
}
```

KẾT QUẢ KHI CHẠY

```
Docker - 100
Laravel - 100
Redis - 100
PHP - 50
Vue - 50
```

Giải thích

- Biểu thức `X ?: Y` (toán tử Elvis) trả `X` nếu `X` truthy, ngược lại trả `Y`.
- `($b['views'] <=> $a['views'])` khác 0 khi views khác nhau, kết quả đó quyết định thứ tự. Khi views bằng nhau, nó là `0` (falsy) nên `?:` chuyển sang vế phải `($a['title'] <=> $b['title'])`, sắp title tăng dần.

- Nhóm 100 views: Docker, Laravel, Redis (A-Z). Nhóm 50 views: PHP, Vue.

Cách viết khác: so sánh mảng

```
fn ($a, $b) => [$b['views'], $a['title']] <=> [$a['views'], $b['title']]
```

Mảng so sánh từng phần tử từ trái sang phải. Để giảm dần theo views, ta đặt views của `$b` bên **trái** và views của `$a` bên **phải**. Ngược lại với title (tăng dần). Gọn hơn nhưng khó đọc hơn cho người mới.

> **Lưu ý:** Không thể nối hai kết quả bằng `+`: `(-1) + (1) = 0` làm mất ý nghĩa ưu tiên.

Nếu muốn bỏ qua hoa thường

Thay `$a['title'] <=> $b['title']` bằng `strcasecmp($a['title'], $b['title'])`. Mặc định `<=>` xếp chữ hoa trước chữ thường (theo mã ký tự).

← Về đề bài 5.3

Đáp án 5.4 Tự viết bubble sort cho mảng bài viết

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$posts = [
    ['title' => 'A', 'views' => 5],
    ['title' => 'B', 'views' => 20],
    ['title' => 'C', 'views' => 11],
    ['title' => 'D', 'views' => 3],
    ['title' => 'E', 'views' => 40],
];

function handleSort(array $arr): array
{
    $n = count($arr);

    for ($i = 0; $i < $n - 1; $i++) {
        for ($j = 0; $j < $n - 1 - $i; $j++) {
            if ($arr[$j]['views'] < $arr[$j + 1]['views']) {
                $temp = $arr[$j];
                $arr[$j] = $arr[$j + 1];
                $arr[$j + 1] = $temp;
            }
        }
    }

    return $arr;
}

echo implode(', ', array_column(handleSort($posts), 'title')), "\n";
echo implode(', ', array_column($posts, 'title')), "\n";
```

KẾT QUẢ KHI CHẠY

```
E, B, C, A, D
A, B, C, D, E
```

Giải thích thuật toán

- Vòng ngoài `$i` đếm số lượt. Sau lượt đầu, phần tử nhỏ nhất (với sắp giảm dần) "chìm" về cuối; sau lượt hai, phần tử nhỏ thứ hai chìm về vị trí kế cuối; v.v.
- Vòng trong `$j` so sánh cặp `($j, $j+1)`. Giới hạn `$n - 1 - $i` vì `$i` phần tử cuối đã vào đúng chỗ, không cần so sánh lại.
- Đổi chỗ bằng biến tạm `$temp`: gán `$arr[$j]` vào `$temp`, rồi `$arr[$j] = $arr[$j + 1]`, rồi `$arr[$j + 1] = $temp`.
- Điều kiện `<` cho **giảm dần** (phần tử trước nhỏ hơn phần tử sau thì đổi). Đổi thành `>` được tăng dần.

Về tham số mảng

`$arr` trong hàm là **bản sao** (PHP truyền mảng theo giá trị, có cơ chế copy-on-write), nên `$posts` gốc không đổi. Dòng in cuối chứng minh.

Độ phức tạp

Hai vòng lồng nhau nên thời gian cỡ n^2 phép so sánh. Dùng để học hiểu; thực tế dùng `usort` (nhanh hơn nhiều, độ phức tạp `n log n`).

← Về đề bài 5.4

Đáp án 5.5 Họ hàm sort: giữ key hay không

KẾT QUẢ KHI CHẠY

```
asort:  {"an":70,"chi":80,"binh":90}
arsort: {"binh":90,"chi":80,"an":70}
ksort:  {"an":70,"binh":90,"chi":80}
krsort: {"chi":80,"binh":90,"an":70}
sort:   [70,80,90]
rsort:  [90,80,70]
values: [70,80,90]
```

Bảng ghi nhớ

Hàm	Sắp theo	Hướng	Giữ key?
`sort`	giá trị	tăng	Không (đánh lại 0,1,2)
`rsort`	giá trị	giảm	Không
`asort`	giá trị	tăng	Có
`arsort`	giá trị	giảm	Có
`ksort`	key	tăng	Có
`krsort`	key	giảm	Có
`usort`	giá trị, tuỳ chỉnh	tuỳ callback	Không
`uasort`	giá trị, tuỳ chỉnh	tuỳ callback	Có
`uksort`	key, tuỳ chỉnh	tuỳ callback	Có

Mẹo nhớ

- Chữ **a** (associative) trong `asort` / `arsort` = giữ key.

- Chữ **k** = sắp theo key.
- Chữ **r** = reverse (giảm dần).
- Chữ **u** = user callback.

Giải thích output

- `asort` giữ tên và sắp theo điểm tăng dần: `an 70, chi 80, binh 90`.
- `arsort` giảm dần: `binh 90, chi 80, an 70`. Đây là hàm bạn dùng ở bài tổng kết chi tiêu để tìm danh mục lớn nhất.
- `ksort` sắp theo tên: `an, binh, chi`; `krsort` ngược lại.
- `sort` và `rsort` **bỏ tên**, chỉ còn danh sách điểm `[70,80,90]` và `[90,80,70]`. Đây là lý do không dùng `sort` cho mảng kết hợp khi còn cần tên.

> **Lưu ý:** Mọi hàm sort ở trên đều sửa mảng tại chỗ và trả `true`, không trả mảng mới.

← Về đề bài 5.5

ĐÁP ÁN — CHƯƠNG 6

Toán tử PHP 8: ??, ?:, ??=, ?->, match

Đáp án 6.1 ?? khác ?:

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$settings = ['theme' => 'dark', 'perPage' => 0, 'lang' => null];

echo $settings['theme'] ?? 'light', "\n";
echo $settings['perPage'] ?? 15, "\n";
echo $settings['perPage'] ?: 15, "\n";
echo $settings['lang'] ?? 'vi', "\n";
echo $settings['timezone'] ?? 'UTC', "\n";
echo isset($settings['perPage']) ? 'có' : 'không', "\n";
```

KẾT QUẢ KHI CHẠY

```
dark
0
15
vi
UTC
có
```

Giải thích

- `??` chỉ xét **null / không tồn tại**. `perPage = 0` là giá trị hợp lệ (không phải null), nên dòng 2 in `0`.
- `?:` xét **falsy**: `0`, `''`, `'0'`, `[]`, `false`, `null` đều là falsy. Dòng 3 thấy `0` là falsy nên lấy `15`.
- `lang` là `null` nên `??` lấy `vi`. `timezone` không tồn tại nên lấy `UTC`, không có warning.
- `isset` dòng 6 là `true` vì `0` không phải null.

Khi nào dùng cái nào

- Muốn mặc định **chỉ khi thiếu dữ liệu**: dùng `??`. Số lượng `0`, chuỗi rỗng, `false` là giá trị hợp lệ, đừng bị thay.
- Muốn mặc định **cho mọi giá trị rỗng**: dùng `?:`. Ví dụ tên hiển thị: `$name ?: 'Ẩn danh'` thay luôn chuỗi rỗng.

> **Lưu ý:** Cạm bẫy của `??`: gõ sai tên biến (`$setting` thay vì `$settings`) thì `??` im lặng trả về mặc định, không báo lỗi. Hãy kiểm tra kỹ tên biến.

← Về đề bài 6.1

Đáp án 6.2 ??= gán nếu chưa có

CODE ĐÁP ÁN

```php
<?php
```

```php
declare(strict_types=1);

$config = ['limit' => 0];

$config['limit'] ??= 15;
$config['sort'] ??= 'desc';
echo json_encode($config), "\n";

$count = [];
foreach (['a', 'b', 'a', 'c', 'a'] as $letter) {
    $count[$letter] ??= 0;
    $count[$letter]++;
}
echo json_encode($count), "\n";
```

KẾT QUẢ KHI CHẠY

```
{"limit":0,"sort":"desc"}
{"a":3,"b":1,"c":1}
```

Giải thích

- `$x ??= v` nghĩa là `$x = $x ?? v;`: chỉ gán khi `$x` chưa tồn tại hoặc là `null`.
- `limit` đã có giá trị `0` (khác null), nên **giữ nguyên** `0`. `sort` chưa có nên được gán `'desc'`.
- Bộ đếm: lần đầu gặp chữ cái nào đó, `$count[$letter]` chưa tồn tại, `??= 0` khởi tạo về 0, rồi `++` tăng lên 1. Các lần sau key đã tồn tại nên `??= 0` không làm gì.
- Nếu bỏ dòng `??= 0`, `$count[$letter]++` trên key chưa tồn tại sẽ gây warning "Undefined array key".

Mẫu này (`??= 0` rồi `++` hoặc `+=`) dùng cực nhiều khi thống kê: đếm, cộng dồn theo nhóm. Bạn đã gặp nó ở bài báo cáo chi tiêu.

← Về đề bài 6.2

Đáp án 6.3 Nullsafe ?->

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

class Address
{
    public function __construct(public string $city) {}
}

class Profile
{
    public function __construct(public ?Address $address = null) {}
}

class User
{
    public function __construct(public ?Profile $profile = null) {}
}

$user1 = new User(new Profile(new Address('Hà Nội')));
$user2 = new User(new Profile());
```

```php
$user3 = new User();

echo $user1->profile?->address?->city ?? 'Chưa rõ', "\n";
echo $user2->profile?->address?->city ?? 'Chưa rõ', "\n";
echo $user3->profile?->address?->city ?? 'Chưa rõ', "\n";
```

KẾT QUẢ KHI CHẠY

```
Hà Nội
Chưa rõ
Chưa rõ
```

Giải thích

- Dấu `?` trước tên kiểu (`?Address`) nghĩa là kiểu đó **hoặc** `null` . Kèm `= null` để tham số trở thành tuỳ chọn. Thiếu phần này, `new User()` sẽ lỗi `ArgumentCountError` .
- `$a?->b` : nếu `$a` là `null` , cả biểu thức trả `null` ngay mà không báo lỗi. Nếu `$a` có giá trị, hoạt động như `->` .
- Nullsafe **cắt ngắn** cả chuỗi: với `$user3` , `profile` là null nên `?->address` và `?->city` đều không được thực thi.
- `?? 'Chưa rõ'` biến `null` cuối cùng thành mặc định.
- Dùng `->` thường ở chỗ chắc chắn không null (`$user1` luôn có), `?->` ở những mắt xích có thể null.

> **Lưu ý:** `?->` chỉ dùng cho **object**. Với **array** vẫn dùng `??` (bài 3.1).

Liên hệ Laravel

Quan hệ Eloquent như `$post->author?->profile?->city` rất hay gặp khi quan hệ có thể trống.

← Về đề bài 6.3

Đáp án 6.4 match thay switch

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function httpLabel(int $code): string
{
    return match ($code) {
        200, 201 => 'Thành công',
        301, 302 => 'Chuyển hướng',
        404 => 'Không tìm thấy',
        500 => 'Lỗi server',
    };
}

echo httpLabel(200), "\n";
echo httpLabel(404), "\n";
echo httpLabel(301), "\n";

try {
    echo httpLabel(418), "\n";
} catch (UnhandledMatchError $e) {
```

```
    echo 'Không hỗ trợ mã này', "\n";
}
```

KẾT QUẢ KHI CHẠY

```
Thành công
Không tìm thấy
Chuyển hướng
Không hỗ trợ mã này
```

Giải thích

- `match` là một **biểu thức** trả về giá trị, nên `return match (...) {...}` gọn hơn nhiều so với `switch` rồi `return` trong từng `case`.
- Các nhánh dùng `=>`. Nhiều giá trị chung một kết quả thì cách nhau bằng dấu phẩy: `200, 201 => '...'`.
- Không có `break`, không "rơi" xuống nhánh dưới như `switch`.
- Nếu **không nhánh nào khớp và không có `default`**, PHP ném `UnhandledMatchError`. Đây là tính năng tốt: bạn biết ngay có giá trị chưa được xử lý thay vì âm thầm bỏ qua.
- Khi muốn "mặc định", thêm `default => '...'` vào cuối.

So sánh với switch

Xem bài 6.6 để hiểu khác biệt về cách so sánh.

← Về đề bài 6.4

Đáp án 6.5 match(true) cho điều kiện khoảng

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function shippingFee(int $total): int
{
    return match (true) {
        $total >= 500000 => 0,
        $total >= 200000 => 15000,
        default => 30000,
    };
}

foreach ([100000, 200000, 450000, 500000] as $total) {
    echo $total, ' => ', shippingFee($total), "\n";
}
```

KẾT QUẢ KHI CHẠY

```
100000 => 30000
200000 => 15000
450000 => 15000
500000 => 0
```

Giải thích

- `match (true)` so sánh `true === (điều kiện của từng nhánh)`. PHP duyệt từ trên xuống và chọn nhánh **đầu tiên** có điều kiện đúng.
- Vì vậy **thứ tự nhánh rất quan trọng**: nhánh `>= 500000` phải đứng trước nhánh `>= 200000`. Nếu đảo, đơn 600000 sẽ khớp nhánh `>= 200000` trước và trả phí 15000 là sai.
- Biên: đề nói "từ 500000 trở lên" là `>=` chứ không phải `>`. Đơn đúng 500000 phải được miễn phí. Luôn kiểm tra các giá trị biên (200000 và 500000).
- `default` ở cuối bao quát phần còn lại (dưới 200000), không cần viết lại điều kiện `< 200000`.

> **Lưu ý:** Sai lầm thường gặp: viết `match ($total) { $total > 500000 => ... }`. Khi đó PHP so sánh `$total === (true/false)`, không bao giờ khớp và ném `UnhandledMatchError`. Với điều kiện khoảng phải dùng `match (true)`.

← Về đề bài 6.5

Đáp án 6.6 match so sánh nghiêm ngặt, switch so sánh lỏng

KẾT QUẢ KHI CHẠY

```
switch khớp
match không khớp
```

Giải thích

- `switch` dùng so sánh **lỏng** (`==`): chuỗi `'1'` bằng số `1`, nên `case 1` khớp.
- `match` dùng so sánh **nghiêm ngặt** (`===`): chuỗi `'1'` khác số `1` về kiểu, nên rơi vào `default`.
- Đây là lý do chính để ưu tiên `match`: tránh các lỗi do ép kiểu ngầm, giống nguyên tắc "luôn dùng `===`".

Tóm tắt khác biệt switch và match

	switch	match
So sánh	`==` (lỏng)	`===` (nghiêm ngặt)
Trả về giá trị	Không	Có (là biểu thức)
Cần `break`	Có	Không
Không khớp, không default	Bỏ qua im lặng	Ném `UnhandledMatchError`

← Về đề bài 6.6

ĐÁP ÁN — CHƯƠNG 7

Class và OOP cơ bản

Đáp án 7.1 Class đầu tiên: Post

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

class Post
{
    public function __construct(public string $title, public int $views = 0) {}

    public function addView(): void
    {
        $this->views++;
    }

    public function summary(): string
    {
        return "{$this->title} ({$this->views} views)";
    }
}

$a = new Post('Hello');
$b = new Post('PHP', 10);

$a->addView();
$a->addView();
$b->addView();

echo $a->summary(), "\n";
echo $b->summary(), "\n";
echo $a->title, "\n";
```

KẾT QUẢ KHI CHẠY

```
Hello (2 views)
PHP (11 views)
Hello
```

Giải thích

- **Constructor property promotion** (PHP 8): viết `public string $title` ngay trong tham số constructor vừa khai báo thuộc tính, vừa tự gán `$this->title = $title`. Thay cho 3 chỗ phải viết trước đây (khai báo, tham số, gán).
- Tham số `$views = 0` có giá trị mặc định nên `new Post('Hello')` không cần truyền views.
- Trong method, `$this` là chính object đang được gọi. `$this->views++` tăng thuộc tính của object đó.
- Chuỗi `"{$this->title} ({$this->views} views)"` dùng `{}` để nội suy biểu thức có `->`. Hai object `$a` và `$b` có dữ liệu **độc lập** nhau.
- Kiểu trả về `: void` nghĩa là hàm không trả giá trị.

So với JS

`this.title` trong JS là `$this->title` trong PHP. Gọi method là `$a->addView()`, không phải `$a.addView()`.

← Về đề bài 7.1

Đáp án 7.2 private và encapsulation

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

class Product
{
    public function __construct(public string $name, private int $price) {}

    public function priceAfterDiscount(int $percent): float
    {
        return $this->price - (($this->price * $percent) / 100);
    }

    public function label(): string
    {
        return "{$this->name} - {$this->price}";
    }
}

$p = new Product('Áo', 200000);
echo $p->priceAfterDiscount(10), "\n";
echo $p->label(), "\n";

try {
    echo $p->price;
} catch (Error $e) {
    echo $e->getMessage(), "\n";
}
```

KẾT QUẢ KHI CHẠY

```
180000
Áo - 200000
Cannot access private property Product::$price
```

Giải thích

- `public`: truy cập được từ mọi nơi. `private`: chỉ truy cập được **bên trong class đó**. (Còn `protected`: class đó và class con.)
- `price` là `private` nên bên ngoài đọc `$p->price` sẽ bị chặn bằng `Error` ("Cannot access private property"). Bên trong class, `$this->price` dùng thoải mái.
- Muốn cho bên ngoài đọc giá thì viết một method `getPrice()`, hoặc cho phép đọc có kiểm soát. Ý nghĩa: bên ngoài không thể đặt giá thành số âm nếu ta không cung cấp cách đó. Đó là **đóng gói (encapsulation)**.
- `priceAfterDiscount` trả `float` vì phép chia `/` trong PHP trả `float` (kết quả `180000`, in ra không có phần thập phân vì là số tròn).

- `Error` là lớp cha của các lỗi nghiêm trọng của engine (TypeError, ...). `Exception` là cho lỗi nghiệp vụ. Cả hai đều implement `Throwable`.

Lưu ý

Trong thực tế, hãy giữ thuộc tính `private` / `protected` mặc định, chỉ mở `public` khi có lý do, và Eloquent Model là một ngoại lệ vì dùng "magic" để truy cập `$user->name`.

← Về đề bài 7.2

Đáp án 7.3 readonly, final và immutable object

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

final class Money
{
    public function __construct(
        public readonly int $amount,
        public readonly string $currency = 'VND',
    ) {}

    public function add(Money $other): Money
    {
        return new Money($this->amount + $other->amount, $this->currency);
    }
}

$a = new Money(100);
$b = $a->add(new Money(50));

echo $a->amount, "\n";
echo $b->amount, "\n";

try {
    $a->amount = 999;
} catch (Error $e) {
    echo $e->getMessage(), "\n";
}
```

KẾT QUẢ KHI CHẠY

```
100
150
Cannot modify readonly property Money::$amount
```

Giải thích

- `readonly` (PHP 8.1): thuộc tính chỉ gán **một lần** (thường trong constructor) rồi không đổi được nữa. Gán lại sẽ ném `Error`.
- Vì không sửa được, `add()` không thể làm `$this->amount += ...`. Thay vào đó nó **tạo object mới**. Cách thiết kế này gọi là **immutable** (bất biến). `$a` vẫn là 100, `$b` mới là 150.
- Lợi ích của bất biến: khi truyền object đi nhiều nơi, không lo nơi khác âm thầm sửa làm hỏng dữ liệu của bạn.
- `final class` không cho phép class khác kế thừa, đảm bảo hành vi không bị ghi đè.

- Kiểu `Money` ở tham số `add(Money $other)` là một **class type**: chỉ nhận object của class đó.

Dùng ở đâu

Value Object (tiền, email, toạ độ) và DTO trong Laravel (Part 21 của tài liệu lộ trình) thường dùng `readonly` và `final`.

← Về đề bài 7.3

Đáp án 7.4 So sánh object: ===, == và cùng instance

KẾT QUẢ KHI CHẠY

```
bool(true)
bool(false)
bool(true)
bool(false)
```

Giải thích

- Biến object trong PHP chứa **tham chiếu (handle)** tới object, không phải bản sao. `$y = $x` làm `$y` trỏ **cùng một object** với `$x`.
- `===` giữa hai object nghĩa là **cùng một instance**. `$x === $y` là `true`; `$x === $z` là `false` vì `$z` là object khác dù cùng dữ liệu.
- `==` giữa hai object nghĩa là **cùng class và các thuộc tính bằng nhau** (so sánh lỏng từng thuộc tính). `$x == $z` là `true` vì cùng class và `amount=1, currency='VND'`.
- `$x == new Money(2)` là `false` vì `amount` khác.

Bài học

- Với object: `===` hỏi "có phải chính nó không", `==` hỏi "có cùng giá trị không".
- Muốn sao chép thật sự một object, dùng `clone`. Với object `readonly` bất biến, bạn thường không cần clone mà tạo object mới.

← Về đề bài 7.4

Đáp án 7.5 BankAccount: ném và bắt exception

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

class BankAccount
{
    private int $balance = 0;

    public function deposit(int $amount): void
    {
        if ($amount <= 0) {
            throw new InvalidArgumentException('Sô´tiền phải lớn hơn 0');
        }

        $this->balance += $amount;
    }
```

```
    public function withdraw(int $amount): void
    {
        if ($amount <= 0) {
            throw new InvalidArgumentException('Sô´tiêǹ phải lớn hơn 0');
        }

        if ($amount > $this->balance) {
            throw new RuntimeException('Sô´dư không đủ');
        }

        $this->balance -= $amount;
    }

    public function balance(): int
    {
        return $this->balance;
    }
}

$acc = new BankAccount();
$acc->deposit(1000);
$acc->withdraw(300);
echo $acc->balance(), "\n";

try {
    $acc->withdraw(5000);
} catch (InvalidArgumentException $e) {
    echo 'Đâù vào sai: ', $e->getMessage(), "\n";
} catch (RuntimeException $e) {
    echo 'Lôĩ nghiệp vụ: ', $e->getMessage(), "\n";
} finally {
    echo 'Xong', "\n";
}

try {
    $acc->deposit(-5);
} catch (InvalidArgumentException $e) {
    echo 'Đâù vào sai: ', $e->getMessage(), "\n";
} catch (RuntimeException $e) {
    echo 'Lôĩ nghiệp vụ: ', $e->getMessage(), "\n";
}

echo $acc->balance(), "\n";
```

KẾT QUẢ KHI CHẠY

```
700
Lôĩ nghiệp vụ: Sô´dư không đủ
Xong
Đâù vào sai: Sô´tiêǹ phải lớn hơn 0
700
```

Giải thích

- Hàm **kiểm tra điều kiện sai trước rồi** `throw` (cách viết "guard clause"), phần xử lý bình thường nằm cuối. Code phẳng, ít `else` lồng nhau.
- `private int $balance = 0` có giá trị khởi tạo ngay tại khai báo, nên không cần constructor.

- `InvalidArgumentException` dùng khi **đầu vào sai** (lập trình viên gọi sai). `RuntimeException` dùng khi **lỗi xảy ra lúc chạy** do trạng thái (hết tiền, file mất). Cả hai kế thừa `Exception`.
- Thứ tự `catch`: PHP chọn khối `catch` **đầu tiên** khớp. Hai class này độc lập nên thứ tự không quan trọng; nhưng nếu bắt `Exception` (cha) trước thì các `catch` con phía sau không bao giờ chạy. Quy tắc: bắt **cụ thể trước, chung sau**.
- `finally` luôn chạy dù có lỗi hay không (dùng để đóng file, mở khoá...). Số dư vẫn là `700` sau hai lần thất bại, vì exception bị ném **trước** khi sửa `balance`.

Triết lý Laravel

Method ném exception, chỗ gọi (hoặc Exception Handler của Laravel) quyết định biến nó thành response. Controller của bạn sẽ ít khi có `try/catch`.

← Về đề bài 7.5

Đáp án 7.6 Cart: kết hợp class và array

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

final class Item
{
    public function __construct(
        public readonly string $name,
        public readonly int $price,
        public readonly int $qty,
    ) {}
}

class Cart
{
    private array $items = [];

    public function add(Item $item): void
    {
        $this->items[] = $item;
    }

    public function total(): int
    {
        return array_reduce(
            $this->items,
            fn (int $carry, Item $i): int => $carry + $i->price * $i->qty,
            0
        );
    }

    public function names(): array
    {
        $names = array_map(fn (Item $i): string => $i->name, $this->items);
        sort($names);

        return $names;
    }
}
```

```php
$cart = new Cart();
$cart->add(new Item('Áo', 200000, 2));
$cart->add(new Item('Quần', 350000, 1));
$cart->add(new Item('Mũ', 50000, 3));

echo $cart->total(), "\n";
echo json_encode($cart->names(), JSON_UNESCAPED_UNICODE), "\n";
```

KẾT QUẢ KHI CHẠY

```
900000
["Mũ","Quần","Áo"]
```

Giải thích

- `Item` là **Value Object** bất biến, `Cart` quản lý trạng thái (mảng `$items`).
- `$this->items[] = $item` thêm object vào cuối mảng thuộc tính.
- `total()`: `array_reduce` với callback nhận `Item $i` (khai báo class type trong tham số callback). Tính `200000*2 + 350000*1 + 50000*3 = 400000 + 350000 + 150000 = 900000`.
- `names()`: `array_map` lấy tên, rồi `sort` (sắp tại chỗ) để theo thứ tự chữ cái. Lưu ý PHP sắp chuỗi theo mã byte nên tiếng Việt có dấu có thể không đúng thứ tự tự nhiên: `'Áo'` có ký tự đầu là `Á` (mã lớn hơn `M`, `Q`), nên đứng cuối. Muốn sắp đúng tiếng Việt cần dùng `Collator` của extension `intl`.
- Kiểu `array` ở thuộc tính chỉ nói "là mảng", không nói phần tử gì; PHP chưa có kiểu `Item[]` thật (chỉ ghi chú docblock).

Mở rộng

Thử thêm method `isEmpty(): bool` và `remove(string $name): void` (dùng `array_filter` rồi `array_values`). Đây chính là cách một Service Laravel quản lý giỏ hàng.

← Về đề bài 7.6

ĐÁP ÁN — CHƯƠNG 8

File, CSV, JSON và xử lý lỗi

Đáp án 8.1 Ghi đè và ghi nối: file_put_contents

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$path = __DIR__ . '/note.txt';

file_put_contents($path, "Dòng 1\n");
file_put_contents($path, "Dòng 2\n", FILE_APPEND);
echo file_get_contents($path);

echo "---\n";

file_put_contents($path, "Làm lại\n");
echo file_get_contents($path);

unlink($path);
```

KẾT QUẢ KHI CHẠY

```
Dòng 1
Dòng 2
---
Làm lại
```

Giải thích

- `file_put_contents($path, $data)` **ghi đè** toàn bộ file (tạo mới nếu chưa có). Thêm cờ `FILE_APPEND` để ghi nối vào cuối.
- `file_get_contents($path)` đọc cả file thành một chuỗi. Nếu file không tồn tại, nó trả `false` kèm warning, nên cần kiểm tra bằng `file_exists` trước.
- Kết quả: sau hai lần ghi đầu, file có hai dòng. Lần ghi thứ 3 **không có** `FILE_APPEND` nên xoá sạch và chỉ còn `Làm lại`.
- `unlink($path)` xoá file. Hãy dọn file tạm sau khi thử.

> **Lưu ý:** Quên `FILE_APPEND` là lỗi mất dữ liệu phổ biến nhất khi ghi log bằng tay. (Laravel ghi log qua `Log::info()` nên bạn ít tự làm.)

Khi nào dùng

Phù hợp file nhỏ (đọc cả vào RAM). File lớn dùng đọc từng dòng (bài 8.2).

← Về đề bài 8.1

Đáp án 8.2 Đọc từng dòng: fopen và fgets

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$path = __DIR__ . '/lines.txt';
file_put_contents($path, "một\nhai\nba\n\nnăm\n");

$handle = fopen($path, 'r');
$n = 0;
while (($line = fgets($handle)) !== false) {
    $n++;
    echo $n, ': ', rtrim($line, "\r\n"), "\n";
}
fclose($handle);

echo "---\n";

$lines = file($path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
echo count($lines), "\n";
echo json_encode($lines, JSON_UNESCAPED_UNICODE), "\n";

unlink($path);
```

KẾT QUẢ KHI CHẠY

```
1: một
2: hai
3: ba
4:
5: năm
---
4
["một","hai","ba","năm"]
```

Giải thích

- `fopen($path, 'r')` mở file chế độ đọc và trả về **handle** (tài nguyên). Chế độ khác: `'w'` ghi (xoá nội dung cũ), `'a'` ghi nối.
- `fgets($handle)` đọc một dòng (**kèm ký tự `\n` ở cuối**); trả `false` khi hết file. Vòng `while (($line = fgets(...)) !== false)` là mẫu chuẩn. Phải so `!== false` chứ không chỉ viết `while ($line = fgets(...))`, vì dòng cuối của file chỉ chứa chữ `0` (không có ký tự xuống dòng) sẽ là chuỗi `"0"`, mà `"0"` là falsy nên vòng lặp dừng sớm và bỏ sót dòng đó.
- `rtrim($line, "\r\n")` cắt ký tự xuống dòng ở cuối, kể cả `\r\n` của file Windows.
- Dòng trống (dòng 4) vẫn là một dòng nên cách 1 in ra `4:` (rỗng).
- `file($path, flags)` đọc cả file thành mảng, mỗi dòng một phần tử. `FILE_IGNORE_NEW_LINES` bỏ `\n`, `FILE_SKIP_EMPTY_LINES` bỏ dòng trống, nên còn 4 dòng.
- `fclose($handle)` đóng handle khi xong.

Chọn cách nào

File lớn (log vài trăm MB): dùng `fgets`, vì chỉ giữ một dòng trong RAM. File nhỏ: `file()` cho code ngắn gọn.

← Về đề bài 8.2

Đáp án 8.3 Ghi CSV bằng fputcsv

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function addExpense(string $path, string $date, string $category, int $amount, string $note):
void
{
    $isNew = !file_exists($path);
    $handle = fopen($path, 'a');

    if ($isNew) {
        fputcsv($handle, ['date', 'category', 'amount', 'note']);
    }

    fputcsv($handle, [$date, $category, $amount, $note]);
    fclose($handle);
}

$path = __DIR__ . '/expenses.csv';
addExpense($path, '2026-10-01', 'food', 45000, 'ăn trưa, cà phê');
addExpense($path, '2026-10-02', 'transport', 30000, 'xăng');
addExpense($path, '2026-10-03', 'rent', 3000000, 'tiền phòng');

echo file_get_contents($path);

unlink($path);
```

KẾT QUẢ KHI CHẠY

```
date,category,amount,note
2026-10-01,food,45000,"ăn trưa, cà phê"
2026-10-02,transport,30000,xăng
2026-10-03,rent,3000000,"tiền phòng"
```

Giải thích

- `fputcsv($handle, $mảng)` ghi một dòng CSV. Nó **tự bọc nháy kép** quanh ô có dấu phẩy, nháy kép hoặc xuống dòng, đúng chuẩn CSV. Tự nối chuỗi bằng `implode(',', ...)` sẽ làm ô `ăn trưa, cà phê` vỡ thành 2 cột.
- Chế độ `'a'` (append) ghi vào cuối file và tạo file nếu chưa có. Biến `$isNew` phải được tính **trước khi** `fopen`, vì sau khi mở bằng `'a'` file đã tồn tại.
- Có tiêu đề chỉ ở lần ghi đầu, các lần sau chỉ thêm dòng dữ liệu.
- Output: ô `"ăn trưa, cà phê"` có nháy kép; `"tiền phòng"` cũng được `fputcsv` bọc nháy do có khoảng trắng.

> **Lưu ý:** Mỗi lần chạy file cũ mà không xoá `expenses.csv`, dữ liệu bị **ghi nối thêm** và nhân đôi. Trong bài này ta `unlink` cuối cho sạch.

← Về đề bài 8.3

Đáp án 8.4 Đọc CSV thành mảng kết hợp

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function seedCsv(string $path): void
{
    $handle = fopen($path, 'w');
    fputcsv($handle, ['date', 'category', 'amount', 'note']);
    fputcsv($handle, ['2026-10-01', 'food', 45000, 'ăn trưa, cà phê']);
    fputcsv($handle, ['2026-10-02', 'transport', 30000, 'xăng']);
    fputcsv($handle, ['2026-10-03', 'rent', 3000000, 'tiền phòng']);
    fputcsv($handle, ['2026-10-04', 'food', 60000, 'ăn tối']);
    fputcsv($handle, ['2026-10-05', 'transport', 20000, 'grab']);
    fputcsv($handle, ['2026-10-06', 'food', 35000, 'bún bò']);
    fclose($handle);
}

function loadExpenses(string $path): array
{
    if (!file_exists($path)) {
        throw new RuntimeException('Không tìm thấy file: ' . basename($path));
    }

    $handle = fopen($path, 'r');
    $header = fgetcsv($handle);
    $rows = [];

    while (($data = fgetcsv($handle)) !== false) {
        $row = array_combine($header, $data);
        $row['amount'] = (int) $row['amount'];
        $rows[] = $row;
    }

    fclose($handle);

    return $rows;
}

$path = __DIR__ . '/expenses.csv';
seedCsv($path);

$rows = loadExpenses($path);
echo count($rows), "\n";
echo json_encode($rows[0], JSON_UNESCAPED_UNICODE), "\n";
var_dump($rows[0]['amount']);

unlink($path);
```

KẾT QUẢ KHI CHẠY

```
6
{"date":"2026-10-01","category":"food","amount":45000,"note":"ăn trưa, cà phê"}
int(45000)
```

Giải thích

- `fgetcsv($handle)` đọc một dòng CSV thành mảng các ô. Lần gọi đầu lấy **dòng tiêu đề** gán vào `$header`.
- `array_combine($header, $data)` ghép mảng tên cột với mảng giá trị thành mảng kết hợp `['date' => '2026-10-01', 'category' => 'food', ...]`.
- Dữ liệu đọc từ CSV **luôn là chuỗi**. Phải ép `amount` thành `int` bằng `(int) $row['amount']`. Chứng minh bằng `var_dump` ra `int(45000)`.
- Kiểm tra `file_exists` rồi `throw new RuntimeException` ở đầu hàm: hàm không tự in lỗi mà **ném lên** cho chỗ gọi quyết định xử lý.
- `basename($path)` chỉ giữ tên file trong message để output không chứa đường dẫn máy.

Sai lầm thường gặp

- Quên đóng `fclose`: nhỏ thì không sao, nhưng tốt nhất đóng.
- Không ép kiểu `amount`: cộng chuỗi vẫn chạy vì PHP ép tự động, nhưng dưới `strict_types` truyền chuỗi vào hàm nhận `int` sẽ gây `TypeError`.

← Về đề bài 8.4

Đáp án 8.5 Báo cáo chi tiêu

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function seedCsv(string $path): void
{
    $handle = fopen($path, 'w');
    fputcsv($handle, ['date', 'category', 'amount', 'note']);
    fputcsv($handle, ['2026-10-01', 'food', 45000, 'ăn trưa, cà phê']);
    fputcsv($handle, ['2026-10-02', 'transport', 30000, 'xăng']);
    fputcsv($handle, ['2026-10-03', 'rent', 3000000, 'tiền phòng']);
    fputcsv($handle, ['2026-10-04', 'food', 60000, 'ăn tối']);
    fputcsv($handle, ['2026-10-05', 'transport', 20000, 'grab']);
    fputcsv($handle, ['2026-10-06', 'food', 35000, 'bún bò']);
    fclose($handle);
}

function loadExpenses(string $path): array
{
    if (!file_exists($path)) {
        throw new RuntimeException('Không tìm thấy file: ' . basename($path));
    }

    $handle = fopen($path, 'r');
    $header = fgetcsv($handle);
    $rows = [];

    while (($data = fgetcsv($handle)) !== false) {
        $row = array_combine($header, $data);
        $row['amount'] = (int) $row['amount'];
        $rows[] = $row;
    }

    fclose($handle);
```

```
    return $rows;
}

function report(array $rows): array
{
    $total = array_reduce($rows, fn (int $carry, array $row): int => $carry + $row['amount'],
0);

    $byCategory = [];
    foreach ($rows as $row) {
        $byCategory[$row['category']] ??= 0;
        $byCategory[$row['category']] += $row['amount'];
    }

    arsort($byCategory);

    return [
        'total' => $total,
        'byCategory' => $byCategory,
        'top' => array_key_first($byCategory),
    ];
}

$path = __DIR__ . '/expenses.csv';
seedCsv($path);

echo json_encode(report(loadExpenses($path)), JSON_UNESCAPED_UNICODE), "\n";

unlink($path);
```

KẾT QUẢ KHI CHẠY

```
{"total":3190000,"byCategory":{"rent":3000000,"food":140000,"transport":50000},"top":"rent"}
```

Giải thích

- `total`: `array_reduce` cộng `amount` từng dòng, giá trị khởi đầu `0`.
- `byCategory`: duyệt từng dòng, dùng danh mục làm key. `??= 0` khởi tạo khi gặp lần đầu, rồi `+=` cộng dồn. Kết quả trước khi sắp: `food 140000, transport 50000, rent 3000000`.
- `arsort` sắp theo **giá trị giảm dần và giữ key** (nếu dùng `rsort`, tên danh mục sẽ mất, xem bài 5.5). Sau sắp: `rent, food, transport`.
- `array_key_first($byCategory)` lấy key đầu tiên, tức danh mục lớn nhất: `rent`.
- Hàm `report` **chỉ tính toán và trả dữ liệu**, không in ra. Tách việc tính và việc hiển thị là thói quen tốt (dễ test, dễ trả JSON).

Mở rộng

Thử tìm danh mục chi **ít nhất** (`array_key_last` sau `arsort`), hoặc tính trung bình mỗi danh mục. Khi có `Collection` của Laravel: `collect($rows)->groupBy('category')->map->sum('amount')`.

← Về đề bài 8.5

Đáp án 8.6 JSON: ghi ra rồi đọc lại

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$report = [
    'total' => 3190000,
    'byCategory' => ['rent' => 3000000, 'food' => 140000, 'transport' => 50000],
    'top' => 'rent',
    'ghiChu' => 'Tổng kết tháng 10',
];

$path = __DIR__ . '/report.json';

file_put_contents(
    $path,
    json_encode($report, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE)
);

echo file_get_contents($path), "\n";

$back = json_decode(file_get_contents($path), true);
echo $back['total'], "\n";
echo $back['top'], "\n";

unlink($path);
```

KẾT QUẢ KHI CHẠY

```
{
    "total": 3190000,
    "byCategory": {
        "rent": 3000000,
        "food": 140000,
        "transport": 50000
    },
    "top": "rent",
    "ghiChu": "Tổng kết tháng 10"
}
3190000
rent
```

Giải thích

- `json_encode($data, flags)`: `JSON_PRETTY_PRINT` xuống dòng và thụt lề cho dễ đọc; `JSON_UNESCAPED_UNICODE` giữ nguyên chữ có dấu. Thiếu cờ này `Tổng kết` thành `T\u1ed5ng k\u1ebft`, hợp lệ về mặt JSON nhưng khó đọc. Hai cờ ghép bằng toán tử `|` (OR bit).
- `json_decode($chuỗi, true)`: tham số thứ hai `true` trả về **mảng kết hợp**. Bỏ nó (hoặc `false`) thì nhận **object** (`stdClass`), và `$back['total']` sẽ lỗi, phải viết `$back->total`.
- Vòng đi-về: mảng, JSON, file, JSON, mảng, cho lại cùng dữ liệu. Đây là cách chương trình lưu trạng thái đơn giản nhất.
- Nếu chuỗi JSON hỏng, `json_decode` trả `null`. Thêm cờ `JSON_THROW_ON_ERROR` để nó ném `JsonException` thay vì trả `null` âm thầm (nên dùng trong code thật).

Liên hệ Laravel

`return response()->json($data)` và việc Laravel tự biến mảng/Resource thành JSON đều dựa trên `json_encode`.

← Về đề bài 8.6

Đáp án 8.7 **Exception: try, catch, finally**

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function readOrFail(string $path): string
{
    if (!file_exists($path)) {
        throw new RuntimeException('Không tìm thấy file: ' . basename($path));
    }

    return file_get_contents($path);
}

try {
    echo readOrFail(__DIR__ . '/khong-ton-tai.csv');
} catch (RuntimeException $e) {
    echo 'Lỗi: ', $e->getMessage(), "\n";
} finally {
    echo 'Đã thử đọc file', "\n";
}

$real = __DIR__ . '/real.txt';
file_put_contents($real, "xin chào\n");

try {
    echo readOrFail($real);
} catch (RuntimeException $e) {
    echo 'Lỗi: ', $e->getMessage(), "\n";
} finally {
    echo 'Đã thử đọc file', "\n";
}

unlink($real);
```

KẾT QUẢ KHI CHẠY

```
Lỗi: Không tìm thấy file: khong-ton-tai.csv
Đã thử đọc file
xin chào
Đã thử đọc file
```

Giải thích

- `throw new RuntimeException('...')` dừng hàm ngay lập tức và đẩy lỗi lên nơi gọi. Dòng `return` phía dưới không được chạy.
- `catch (RuntimeException $e)` bắt lỗi đúng loại; `$e->getMessage()` lấy thông điệp. Còn có `getCode()`, `getFile()`, `getLine()`, `getTraceAsString()`.

- `finally` **luôn chạy** dù có lỗi hay không. Cả hai lần gọi đều in `Đã thử đọc file`. Dùng để dọn dẹp tài nguyên (đóng file, mở khoá).
- Nếu không có `catch` phù hợp, lỗi lan lên cao hơn và cuối cùng làm dừng chương trình với `Fatal error: Uncaught ...`.

Nguyên tắc

Hàm có thể thất bại thì **ném exception** thay vì `echo` lỗi hoặc trả `false`. Chỗ gọi mới biết nên in, ghi log hay trả HTTP 404. Trong Laravel, bạn ném và để **Exception Handler** tập trung xử lý (Part 07).

← Về đề bài 8.7

ĐÁP ÁN — CHƯƠNG 9

Luyện logic: sắp xếp, tìm cặp, nhóm

Đáp án 9.1 Bubble sort có tham số hướng

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function bubbleSort(array $arr, bool $desc = false): array
{
    $n = count($arr);

    do {
        $swapped = false;

        for ($i = 0; $i < $n - 1; $i++) {
            $outOfOrder = $desc ? $arr[$i] < $arr[$i + 1] : $arr[$i] > $arr[$i + 1];

            if ($outOfOrder) {
                [$arr[$i], $arr[$i + 1]] = [$arr[$i + 1], $arr[$i]];
                $swapped = true;
            }
        }
    } while ($swapped);

    return $arr;
}

echo implode(', ', bubbleSort([5, 2, 9, 1, 7])), "\n";
echo implode(', ', bubbleSort([5, 2, 9, 1, 7], true)), "\n";
echo implode(', ', bubbleSort([])), "|\n";
echo implode(', ', bubbleSort([3])), "\n";
```

KẾT QUẢ KHI CHẠY

```
1, 2, 5, 7, 9
9, 7, 5, 2, 1
|
3
```

Giải thích

- Vòng `do ... while ($swapped)` lặp lại các lượt duyệt cho tới khi một lượt **không đổi chỗ nào**, nghĩa là mảng đã đúng thứ tự. Đây là phiên bản có "dừng sớm", nhanh hơn bản hai vòng `for` cố định khi mảng gần như đã sắp.
- `$outOfOrder` quyết định hai phần tử cạnh nhau có sai thứ tự không. Với tăng dần, sai khi phần tử trước **lớn hơn** phần tử sau; với giảm dần, sai khi phần tử trước **nhỏ hơn**. Đổi cả hai hướng chỉ bằng đổi dấu so sánh, trong một biểu thức điều kiện.
- `[$arr[$i], $arr[$i + 1]] = [$arr[$i + 1], $arr[$i]];` đổi chỗ hai phần tử mà không cần biến tạm (list destructuring). Vế phải tạo mảng tạm trước, rồi vế trái gán lại.

- Mảng rỗng: `$n - 1 = -1`, vòng `for` không chạy, trả mảng rỗng. Mảng 1 phần tử: tương tự. Luôn thử các trường hợp biên này.

Trace tay `[3, 1, 2]` tăng dần

Lượt 1: so `3,1` đổi thành `[1,3,2]`; so `3,2` đổi thành `[1,2,3]`; có đổi nên lặp lượt 2. Lượt 2: không đổi, dừng. Kết quả `[1,2,3]`.

← Về đề bài 9.1

Đáp án 9.2 ## Selection sort

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function selectionSort(array $arr): array
{
    $n = count($arr);

    for ($i = 0; $i < $n - 1; $i++) {
        $minIndex = $i;

        for ($j = $i + 1; $j < $n; $j++) {
            if ($arr[$j] < $arr[$minIndex]) {
                $minIndex = $j;
            }
        }

        if ($minIndex !== $i) {
            [$arr[$i], $arr[$minIndex]] = [$arr[$minIndex], $arr[$i]];
        }
    }

    return $arr;
}

echo implode(', ', selectionSort([64, 25, 12, 22, 11])), "\n";
echo implode(', ', selectionSort([])), "|\n";
```

KẾT QUẢ KHI CHẠY

```
11, 12, 22, 25, 64
|
```

Giải thích

- Vòng ngoài `$i` xác định "vị trí cần điền". Các vị trí `0..i-1` đã đúng thứ tự cuối cùng.
- Vòng trong tìm **chỉ số** của phần tử nhỏ nhất trong đoạn chưa sắp (`$i` đến cuối). Ta nhớ **chỉ số** (`$minIndex`), không nhớ giá trị, để đổi chỗ được.
- Đổi chỗ chỉ khi `$minIndex !== $i` để tránh đổi vô ích.
- Khác bubble sort: mỗi lượt chỉ đổi chỗ **một lần** (ít thao tác ghi), còn bubble đổi nhiều lần. Cả hai đều cỡ n^2 phép so sánh.

Trace `[64, 25, 12, 22, 11]`

`i=0`: nhỏ nhất là 11 (vị trí 4) đổi với 64 thành `[11,25,12,22,64]`. `i=1`: nhỏ nhất là 12 đổi với 25 thành `[11,12,25,22,64]`. `i=2`: nhỏ nhất là 22 đổi với 25 thành `[11,12,22,25,64]`. `i=3`: 25 đã đúng chỗ. Xong.

← Về đề bài 9.2

Đáp án 9.3 — Max, min và lớn thứ hai trong một lượt duyệt

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function stats(array $arr): array
{
    if ($arr === []) {
        return ['max' => null, 'min' => null, 'second' => null];
    }

    $max = $min = $arr[0];
    $second = null;

    foreach ($arr as $x) {
        if ($x > $max) {
            $second = $max;
            $max = $x;
        } elseif ($x !== $max && ($second === null || $x > $second)) {
            $second = $x;
        }

        if ($x < $min) {
            $min = $x;
        }
    }

    return ['max' => $max, 'min' => $min, 'second' => $second];
}

echo json_encode(stats([5, 9, 2, 9, 7])), "\n";
echo json_encode(stats([5, 5, 5])), "\n";
echo json_encode(stats([])), "\n";
```

KẾT QUẢ KHI CHẠY

```
{"max":9,"min":2,"second":7}
{"max":5,"min":5,"second":null}
{"max":null,"min":null,"second":null}
```

Giải thích

- Ba biến theo dõi: `$max`, `$min`, `$second`. Khởi đầu `max` và `min` bằng phần tử đầu, `second` là `null` (chưa có).
- Gặp số **lớn hơn** `max`: số `max` cũ trở thành `second`, rồi `max` nhận số mới. (Thứ tự gán quan trọng: phải lưu `max` cũ vào `second` **trước** khi ghi đè `max`.)
- Gặp số không lớn hơn `max`: nó có thể là ứng viên `second`, nếu **khác** `max` (loại trùng) và lớn hơn `second` hiện tại (hoặc `second` còn `null`).
- `min` cập nhật độc lập.

- Với `[5, 9, 2, 9, 7]`: số 9 thứ hai không làm `second = 9` vì `$x !== $max` là sai. Kết quả `second = 7`.
- Với `[5, 5, 5]`: mọi số đều bằng `max` nên `second` mãi `null`.

Độ phức tạp

Một lần duyệt = `n` bước, so với sort rồi lấy phần tử = `n log n`. Với mảng lớn khác biệt đáng kể.

← Về đề bài 9.3

Đáp án 9.4 Tìm kiếm nhị phân và so sánh với tìm tuần tự

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function binarySearch(array $sorted, int $target, int &$steps = 0): int
{
    $low = 0;
    $high = count($sorted) - 1;
    $steps = 0;

    while ($low <= $high) {
        $steps++;
        $mid = intdiv($low + $high, 2);

        if ($sorted[$mid] === $target) {
            return $mid;
        }

        if ($sorted[$mid] < $target) {
            $low = $mid + 1;
        } else {
            $high = $mid - 1;
        }
    }

    return -1;
}

function linearSearch(array $arr, int $target, int &$steps = 0): int
{
    $steps = 0;

    foreach ($arr as $i => $x) {
        $steps++;

        if ($x === $target) {
            return $i;
        }
    }

    return -1;
}

$nums = range(1, 1000);
```

```php
$binSteps = 0;
$linSteps = 0;
echo 'binary: vị trí ', binarySearch($nums, 777, $binSteps), ', ', $binSteps, " bước\n";
echo 'linear: vị trí ', linearSearch($nums, 777, $linSteps), ', ', $linSteps, " bước\n";
echo 'không có: ', binarySearch($nums, 5000), "\n";
```

KẾT QUẢ KHI CHẠY

```
binary: vị trí 776, 8 bước
linear: vị trí 776, 777 bước
không có: -1
```

Giải thích

- **Ý tưởng**: mảng đã sắp thì so sánh với phần tử **giữa**. Nếu nhỏ hơn mục tiêu, mục tiêu nằm bên phải, bỏ cả nửa trái; ngược lại bỏ nửa phải. Mỗi bước loại một nửa.
- `$low` và `$high` là hai đầu vùng còn nghi ngờ. Vòng lặp chạy khi `$low <= $high`. Hết vùng mà chưa thấy thì trả `-1`.
- `intdiv($low + $high, 2)` chia lấy phần nguyên. Dùng `/` sẽ ra số thực.
- Quan trọng: khi bỏ nửa, phải `+ 1` hoặc `- 1` (`$low = $mid + 1`, `$high = $mid - 1`). Nếu viết `$low = $mid` có thể lặp vô hạn.
- `int &$steps`: dấu `&` truyền **tham chiếu**, nên hàm sửa trực tiếp biến `$binSteps` bên ngoài. Cách PHP "trả thêm một giá trị" ngoài `return`.
- Vị trí 776 chính là số 777 (đếm từ 0).

So sánh

Mảng 1000 phần tử: nhị phân tối đa khoảng 10 bước (`log2(1000)`), tuần tự 777 bước. Khi mảng lớn gấp đôi, nhị phân chỉ thêm 1 bước. Đó là sức mạnh của **chia để trị**, và là lý do index cơ sở dữ liệu nhanh.

> **Lưu ý:** Điều kiện bắt buộc: mảng phải **đã sắp xếp**. Nếu chưa, kết quả sai mà không báo lỗi.

← Về đề bài 9.4

Đáp án 9.5 Tìm cặp có tổng bằng K (hai cách)

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function pairsNested(array $arr, int $k): array
{
    $pairs = [];
    $n = count($arr);

    for ($i = 0; $i < $n; $i++) {
        for ($j = $i + 1; $j < $n; $j++) {
            if ($arr[$i] + $arr[$j] === $k) {
                $pairs[] = [$arr[$i], $arr[$j]];
            }
        }
    }

    return $pairs;
```

```php
}

function pairsWithSum(array $arr, int $k): array
{
    $seen = [];
    $pairs = [];

    foreach ($arr as $x) {
        $need = $k - $x;

        if (isset($seen[$need])) {
            $pairs[] = [$need, $x];
        }

        $seen[$x] = true;
    }

    return $pairs;
}

$nums = [1, 5, 3, 7, 4, 6, 2];
echo 'nested: ', json_encode(pairsNested($nums, 8)), "\n";
echo 'hash:   ', json_encode(pairsWithSum($nums, 8)), "\n";
```

KẾT QUẢ KHI CHẠY

```
nested: [[1,7],[5,3],[6,2]]
hash:   [[5,3],[1,7],[6,2]]
```

Cách 1: hai vòng lồng nhau

Vòng ngoài chọn `$arr[$i]`, vòng trong chọn `$arr[$j]` với `$j` bắt đầu từ `$i + 1` (tránh ghép số với chính nó và tránh đếm trùng cặp `(a,b)` và `(b,a)`). Số phép so sánh cỡ `n²/2`.

Cách 2: dùng sổ (hash map)

- `$seen` là mảng kết hợp với key là các số đã gặp. Kiểm tra `isset($seen[$need])` gần như **tức thì** (PHP dùng bảng băm), không phải quét lại cả mảng.
- Mỗi số `x` chỉ cần **một lần kiểm tra** `K - x` đã có trong sổ chưa, rồi ghi `x` vào sổ. Tổng chỉ `n` bước.
- Cặp được phát hiện khi gặp số thứ hai nên thứ tự cặp trong kết quả khác cách 1, nhưng **tập cặp giống nhau**: `{1,7}`, `{5,3}`, `{6,2}`.
- Số `4`: `K - 4 = 4` nhưng trong mảng chỉ có một số 4, nên không tạo cặp `(4,4)`. Cách 2 xử lý đúng vì ta chỉ ghi `4` vào sổ **sau khi** kiểm tra.

Bài học quan trọng

Dùng mảng kết hợp để **đổi thời gian lấy bộ nhớ**: từ `n²` xuống `n`. Đây là kỹ thuật dùng nhiều nhất trong các bài thuật toán. Ngược lại, `in_array()` trên mảng thường phải quét tuần tự nên chậm hơn `isset` trên key.

← Về đề bài 9.5

Đáp án 9.6 Phần tử trùng và số lần xuất hiện

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);
```

```php
function duplicates(array $arr): array
{
    $count = [];

    foreach ($arr as $x) {
        $count[$x] ??= 0;
        $count[$x]++;
    }

    return array_filter($count, fn (int $c): bool => $c > 1);
}

echo json_encode(duplicates([1, 2, 2, 3, 3, 3, 4])), "\n";
echo json_encode(duplicates(['a', 'b', 'a'])), "\n";
echo json_encode(duplicates([1, 2, 3])), "\n";
```

KẾT QUẢ KHI CHẠY

```
{"2":2,"3":3}
{"a":2}
[]
```

Giải thích

- Mẫu "bộ đếm": `$count[$x] ??= 0;` rồi `$count[$x]++;`. Key của `$count` là giá trị cần đếm, value là số lần gặp.
- `array_filter` giữ lại các mục có `$c > 1`. Vì `array_filter` giữ key, kết quả là `{"2":2,"3":3}` (key là chính giá trị trùng), hợp ý đề.
- Trường hợp không có trùng, kết quả là mảng rỗng, `json_encode([])` ra `[]`.
- Có sẵn hàm `array_count_values($arr)` làm bước đếm: `array_filter(array_count_values($arr), fn ($c) => $c > 1)`. Nhưng tự viết giúp bạn hiểu mẫu bộ đếm.

Lưu ý về key

Giá trị số dùng làm key bị **ép thành số nguyên**; chuỗi `'1'` và số `1` chung một key `1`. Với `true`, `null`, số thực cũng có quy tắc ép riêng, nên đếm dữ liệu hỗn hợp cần cẩn thận.

← Về đề bài 9.6

Đáp án 9.7 Nhóm các từ đảo chữ (anagram)

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function groupAnagrams(array $words): array
{
    $groups = [];

    foreach ($words as $word) {
        $letters = str_split($word);
        sort($letters);
        $groups[implode('', $letters)][] = $word;
    }
```

```
    return array_values($groups);
}

echo json_encode(
    groupAnagrams(['listen', 'silent', 'enlist', 'google', 'gogole', 'cat', 'act'])
), "\n";
```

KẾT QUẢ KHI CHẠY

```
[["listen","silent","enlist"],["google","gogole"],["cat","act"]]
```

Giải thích

- Với mỗi từ: `str_split` tách thành mảng chữ cái, `sort` sắp xếp chữ cái, `implode('', ...)` nối lại. `listen`, `silent`, `enlist` đều cho `eilnst`. `cat` và `act` cùng cho `act`.
- Chuỗi đã sắp (**khoá chính tắc**) làm key: `$groups[$key][] = $word` thêm từ vào nhóm của key đó. PHP tự **tạo mảng con** ở lần đầu `[]=` vào key chưa tồn tại (autovivification), không cần khởi tạo trước.
- `array_values` bỏ các key kỹ thuật (`eilnst`, `act`...) để kết quả là danh sách nhóm sạch.
- Ý tưởng "đổi mỗi phần tử thành một khoá chuẩn rồi nhóm theo khoá" là mẫu **group by** cực phổ biến (tương tự `groupBy` của Laravel Collection).

Lưu ý

`str_split` tách theo **byte**, nên chữ có dấu tiếng Việt (nhiều byte) sẽ bị tách sai. Với tiếng Việt dùng `mb_str_split`.

← Về đề bài 9.7

Đáp án 9.8 Kiểm tra ngoặc hợp lệ bằng ngăn xếp

CODE ĐÁP ÁN

```
<?php

declare(strict_types=1);

function isValidBrackets(string $s): bool
{
    $pairs = [')' => '(', ']' => '[', '}' => '{'];
    $stack = [];

    foreach (str_split($s) as $ch) {
        if (in_array($ch, $pairs, true)) {
            $stack[] = $ch;
            continue;
        }

        if (array_pop($stack) !== $pairs[$ch]) {
            return false;
        }
    }

    return $stack === [];
}

foreach (['([]{})', '([)]', '((', '', ')('] as $s) {
```

```
    echo json_encode($s), ' => ', var_export(isValidBrackets($s), true), "\n";
}
```

KẾT QUẢ KHI CHẠY

```
"([]{})" => true
"([)]" => false
"((" => false
"" => true
")(" => false
```

Giải thích

- `$pairs` ánh xạ **ngoặc đóng sang ngoặc mở tương ứng**. Dùng làm hai việc: `in_array($ch, $pairs, true)` kiểm tra `$ch` có phải ngoặc mở không (giá trị của `$pairs`), và `$pairs[$ch]` cho ngoặc mở cần khớp.
- Ngoặc mở: `$stack[] = $ch` đẩy vào (push). `continue` sang ký tự tiếp.
- Ngoặc đóng: `array_pop($stack)` lấy phần tử cuối ra (pop). Nếu ngăn xếp **rỗng**, `array_pop` trả `null`, khác `$pairs[$ch]`, nên trả `false` ngay (trường hợp `')('`, đóng khi chưa mở). Nếu lấy ra **không khớp** loại ngoặc (`'([)]'`), cũng `false`.
- Cuối cùng, ngăn xếp phải **rỗng** (`$stack === []`). Chuỗi `'(('` còn dư 2 ngoặc mở nên `false`.
- Chuỗi rỗng hợp lệ (`true`) vì không có gì sai.

Vì sao dùng stack

Ngoặc mở **gần nhất** phải được đóng **trước** (vào sau, ra trước: LIFO). Đó chính là tính chất của ngăn xếp. Cơ chế tương tự dùng khi trình biên dịch phân tích cú pháp và khi duyệt cây thư mục.

← Về đề bài 9.8

Đáp án 9.9 FizzBuzz

CODE ĐÁP ÁN

```
<?php

declare(strict_types=1);

for ($i = 1; $i <= 20; $i++) {
    echo match (true) {
        $i % 15 === 0 => 'FizzBuzz',
        $i % 3 === 0 => 'Fizz',
        $i % 5 === 0 => 'Buzz',
        default => $i,
    }, "\n";
}
```

KẾT QUẢ KHI CHẠY

```
1
2
Fizz
4
Buzz
Fizz
7
8
```

```
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
```

Giải thích

- Chia hết cho 3 **và** 5 nghĩa là chia hết cho 15. Nhánh `FizzBuzz` **phải đứng đầu**, nếu không số 15 sẽ khớp nhánh `Fizz` trước. Đây là bài học về thứ tự nhánh của `match (true)` (bài 6.5).
- `$i % 3 === 0`: phép chia lấy dư bằng 0 nghĩa là chia hết.
- Nhánh `default => $i` trả về số nguyên; `echo` tự đổi thành chuỗi.
- Bài kiểm tra tư duy: phải nghĩ đến thứ tự điều kiện, không chỉ biết cú pháp.

← Về đề bài 9.9

Đáp án 9.10 Chuỗi đối xứng (palindrome)

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function isPalindrome(string $s): bool
{
    $clean = strtolower(preg_replace('/[^A-Za-z0-9]/', '', $s));

    return $clean === strrev($clean);
}

foreach (['Madam', 'A man, a plan, a canal: Panama', 'hello'] as $s) {
    echo $s, ' => ', var_export(isPalindrome($s), true), "\n";
}
```

KẾT QUẢ KHI CHẠY

```
Madam => true
A man, a plan, a canal: Panama => true
hello => false
```

Giải thích

- `preg_replace('/[^A-Za-z0-9]/', '', $s)` xoá mọi ký tự **không** nằm trong `A-Z`, `a-z`, `0-9`. Dấu `^` đầu trong `[]` nghĩa là "phủ định".
- `strtolower` đưa về chữ thường để bỏ qua hoa thường.
- `strrev` đảo ngược chuỗi. So sánh bằng `===` với chuỗi gốc đã làm sạch.
- Kết quả: `Madam` là `true`, câu `A man, a plan...` là `true` vì sau khi làm sạch là `amanaplanacanalpanama`, `hello` là `false`.

> **Lưu ý:** Giới hạn: `strrev` và regex trên đều xử lý theo **byte**, không đúng với chữ tiếng Việt có dấu. Với Unicode dùng hàm `mb_*` hoặc cờ `u` của regex. Bài này chỉ chữ Latin không dấu.

← Về đề bài 9.10

Đáp án 9.11 Số nguyên tố

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function isPrime(int $n): bool
{
    if ($n < 2) {
        return false;
    }

    for ($i = 2; $i * $i <= $n; $i++) {
        if ($n % $i === 0) {
            return false;
        }
    }

    return true;
}

$primes = array_filter(range(1, 49), 'isPrime');
echo implode(', ', $primes), "\n";
```

KẾT QUẢ KHI CHẠY

```
2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47
```

Giải thích

- Số nhỏ hơn 2 (0, 1, âm) không phải số nguyên tố.
- Nếu `n` có ước lớn hơn `√n` thì nó cũng có ước nhỏ hơn `√n` (ước đối xứng). Nên chỉ cần thử `i` từ 2 đến `√n`, tức điều kiện `$i * $i <= $n`. Viết `$i * $i <= $n` thay `sqrt($n)` để tránh số thực.
- Gặp một ước thì trả `false` ngay; duyệt hết mà không có ước thì `true`.
- `array_filter(range(1, 49), 'isPrime')`: truyền **tên hàm dạng chuỗi** làm callback (PHP chấp nhận). Kết quả `2, 3, 5, 7, 11, ...`.

Mở rộng

Thử tính xem với `n = 1000003`, thử đến căn bậc hai tốn bao nhiêu bước so với thử đến `n - 1`.

← Về đề bài 9.11

Đáp án 9.12 Đếm tần suất từ trong câu

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);
```

```php
$text = 'php laravel php vue laravel php';

$count = [];
foreach (explode(' ', $text) as $word) {
    $count[$word] ??= 0;
    $count[$word]++;
}

arsort($count);

foreach ($count as $word => $n) {
    echo $word, ': ', $n, "\n";
}
```

KẾT QUẢ KHI CHẠY

```
php: 3
laravel: 2
vue: 1
```

Giải thích

- `explode(' ', $text)` tách chuỗi thành mảng từ theo dấu cách (như `split(' ')` của JS). Ngược lại là `implode`.
- Mẫu bộ đếm giống bài 6.2 và 9.6.
- `arsort` sắp theo giá trị giảm dần **giữ key** (từ), nên `foreach ($count as $word => $n)` vẫn có đủ tên từ.
- Kết quả: `php: 3`, `laravel: 2`, `vue: 1`.

Lưu ý

Với văn bản thật có dấu câu và hoa/thường, nên chuẩn hoá trước: `strtolower` và `preg_split('/\W+/u', $text, -1, PREG_SPLIT_NO_EMPTY)`. Nhiều khoảng trắng liên tiếp sẽ sinh ra phần tử rỗng nếu chỉ `explode(' ')`.

← Về đề bài 9.12

ĐÁP ÁN — CHƯƠNG 10

Closure và arrow function

Đáp án 10.1 Closure bắt biến thế nào? Thí nghiệm A, B, C

KẾT QUẢ KHI CHẠY

```
A: 10
B: 99
C: 10
```

Giải thích

- **A (`use ($x)`)**: `use` **sao chép giá trị** tại thời điểm tạo closure. Closure giữ lại `10`; việc đổi `$x = 99` sau đó không ảnh hưởng. In `10`.
- **B (`use (&$x)`)**: `&` là **tham chiếu**, closure dùng **chính biến** `$x` ngoài kia. Khi gọi, nó thấy giá trị hiện tại `99`. In `99`.
- **C (`fn`)**: arrow function **bắt theo giá trị** tại lúc tạo (giống A, nhưng tự động). Giữ `10`. In `10`.

Bài học

Hai thứ khác nhau cần phân biệt: **bắt theo giá trị** (chụp ảnh tại thời điểm tạo) và **bắt theo tham chiếu** (nhìn trực tiếp vào biến). `use` mặc định chụp ảnh, thêm `&` mới nhìn trực tiếp. `fn` luôn chụp ảnh nên không sửa được biến ngoài.

> **Lưu ý:** Khác JS: closure JS luôn nhìn trực tiếp biến ngoài (như B). Người từ JS sang rất dễ nhầm A và C.

← Về đề bài 10.1

Đáp án 10.2 array_map: fn và function use

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$prices = [100, 250, 80, 400];
$tax = 0.1;

$withFn = array_map(fn (int $p): float => $p * (1 + $tax), $prices);

$withUse = array_map(function (int $p) use ($tax): float {
    return $p * (1 + $tax);
}, $prices);

echo implode(', ', $withFn), "\n";
echo implode(', ', $withUse), "\n";
var_dump($withFn === $withUse);
```

KẾT QUẢ KHI CHẠY

```
110, 275, 88, 440
110, 275, 88, 440
```

```
bool(true)
```

Giải thích

- Cách 1: `fn` tự thấy `$tax`, không cần khai báo gì.
- Cách 2: closure thường **phải** `use ($tax)`; thiếu thì `$tax` trong thân hàm là biến chưa tồn tại (warning "Undefined variable", kết quả sai).
- Hai cách cho kết quả giống hệt (`var_dump(... === ...)` là `true`). Với phép tính một dòng, ưu tiên `fn` cho gọn.
- Kiểu trả về `: float` đặt **sau** `use (...)` ở closure thường: `function (...) use (...): float { }`.

Lưu ý số thực

`echo` làm tròn số thực khi in, nên `100 * 1.1` hiển thị `110` dù bên trong là `110.00000000000001`. `json_encode` thì in đủ số lẻ. Nếu cần tiền, hãy dùng số nguyên (đồng) hoặc `round()`.

← Về đề bài 10.2

Đáp án 10.3 makeCounter: closure nhớ trạng thái

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function makeCounter(): Closure
{
    $count = 0;

    return function () use (&$count): int {
        return ++$count;
    };
}

$a = makeCounter();
$b = makeCounter();

echo $a(), "\n";
echo $a(), "\n";
echo $b(), "\n";
echo $a(), "\n";
```

KẾT QUẢ KHI CHẠY

```
1
2
1
3
```

Giải thích

- Mỗi lần gọi `makeCounter()` tạo một biến `$count` **riêng** (phạm vi của lần gọi đó). Closure trả về "nhớ" biến ấy: đây chính là **closure** theo nghĩa thật.
- `use (&$count)` (tham chiếu) cho phép closure **sửa** `$count`. Không có `&`, mỗi lần gọi closure sẽ tăng một bản sao rồi bỏ, kết quả luôn `1, 1, 1`.

- `++$count` tăng rồi trả giá trị mới. (`$count++` trả giá trị **cũ** rồi mới tăng, nên sẽ ra `0, 1, 0, 2` .)
- `$a` và `$b` là hai closure với hai `$count` khác nhau, nên gọi `$b` không ảnh hưởng `$a` .
- Không dùng `fn` được vì `fn` chỉ bắt theo giá trị, không sửa được biến ngoài.

Liên hệ Laravel

Container của Laravel giữ closure đăng ký service cùng các biến nó cần, theo cùng cơ chế "closure nhớ môi trường".

← Về đề bài 10.3

Đáp án 10.4 **Hàm trả về hàm: multiplier**

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function multiplier(int $factor): Closure
{
    return fn (int $x): int => $x * $factor;
}

$double = multiplier(2);
$triple = multiplier(3);

echo $double(5), "\n";
echo $triple(5), "\n";

function repeatTimes(int $n, callable $action): void
{
    for ($i = 1; $i <= $n; $i++) {
        $action($i);
    }
}

repeatTimes(3, fn (int $i) => print("lần $i\n"));
```

KẾT QUẢ KHI CHẠY

```
10
15
lần 1
lần 2
lần 3
```

Giải thích

- `multiplier` là **hàm sinh hàm**: mỗi lần gọi trả về một closure "nhớ" `$factor` của lần gọi tạo ra nó. `$double` nhớ 2, `$triple` nhớ 3, hai closure độc lập.
- `fn` đủ dùng vì chỉ **đọc** `$factor` .
- `callable` là kiểu "cái gì gọi được": closure, tên hàm dạng chuỗi (`'strtoupper'`), mảng `[$obj, 'method']` . Hàm `repeatTimes` nhận hành động từ bên ngoài, đây là **truyền closure làm tham số**, mẫu mà `array_map` , `usort` , route Laravel đều dùng.

- `print` là cấu trúc ngôn ngữ trả về `1`, dùng được trong biểu thức (khác `echo`), nên hợp với `fn` một biểu thức. (Kiểu trả về của callback ở đây không bị kiểm tra vì không khai báo.)

← Về đề bài 10.4

Đáp án 10.5 compose: ghép hai hàm

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function compose(Closure $f, Closure $g): Closure
{
    return fn (int $x): int => $g($f($x));
}

$addOne = fn (int $x): int => $x + 1;
$double = fn (int $x): int => $x * 2;

$h1 = compose($addOne, $double);
$h2 = compose($double, $addOne);

echo $h1(5), "\n";
echo $h2(5), "\n";
```

KẾT QUẢ KHI CHẠY

```
12
11
```

Giải thích

- `compose` nhận **hai closure** và trả về **closure thứ ba**. Closure trả về cần `$f` và `$g` của hàm ngoài, nên dùng `fn` (tự bắt) là gọn nhất. Với `function` thì phải `use ($f, $g)`.
- `$g($f($x))`: gọi `$f` với `$x` trước, đưa kết quả vào `$g`.
- `h1 = compose(addOne, double)`: `(5 + 1) * 2 = 12`.
- `h2 = compose(double, addOne)`: `(5 * 2) + 1 = 11`. **Thứ tự quan trọng**: phép ghép hàm không giao hoán.
- Gọi trực tiếp `compose($f, $g)(5)` cũng được trong PHP 8 (gọi ngay kết quả trả về), nhưng gán biến trung gian dễ đọc hơn.

Liên hệ

Middleware trong Laravel là cùng ý tưởng: nhiều hàm ghép thành chuỗi, mỗi hàm bọc quanh hàm sau nó.

← Về đề bài 10.5

Đáp án 10.6 pipeline: chuỗi nhiều hàm

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);
```

```php
function pipeline(array $fns, mixed $value): mixed
{
    return array_reduce(
        $fns,
        fn (mixed $carry, Closure $fn): mixed => $fn($carry),
        $value
    );
}

$steps = [
    fn (int $x): int => $x + 1,
    fn (int $x): int => $x * 2,
    fn (int $x): int => $x - 3,
];

echo pipeline($steps, 5), "\n";
echo pipeline([], 5), "\n";
```

KẾT QUẢ KHI CHẠY

```
9
5
```

Giải thích

- Ánh xạ sang `array_reduce`: **giá trị khởi đầu** (`$value`) là dữ liệu vào; `$carry` là kết quả hiện tại; mỗi vòng lấy một closure `$fn` trong mảng và trả `$fn($carry)`. Kết quả cuối là đầu ra của pipeline.
- Trace: `5` qua `+1` ra `6`, qua `*2` ra `12`, qua `-3` ra `9`.
- Mảng hàm rỗng: `array_reduce` không chạy lần nào và trả đúng giá trị khởi đầu `5`. Không cần xử lý riêng.
- Kiểu `mixed` nghĩa là chấp nhận mọi kiểu dữ liệu (giá trị có thể là số, chuỗi, mảng, object).

Liên hệ Laravel

Request đi qua danh sách middleware (mỗi cái là một bước) giống hệt pipeline này. Laravel có lớp `Illuminate\Pipeline\Pipeline` cài đặt ý tưởng đó.

← Về đề bài 10.6

Đáp án 10.7 filterBy: tự cài lại array_filter

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

function filterBy(array $items, Closure $predicate): array
{
    $result = [];

    foreach ($items as $key => $item) {
        if ($predicate($item)) {
            $result[$key] = $item;
        }
    }

    return $result;
}
```

```php
$posts = [
    ['title' => 'Vue', 'views' => 50],
    ['title' => 'Laravel', 'views' => 100],
    ['title' => 'Docker', 'views' => 100],
    ['title' => 'PHP', 'views' => 50],
    ['title' => 'Redis', 'views' => 100],
];

$mine = filterBy($posts, fn (array $p): bool => $p['views'] > 50);
$builtin = array_filter($posts, fn (array $p): bool => $p['views'] > 50);

echo json_encode(array_column($mine, 'title')), "\n";
echo json_encode(array_keys($mine)), "\n";
var_dump($mine === $builtin);
```

KẾT QUẢ KHI CHẠY

```
["Laravel","Docker","Redis"]
[1,2,4]
bool(true)
```

Giải thích

- Hàm nhận `Closure $predicate` (hàm điều kiện) và gọi nó cho **từng phần tử**. Phần tử nào cho `true` thì giữ.
- `foreach ($items as $key => $item)` lấy cả key. Gán `$result[$key] = $item` (dùng `$key`, **không** dùng `$result[] =`) mới giữ nguyên key gốc, giống `array_filter`. Dùng `[]=` thì key bị đánh lại từ 0.
- Key giữ lại là `1, 2, 4` (Laravel, Docker, Redis), chứng minh bằng `array_keys`.
- `$mine === $builtin` là `true`: hai mảng giống hệt cả key lẫn giá trị lẫn thứ tự, nghĩa là bạn đã cài đúng hành vi của `array_filter`.

Bài học

Hàm bậc cao (higher-order function) là hàm nhận hoặc trả về hàm. `array_map`, `array_filter`, `usort` đều là hàm bậc cao. Tự viết được một cái nghĩa là bạn hiểu chúng không có phép màu nào.

← Về đề bài 10.7

ĐÁP ÁN — CHƯƠNG 11

Tổng hợp: mini project đơn hàng

Đáp án 11.1 Tính tiền đơn hàng với mã giảm giá

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

final class Product
{
    public function __construct(
        public readonly string $name,
        public readonly int $price,
    ) {}
}

final class Line
{
    public function __construct(
        public readonly Product $product,
        public readonly int $qty,
    ) {
        if ($qty <= 0) {
            throw new InvalidArgumentException('Số lượng phải lớn hơn 0');
        }
    }

    public function subtotal(): int
    {
        return $this->product->price * $this->qty;
    }
}

function shippingFee(int $total): int
{
    return match (true) {
        $total >= 500000 => 0,
        $total >= 200000 => 15000,
        default => 30000,
    };
}

$discounts = [
    'none' => fn (int $t): int => 0,
    'vip10' => fn (int $t): int => intdiv($t * 10, 100),
    'minus50k' => fn (int $t): int => min($t, 50000),
];

function checkout(array $lines, string $code, array $discounts): array
{
    if (!isset($discounts[$code])) {
        throw new InvalidArgumentException("Mã giảm giá không tồn tại: $code");
    }
```

```
    $subtotal = array_reduce(
        $lines,
        fn (int $carry, Line $line): int => $carry + $line->subtotal(),
        0
    );

    $discount = $discounts[$code]($subtotal);
    $shipping = shippingFee($subtotal - $discount);

    return [
        'subtotal' => $subtotal,
        'discount' => $discount,
        'shipping' => $shipping,
        'total' => $subtotal - $discount + $shipping,
    ];
}

$lines = [
    new Line(new Product('Áo', 200000), 2),
    new Line(new Product('Mũ', 50000), 1),
];

foreach (['none', 'vip10', 'minus50k'] as $code) {
    echo $code, ': ', json_encode(checkout($lines, $code, $discounts)), "\n";
}

try {
    checkout($lines, 'abc', $discounts);
} catch (InvalidArgumentException $e) {
    echo $e->getMessage(), "\n";
}

try {
    new Line(new Product('Áo', 200000), 0);
} catch (InvalidArgumentException $e) {
    echo $e->getMessage(), "\n";
}
```

KẾT QUẢ KHI CHẠY

```
none: {"subtotal":450000,"discount":0,"shipping":15000,"total":465000}
vip10: {"subtotal":450000,"discount":45000,"shipping":15000,"total":420000}
minus50k: {"subtotal":450000,"discount":50000,"shipping":15000,"total":415000}
Mã giảm giá không tồn tại: abc
Số lượng phải lớn hơn 0
```

Thiết kế

- `Product` và `Line` là **Value Object bất biến** (`final` , `readonly`). Kiểm tra dữ liệu hợp lệ ngay trong constructor của `Line` : không thể tồn tại một `Line` có `qty <= 0` . Nguyên tắc: đối tượng hợp lệ ngay từ lúc sinh ra.
- `$discounts` là **mảng ánh xạ mã sang closure**. Thêm mã mới chỉ cần thêm một dòng, không sửa hàm `checkout` (thiết kế mở rộng được). Mỗi closure nhận tổng tiền và trả số tiền giảm.
- `checkout` kiểm tra mã bằng `isset($discounts[$code])` , rồi gọi `$discounts[$code]($subtotal)` : lấy closure từ mảng bằng key rồi gọi ngay.

Tính toán (đơn mẫu)

- Subtotal: `200000*2 + 50000*1 = 450000`.
- `none`: giảm 0, tiền sau giảm 450000 thuộc khoảng 200000 đến dưới 500000 nên ship 15000, total = 450000 + 15000 = `465000`.
- `vip10`: giảm 45000, còn 405000 (vẫn trong khoảng 200000 đến dưới 500000) nên ship 15000, total = 450000 − 45000 + 15000 = `420000`.
- `minus50k`: giảm 50000, còn 400000, ship 15000, total = 450000 − 50000 + 15000 = `415000`.
- Phí ship tính trên số tiền **sau giảm** (quyết định thiết kế; đề có thể chọn tính trước giảm, hãy thử đổi và xem total thay đổi).

Điểm hay để thảo luận

- `min($t, 50000)` đảm bảo không giảm quá tổng tiền hàng.
- `intdiv` trả số nguyên (làm tròn xuống), tránh số thực với tiền.
- Hai khối `try/catch` đều bắt `InvalidArgumentException` nhưng đến từ hai nơi khác nhau: chứng tỏ exception đi xuyên qua nhiều lớp gọi hàm.

← Về đề bài 11.1

Đáp án 11.2 Báo cáo đơn hàng: sắp, lưu JSON, đọc lại

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

final class Product
{
    public function __construct(
        public readonly string $name,
        public readonly int $price,
    ) {}
}

final class Line
{
    public function __construct(
        public readonly Product $product,
        public readonly int $qty,
    ) {
        if ($qty <= 0) {
            throw new InvalidArgumentException('Số lượng phải lớn hơn 0');
        }
    }

    public function subtotal(): int
    {
        return $this->product->price * $this->qty;
    }
}

function loadOrder(string $path): array
{
    if (!file_exists($path)) {
        throw new RuntimeException('Không tìm thấy file: ' . basename($path));
    }
```

```
    return json_decode(file_get_contents($path), true, 512, JSON_THROW_ON_ERROR);
}

$lines = [
    new Line(new Product('Áo', 200000), 2),
    new Line(new Product('Mũ', 50000), 1),
    new Line(new Product('Quần', 350000), 1),
];

$rows = array_map(
    fn (Line $l): array => [
        'name' => $l->product->name,
        'qty' => $l->qty,
        'subtotal' => $l->subtotal(),
    ],
    $lines
);

usort($rows, fn (array $a, array $b): int => $b['subtotal'] <=> $a['subtotal']);

$report = ['topProduct' => $rows[0]['name'], 'lines' => $rows];

$path = __DIR__ . '/order.json';
file_put_contents($path, json_encode($report, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE));

$back = loadOrder($path);
echo $back['topProduct'], "\n";
echo count($back['lines']), "\n";

unlink($path);

try {
    loadOrder(__DIR__ . '/khong-co.json');
} catch (RuntimeException $e) {
    echo $e->getMessage(), "\n";
}
```

KẾT QUẢ KHI CHẠY

```
Áo
3
Không tìm thấy file: khong-co.json
```

Giải thích

- `array_map` biến mỗi `Line` (object) thành một **mảng kết hợp** gọn để lưu JSON. Object `readonly` có thuộc tính `public` nên đọc được bằng `->`, nhưng với những object phức tạp nên viết method `toArray()` riêng.
- `usort` với `$b['subtotal'] <=> $a['subtotal']` sắp giảm dần: Áo 400000, Quần 350000, Mũ 50000. `topProduct` là `Áo`.
- `json_decode($json, true, 512, JSON_THROW_ON_ERROR)`: tham số 3 là độ sâu tối đa, tham số 4 là cờ. `JSON_THROW_ON_ERROR` ném `JsonException` khi JSON hỏng thay vì trả `null` âm thầm.
- `loadOrder` ném `RuntimeException` khi file không có, **không tự in lỗi**; chỗ gọi quyết định.
- Vòng đi-về: object, mảng, JSON, file, mảng. Đây là cách tạo và đọc dữ liệu API đơn giản.

Kết nối với Laravel

Bài này chứa đủ ba tầng bạn sẽ thấy ở dự án Laravel: **Model/Value Object** (Product, Line), **Service** (checkout, tính toán), **Resource/Response** (mảng báo cáo, JSON). Chỉ khác framework lo phần định tuyến, database và HTTP.

← Về đề bài 11.2

ĐÁP ÁN — CHƯƠNG 12

SQL nền tảng với SQLite (chạy bằng PDO)

Đáp án 12.1 Dựng database mẫu bằng PDO

CODE ĐÁP ÁN

```
<?php

declare(strict_types=1);

function makeDb(): PDO
{
    $pdo = new PDO('sqlite::memory:');
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
    $pdo->exec('PRAGMA foreign_keys = ON');

    $pdo->exec('CREATE TABLE users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE
    )');

    $pdo->exec('CREATE TABLE posts (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER NOT NULL REFERENCES users(id),
        title TEXT NOT NULL,
        views INTEGER NOT NULL DEFAULT 0,
        published_at TEXT
    )');

    $pdo->exec('CREATE TABLE comments (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        post_id INTEGER NOT NULL REFERENCES posts(id),
        body TEXT NOT NULL
    )');

    $pdo->exec("INSERT INTO users (name, email) VALUES
        ('An', 'an@x.vn'), ('Bình', 'binh@x.vn'), ('Chi', 'chi@x.vn')");

    $pdo->exec("INSERT INTO posts (user_id, title, views, published_at) VALUES
        (1, 'Học PHP', 120, '2026-01-05'),
        (1, 'Học Laravel', 300, '2026-02-10'),
        (1, 'Học SQL', 80, NULL),
        (2, 'Docker cơ bản', 150, '2026-03-01'),
        (2, 'Redis là gì', 45, '2026-03-15')");

    $pdo->exec("INSERT INTO comments (post_id, body) VALUES
        (1, 'Hay quá'), (1, 'Cảm ơn bạn'),
        (2, 'Rất chi tiết'), (2, 'Có ví dụ không?'), (2, 'Đã thử, chạy tốt'),
        (4, 'Cần thêm phần Sail')");

    return $pdo;
}

function show(PDO $pdo, string $sql, array $params = []): void
```

```
{
    $stmt = $pdo->prepare($sql);
    $stmt->execute($params);

    foreach ($stmt->fetchAll() as $row) {
        echo implode(' | ', array_map(fn ($v) => $v === null ? 'NULL' : (string) $v, $row)),
"\n";
    }
}

$pdo = makeDb();

show($pdo, 'SELECT COUNT(*) FROM users');
show($pdo, 'SELECT COUNT(*) FROM posts');
show($pdo, 'SELECT COUNT(*) FROM comments');
show($pdo, 'SELECT id, title, views, published_at FROM posts');
```

KẾT QUẢ KHI CHẠY

```
3
5
6
1 | Học PHP | 120 | 2026-01-05
2 | Học Laravel | 300 | 2026-02-10
3 | Học SQL | 80 | NULL
4 | Docker cơ bản | 150 | 2026-03-01
5 | Redis là gì | 45 | 2026-03-15
```

Giải thích

- `new PDO('sqlite::memory:')` mở một database SQLite nằm hoàn toàn trong RAM. Không có file nào được tạo, tắt chương trình là mất, rất tiện để luyện SQL. Muốn lưu thành file thì viết `'sqlite:' . __DIR__ . '/blog.sqlite'`.
- `ERRMODE_EXCEPTION`: khi câu SQL sai, PDO ném `PDOException` thay vì im lặng trả `false`. Đây cũng là cách Laravel cấu hình PDO.
- **Khoá chính** `id INTEGER PRIMARY KEY AUTOINCREMENT`: định danh duy nhất của dòng, tự tăng. **Khoá ngoại** `user_id ... REFERENCES users(id)`: ràng buộc giá trị `user_id` phải là một `id` có thật trong `users`. Đó là thứ làm nên "quan hệ" giữa các bảng.
- `PRAGMA foreign_keys = ON` phải gọi **mỗi lần mở kết nối** vì SQLite mặc định không kiểm tra khoá ngoại (Laravel bật nó qua cấu hình `foreign_key_constraints`).
- `NOT NULL`, `UNIQUE`, `DEFAULT 0`: các ràng buộc để database tự chặn dữ liệu sai, không phụ thuộc vào code PHP.
- `prepare` rồi `execute($params)` là cách an toàn truyền giá trị vào câu SQL bằng dấu `?` (xem bài 12.11). `fetchAll()` lấy mọi dòng.
- Dòng `published_at` của bài `Học SQL` in ra `NULL`, vì bài này chưa xuất bản.

← Về đề bài 12.1

Đáp án 12.2 SELECT, WHERE, ORDER BY, LIMIT

CODE ĐÁP ÁN

```
<?php

declare(strict_types=1);
```

```php
$pdo = makeDb();

show($pdo, 'SELECT title, views FROM posts WHERE views > 100 ORDER BY views DESC');
echo "---\n";
show($pdo, 'SELECT title, views FROM posts ORDER BY views DESC LIMIT 2');
echo "---\n";
show($pdo, 'SELECT title, views FROM posts ORDER BY views DESC LIMIT 2 OFFSET 2');
echo "---\n";
show($pdo, "SELECT title FROM posts WHERE title LIKE 'Học%' ORDER BY id");
echo "---\n";
show($pdo, 'SELECT id FROM posts WHERE views BETWEEN 80 AND 150 OR user_id = 2 ORDER BY id');
```

Đoạn code trên chạy kèm phần chuẩn bị chung (hàm `makeDb()` và `show()` ở bài 12.1). Hãy chép phần đó vào đầu file của bạn.

KẾT QUẢ KHI CHẠY

```
Học Laravel | 300
Docker cơ bản | 150
Học PHP | 120
---
Học Laravel | 300
Docker cơ bản | 150
---
Học PHP | 120
Học SQL | 80
---
Học PHP
Học Laravel
Học SQL
---
1
3
4
5
```

Giải thích

- Thứ tự viết một câu `SELECT`: `SELECT` (cột) rồi `FROM` (bảng) rồi `WHERE` (lọc dòng) rồi `GROUP BY` rồi `HAVING` rồi `ORDER BY` rồi `LIMIT`. Thứ tự **thực thi** khác: `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT`.
- `ORDER BY views DESC` giảm dần, mặc định là `ASC` (tăng dần). Có thể sắp nhiều cột: `ORDER BY views DESC, title ASC` (giống `usort` nhiều tiêu chí ở bài 5.3).
- `LIMIT n` lấy tối đa `n` dòng; `OFFSET m` bỏ qua `m` dòng đầu. Trang thứ `p` với `perPage` dòng là `LIMIT perPage OFFSET (p - 1) * perPage`. Laravel `paginate()` sinh đúng câu này. Offset càng lớn càng chậm vì database vẫn phải đếm qua, đó là lý do Part 12 của lộ trình Laravel có `cursorPaginate`.
- `LIKE 'Học%'`: `%` khớp chuỗi bất kỳ, `_` khớp đúng một ký tự. Mẫu bắt đầu bằng chữ (`'Học%'`) có thể tận dụng index, nhưng mẫu `'%Học%'` thì không.
- `BETWEEN a AND b` gồm **cả hai đầu**. Điều kiện `OR` và `AND`: `AND` ưu tiên hơn `OR`, nên khi trộn hãy dùng ngoặc cho rõ.
- Chuỗi trong SQL dùng **nháy đơn** `'...'`. Nháy kép trong SQL chuẩn dành cho tên cột/bảng.

← Về đề bài 12.2

Đáp án 12.3 NULL và COUNT

KẾT QUẢ KHI CHẠY

```
1: 0
2: 1
3: 4
4: 5 | 4
5: 3
6: Học PHP
Học Laravel
Học SQL
```

Giải thích

- **Câu 1**: `= NULL` **luôn không đúng** với mọi dòng, vì so sánh bất kỳ với NULL cho kết quả "không biết" (không phải TRUE). Kết quả `0`. Phải dùng `IS NULL`.
- **Câu 2**: `IS NULL` đúng, tìm được bài nháp `Học SQL`, kết quả `1`.
- **Câu 3**: `IS NOT NULL` cho `4` bài đã xuất bản.
- **Câu 4**: `COUNT(*)` đếm **số dòng** (5), còn `COUNT(published_at)` đếm số dòng mà cột đó **khác NULL** (4). Hai hàm này khác nhau đúng ở NULL.
- **Câu 5**: `published_at != '2026-01-05'` với bài nháp (NULL) cho "không biết", nên bài nháp **bị loại**, kết quả `3` chứ không phải 4. Đây là bẫy kinh điển: muốn lấy cả dòng NULL phải viết thêm `OR published_at IS NULL`.
- **Câu 6**: kết hợp `IS NULL OR ...` lấy bài nháp và các bài xuất bản trước 01/03/2026 (so sánh chuỗi ngày dạng `YYYY-MM-DD` đúng thứ tự thời gian): `Học PHP`, `Học Laravel`, `Học SQL`.

Liên hệ Laravel

`->whereNull('published_at')` sinh `IS NULL`. Viết `->where('published_at', null)` Laravel cũng tự đổi thành `IS NULL`. Cột `$table->timestamp('published_at')->nullable()` trong migration là để cho phép NULL.

← Về đề bài 12.3

Đáp án 12.4 UPDATE, DELETE và ràng buộc khoá ngoại

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$pdo = makeDb();

$pdo->exec('UPDATE posts SET views = views + 1 WHERE id = 1');
show($pdo, 'SELECT views FROM posts WHERE id = 1');

try {
    $pdo->exec('DELETE FROM users WHERE id = 1');
} catch (PDOException $e) {
    echo $e->getMessage(), "\n";
}

$stmt = $pdo->prepare('DELETE FROM users WHERE id = ?');
$stmt->execute([3]);
```

```
echo $stmt->rowCount(), "\n";
show($pdo, 'SELECT COUNT(*) FROM users');
```

Đoạn code trên chạy kèm phần chuẩn bị chung (hàm `makeDb()` và `show()` ở bài 12.1). Hãy chép phần đó vào đầu file của bạn.

KẾT QUẢ KHI CHẠY

```
121
SQLSTATE[23000]: Integrity constraint violation: 19 FOREIGN KEY constraint failed
1
2
```

Giải thích

- `UPDATE posts SET views = views + 1 WHERE id = 1`: database tự đọc giá trị hiện tại, cộng 1, ghi lại **trong một thao tác nguyên tử**. Nếu đọc ra bằng PHP, cộng rồi ghi, hai request cùng lúc có thể ghi đè nhau và mất lượt xem (race condition). Laravel có `$post->increment('views')` sinh đúng câu này.
- **Luôn có** `WHERE` trong `UPDATE` và `DELETE`. Thiếu `WHERE` thì **mọi dòng** bị sửa hoặc xoá, và database không hỏi lại.
- Xoá user 1 thất bại vì các bài `posts.user_id = 1` đang trỏ tới user này. Khoá ngoại **bảo vệ tính toàn vẹn**: không để tồn tại bài viết mồ côi. Thông báo chứa `FOREIGN KEY constraint failed`.
- Có ba cách xử lý khi xoá cha: (1) chặn (mặc định, như trên), (2) `ON DELETE CASCADE`: xoá cha thì tự xoá con (trong Laravel: `->constrained()->cascadeOnDelete()`), (3) `ON DELETE SET NULL`: đặt khoá ngoại con về NULL (cột phải cho phép NULL).
- User 3 chưa có bài nào nên xoá được; `rowCount()` cho biết số dòng bị ảnh hưởng (`1`). Còn lại 2 user.

← Về đề bài 12.4

Đáp án 12.5 GROUP BY, hàm gộp và HAVING

CODE ĐÁP ÁN

```
<?php

declare(strict_types=1);

$pdo = makeDb();

show($pdo, 'SELECT user_id, COUNT(*) AS bai, SUM(views) AS tong_view FROM posts GROUP BY
user_id ORDER BY user_id');
echo "---\n";
show($pdo, 'SELECT user_id, SUM(views) AS tong_view FROM posts GROUP BY user_id HAVING
SUM(views) > 200');
echo "---\n";
show($pdo, 'SELECT ROUND(AVG(views), 1), MAX(views), MIN(views) FROM posts WHERE published_at
IS NOT NULL');
echo "---\n";
show($pdo, 'SELECT user_id, COUNT(*) FROM posts WHERE published_at IS NOT NULL GROUP BY user_id
HAVING COUNT(*) >= 2 ORDER BY user_id');
```

Đoạn code trên chạy kèm phần chuẩn bị chung (hàm `makeDb()` và `show()` ở bài 12.1). Hãy chép phần đó vào đầu file của bạn.

KẾT QUẢ KHI CHẠY

```
1 | 3 | 500
2 | 2 | 195
---
1 | 500
---
153.8 | 300 | 45
---
1 | 2
2 | 2
```

Giải thích

- `GROUP BY user_id` gộp các dòng cùng `user_id` thành một nhóm; hàm gộp (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) tính trên **từng nhóm**. User 1 có 3 bài (120 + 300 + 80 = 500 lượt xem), user 2 có 2 bài (150 + 45 = 195).
- `WHERE` **lọc dòng trước khi gom nhóm;** `HAVING` **lọc nhóm sau khi gom**. Điều kiện trên kết quả hàm gộp (`SUM(views) > 200`) phải để ở `HAVING`, `WHERE` không dùng được hàm gộp. Nhóm 2 chỉ còn user 1 (500 > 200); user 2 (195) bị loại.
- Nhóm 3: bài đã xuất bản có views 120, 300, 150, 45; trung bình 153.75 làm tròn `153.8`, lớn nhất `300`, nhỏ nhất `45`. Bài nháp (80) không được tính nhờ `WHERE`.
- Nhóm 4: sau khi `WHERE` loại bài nháp, user 1 còn 2 bài, user 2 còn 2 bài, cả hai thoả `COUNT(*) >= 2`. Nếu không có `WHERE`, user 1 sẽ có 3 bài. Thử bỏ `WHERE` để thấy khác biệt.
- `AS bai` đặt **bí danh** cho cột. Trong `SELECT` của câu `GROUP BY`, mọi cột không nằm trong hàm gộp phải có trong `GROUP BY`.

Liên hệ Laravel

`Post::selectRaw('user_id, COUNT(*) as bai')->groupBy('user_id')->having('bai', '>', 1)->get()`; và `withCount('comments')` ở Part 08 thực chất sinh một truy vấn con kiểu này.

← Về đề bài 12.5

Đáp án 12.6 INNER JOIN: ghép bảng theo khoá ngoại

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$pdo = makeDb();

show($pdo, 'SELECT p.title, u.name FROM posts p JOIN users u ON u.id = p.user_id ORDER BY p.id');
echo "---\n";
show($pdo, 'SELECT c.body, p.title, u.name
    FROM comments c
    JOIN posts p ON p.id = c.post_id
    JOIN users u ON u.id = p.user_id
    ORDER BY c.id');
echo "---\n";
show($pdo, 'SELECT p.title, COUNT(c.id) AS so_comment
    FROM posts p
    JOIN comments c ON c.post_id = p.id
```

```
    GROUP BY p.id
    ORDER BY so_comment DESC, p.id');
```

Đoạn code trên chạy kèm phần chuẩn bị chung (hàm `makeDb()` và `show()` ở bài 12.1). Hãy chép phần đó vào đầu file của bạn.

KẾT QUẢ KHI CHẠY

```
Học PHP | An
Học Laravel | An
Học SQL | An
Docker cơ bản | Bình
Redis là gì | Bình
---
Hay quá | Học PHP | An
Cảm ơn bạn | Học PHP | An
Rất chi tiết | Học Laravel | An
Có ví dụ không? | Học Laravel | An
Đã thử, chạy tốt | Học Laravel | An
Cần thêm phần Sail | Docker cơ bản | Bình
---
Học Laravel | 3
Học PHP | 2
Docker cơ bản | 1
```

Giải thích

- `JOIN ... ON` ghép hai bảng theo điều kiện, thường là `khoá_ngoại = khoá_chính`. Mỗi dòng của `posts` được ghép với dòng `users` có `u.id = p.user_id`. `p` và `u` là **bí danh bảng**, ngắn gọn hơn và bắt buộc khi hai bảng có cột trùng tên (đều có `id`).
- `JOIN` mặc định là **INNER JOIN**: chỉ giữ những dòng ghép được ở **cả hai** phía. Bài viết nào không có tác giả (không xảy ra do khoá ngoại) hay user nào không có bài sẽ không xuất hiện.
- Ghép ba bảng thì viết nối tiếp: `comments` ghép `posts`, rồi ghép tiếp `users`. Điều kiện của mỗi `JOIN` nối vào bảng đã có.
- Câu 3 dùng `INNER JOIN` nên bài không có bình luận (`Học SQL`, `Redis là gì`) **biến mất khỏi kết quả**. Để giữ chúng lại với số 0 cần `LEFT JOIN` (bài 12.7).
- `COUNT(c.id)` đếm số `id` bình luận khác NULL trong nhóm.

Liên hệ Laravel

`Post::join('users', 'users.id', '=', 'posts.user_id')->select('posts.title', 'users.name')`. Quan hệ `belongsTo` và `hasMany` của Eloquent **không** dùng JOIN: nó chạy truy vấn riêng cho mỗi bảng (xem bài 12.8).

← Về đề bài 12.6

Đáp án 12.7 LEFT JOIN: giữ lại dòng không có bên kia

CODE ĐÁP ÁN

```
<?php

declare(strict_types=1);

$pdo = makeDb();

show($pdo, 'SELECT u.name, COUNT(p.id) AS so_bai
```

```
    FROM users u
    LEFT JOIN posts p ON p.user_id = u.id
    GROUP BY u.id
    ORDER BY u.id');
echo "---\n";
show($pdo, 'SELECT p.title
    FROM posts p
    LEFT JOIN comments c ON c.post_id = p.id
    WHERE c.id IS NULL
    ORDER BY p.id');
echo "---\n";
show($pdo, 'SELECT p.title, COUNT(c.id) AS so_comment
    FROM posts p
    LEFT JOIN comments c ON c.post_id = p.id
    GROUP BY p.id
    ORDER BY so_comment DESC, p.id');
```

Đoạn code trên chạy kèm phần chuẩn bị chung (hàm `makeDb()` và `show()` ở bài 12.1). Hãy chép phần đó vào đầu file của bạn.

KẾT QUẢ KHI CHẠY

```
An | 3
Bình | 2
Chi | 0
---
Học SQL
Redis là gì
---
Học Laravel | 3
Học PHP | 2
Docker cơ bản | 1
Học SQL | 0
Redis là gì | 0
```

Giải thích

- `LEFT JOIN` giữ **mọi dòng của bảng bên trái** (đứng trước `LEFT JOIN`). Dòng nào không ghép được với bảng bên phải thì các cột bên phải là **NULL**.
- Câu 1: Chi không có bài nào nên các cột `p.*` là NULL. Dùng `COUNT(p.id)` (đếm cột khác NULL) cho ra `0` là đúng, còn `COUNT(*)` sẽ đếm cả dòng NULL đó và ra `1`, sai. Đây là điểm bài 12.3 đã nhắc về `COUNT(*)` và `COUNT(cột)`.
- Câu 2: mẫu **"tìm dòng không có dòng tương ứng"** (anti-join): `LEFT JOIN` rồi `WHERE bảng_phải.id IS NULL`. Kết quả `Học SQL` và `Redis là gì`. Cách viết tương đương: `WHERE NOT EXISTS (SELECT 1 FROM comments c WHERE c.post_id = p.id)`; Eloquent là `Post::doesntHave('comments')`.
- Câu 3: giống câu 3 của bài 12.6 nhưng dùng `LEFT JOIN` nên hai bài 0 bình luận có mặt ở cuối.
- Quy tắc chọn: cần **chỉ những dòng có quan hệ** thì `INNER JOIN`; cần **mọi dòng bên trái, có hay không bên phải** thì `LEFT JOIN`.

> **Lưu ý:** Cạm bẫy: đặt điều kiện lọc của bảng bên phải trong `WHERE` (thay vì trong `ON`) làm `LEFT JOIN` biến thành `INNER JOIN`, vì dòng NULL bị lọc mất. Điều kiện chỉ ảnh hưởng bảng phải nên để ở `ON`.

← Về đề bài 12.7

Đáp án 12.8 Vấn đề N+1 ở tầng SQL

CODE ĐÁP ÁN

```
<?php

declare(strict_types=1);

$pdo = makeDb();

$queries = 0;
$run = function (string $sql, array $params = []) use ($pdo, &$queries): array {
    $queries++;
    $stmt = $pdo->prepare($sql);
    $stmt->execute($params);

    return $stmt->fetchAll();
};

$posts = $run('SELECT id, user_id, title FROM posts ORDER BY id');
foreach ($posts as $post) {
    $author = $run('SELECT name FROM users WHERE id = ?', [$post['user_id']]);
}
echo 'A (N+1): ', $queries, " câu\n";

$queries = 0;
$posts = $run('SELECT id, user_id, title FROM posts ORDER BY id');
$ids = array_values(array_unique(array_column($posts, 'user_id')));
$marks = implode(', ', array_fill(0, count($ids), '?'));
$users = $run("SELECT id, name FROM users WHERE id IN ($marks)", $ids);
$names = array_column($users, 'name', 'id');
foreach ($posts as $post) {
    $name = $names[$post['user_id']];
}
echo 'B (IN): ', $queries, " câu\n";

$queries = 0;
$rows = $run('SELECT p.title, u.name FROM posts p JOIN users u ON u.id = p.user_id ORDER BY
p.id');
echo 'C (JOIN): ', $queries, " câu\n";
```

Đoạn code trên chạy kèm phần chuẩn bị chung (hàm `makeDb()` và `show()` ở bài 12.1). Hãy chép phần đó vào đầu file của bạn.

KẾT QUẢ KHI CHẠY

```
A (N+1): 6 câu
B (IN): 2 câu
C (JOIN): 1 câu
```

Giải thích

- **Cách A: 6 câu** (1 câu lấy 5 bài + 5 câu lấy tác giả). Tổng quát là `1 + N` câu với `N` bài, nên gọi là **N+1**. Với 1000 bài là 1001 lần gọi database, và mỗi lần tốn thời gian mạng và phân tích câu lệnh. Đây là lỗi hiệu năng phổ biến nhất của mọi ORM (Part 08 của lộ trình).
- **Cách B: 2 câu**, bất kể có bao nhiêu bài. Đây chính xác là thứ `Post::with('author')->get()` của Eloquent làm: một câu lấy bài, một câu `WHERE id IN (...)` lấy tác giả rồi **ghép trong PHP**. Điểm mấu chốt là dùng

`array_unique` để chỉ hỏi mỗi user một lần và `array_column($users, 'name', 'id')` tạo bảng tra cứu `id sang tên`.

- **Cách C: 1 câu** bằng JOIN. Gọn nhất với quan hệ **một-một / nhiều-một**. Với quan hệ **một-nhiều** (bài và bình luận), JOIN nhân bản dòng cha theo số con nên kết quả to; Eloquent chọn cách B vì lý do này.
- `use ($pdo, &$queries)` : `$pdo` đọc được bằng giá trị (object là handle), còn `$queries` cần `&` để closure **tăng được biến bên ngoài**.
- `array_fill(0, n, '?')` tạo mảng `n` dấu `?`; `implode(', ', ...)` thành chuỗi `?, ?` để đặt vào `IN (...)`. Luôn dùng `?` (tham số hoá) thay vì ghép giá trị vào chuỗi.

Cách phát hiện N+1 trong Laravel

Bật `Model::preventLazyLoading()` ở môi trường dev để lazy loading ném lỗi, hoặc xem số query bằng Telescope/Debugbar. Chữa bằng `with()`.

← Về đề bài 12.8

Đáp án 12.9 **Index và EXPLAIN QUERY PLAN**

CODE ĐÁP ÁN

```php
<?php

declare(strict_types=1);

$pdo = new PDO('sqlite::memory:');
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);

$pdo->exec('CREATE TABLE logs (id INTEGER PRIMARY KEY, user_id INTEGER NOT NULL, action TEXT
NOT NULL)');

$pdo->beginTransaction();
$stmt = $pdo->prepare('INSERT INTO logs (user_id, action) VALUES (?, ?)');
for ($i = 0; $i < 20000; $i++) {
    $stmt->execute([$i % 1000, $i % 2 === 0 ? 'login' : 'view']);
}
$pdo->commit();

function plan(PDO $pdo, string $sql): void
{
    foreach ($pdo->query('EXPLAIN QUERY PLAN ' . $sql)->fetchAll() as $row) {
        echo $row['detail'], "\n";
    }
}

echo "1. Chưa có index:\n";
plan($pdo, 'SELECT * FROM logs WHERE user_id = 42');

$pdo->exec('CREATE INDEX idx_logs_user_id ON logs (user_id)');

echo "2. Đã có index user_id:\n";
plan($pdo, 'SELECT * FROM logs WHERE user_id = 42');

echo "3. Lọc theo cột chưa có index:\n";
plan($pdo, "SELECT * FROM logs WHERE action = 'login'");

$pdo->exec('CREATE INDEX idx_logs_user_action ON logs (user_id, action)');
```

```
echo "4. Index hợp (user_id, action):\n";
plan($pdo, "SELECT * FROM logs WHERE user_id = 42 AND action = 'login'");
```

KẾT QUẢ KHI CHẠY

```
1. Chưa có index:
SCAN logs
2. Đã có index user_id:
SEARCH logs USING INDEX idx_logs_user_id (user_id=?)
3. Lọc theo cột chưa có index:
SCAN logs
4. Index hợp (user_id, action):
SEARCH logs USING COVERING INDEX idx_logs_user_action (user_id=? AND action=?)
```

Giải thích

- **Index** giống mục lục cuối sách. Không có index, database phải đọc **từng dòng** của bảng (quét toàn bảng, `SCAN` trong SQLite, `type: ALL` trong MySQL). Có index, nó nhảy thẳng tới các dòng khớp (`SEARCH ... USING INDEX`). Cấu trúc bên dưới thường là **B-tree**, cho phép tìm trong `log n` bước thay vì `n` (cùng ý tưởng binary search ở bài 9.4).
- Kết quả 1: `SCAN logs` , quét cả 20000 dòng để tìm user 42. Kết quả 2: `SEARCH logs USING INDEX idx_logs_user_id (user_id=?)` , chỉ chạm vào khoảng 20 dòng của user 42.
- Kết quả 3: cột `action` chưa có index nên vẫn `SCAN` . Index chỉ giúp những câu lọc/sắp xếp/ghép **đúng cột đó**.
- Kết quả 4: SQLite chọn index hợp `(user_id, action)` vì nó phủ cả hai điều kiện. Chữ `COVERING INDEX` nghĩa là mọi cột câu truy vấn cần (`id` , `user_id` , `action`) đã nằm sẵn trong index, nên database không phải đọc thêm bảng gốc: nhanh hơn nữa. Quy tắc **tiền tố trái (leftmost prefix)**: index `(a, b)` giúp được `WHERE a = ?` và `WHERE a = ? AND b = ?` , nhưng **không** giúp `WHERE b = ?` đứng riêng. Thứ tự cột trong index hợp rất quan trọng.
- **Cái giá của index**: chiếm dung lượng và làm `INSERT/UPDATE/DELETE` chậm hơn vì phải cập nhật cả index. Chỉ tạo index cho cột hay xuất hiện ở `WHERE` , `JOIN` , `ORDER BY` . Đừng đánh index mọi cột.
- Chèn 20000 dòng trong **một transaction** (`beginTransaction` / `commit`) nhanh hơn nhiều so với để mỗi `INSERT` tự commit.

Liên hệ Laravel

Trong migration: `$table->index('user_id')` , `$table->index(['user_id', 'action'])` , `$table->unique('email')` . Cột `foreignId('user_id')->constrained()` tạo **khoá ngoại**; MySQL tự đánh index cho cột khoá ngoại, còn SQLite thì không, nên hãy chủ động `->index()` khi cần. Xem kế hoạch trong MySQL/PostgreSQL bằng `EXPLAIN SELECT ...` chạy trong client SQL, hoặc từ code qua `DB::select('EXPLAIN SELECT ...')` .

← Về đề bài 12.9

Đáp án 12.10 Transaction: tất cả hoặc không gì cả

CODE ĐÁP ÁN

```
<?php

declare(strict_types=1);

$pdo = new PDO('sqlite::memory:');
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

```php
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);

$pdo->exec('CREATE TABLE accounts (id INTEGER PRIMARY KEY, name TEXT NOT NULL, balance INTEGER
NOT NULL)');
$pdo->exec("INSERT INTO accounts (id, name, balance) VALUES (1, 'An', 1000), (2, 'Bình',
500)");

function balances(PDO $pdo): string
{
    $rows = $pdo->query('SELECT name, balance FROM accounts ORDER BY id')->fetchAll();

    return implode(', ', array_map(fn (array $r): string => $r['name'] . '=' . $r['balance'],
$rows));
}

function transfer(PDO $pdo, int $from, int $to, int $amount): void
{
    $pdo->beginTransaction();

    try {
        $debit = $pdo->prepare('UPDATE accounts SET balance = balance - ? WHERE id = ? AND
balance >= ?');
        $debit->execute([$amount, $from, $amount]);

        if ($debit->rowCount() === 0) {
            throw new RuntimeException('Sô´dư không đủ');
        }

        $credit = $pdo->prepare('UPDATE accounts SET balance = balance + ? WHERE id = ?');
        $credit->execute([$amount, $to]);

        $pdo->commit();
    } catch (Throwable $e) {
        $pdo->rollBack();
        throw $e;
    }
}

echo 'Ban đâù: ', balances($pdo), "\n";

transfer($pdo, 1, 2, 300);
echo 'Sau khi chuyêñ 300: ', balances($pdo), "\n";

try {
    transfer($pdo, 1, 2, 5000);
} catch (RuntimeException $e) {
    echo 'Lôĩ: ', $e->getMessage(), "\n";
}
echo 'Sau lâǹ lôĩ: ', balances($pdo), "\n";
```

KẾT QUẢ KHI CHẠY

```
Ban đâù: An=1000, Bình=500
Sau khi chuyêñ 300: An=700, Bình=800
Lôĩ: Sô´dư không đủ
Sau lâǹ lôĩ: An=700, Bình=800
```

Giải thích

- **Transaction** gom nhiều câu lệnh thành một đơn vị: hoặc **tất cả thành công** (`commit`), hoặc **không gì thay đổi** (`rollBack`). Đây là tính **nguyên tử (Atomicity)** trong ACID. Nếu trừ tiền xong mà cộng tiền thất bại, không có transaction thì tiền "biến mất".
- Mẫu chuẩn: `beginTransaction()` , thao tác trong `try` , `commit()` nếu ổn, `catch` thì `rollBack()` rồi **ném lại** lỗi (`throw $e`) để chỗ gọi biết.
- `UPDATE ... WHERE id = ? AND balance >= ?` : kiểm tra đủ tiền **ngay trong câu UPDATE**, không đọc ra kiểm tra rồi ghi (sẽ dính race condition như bài 12.4). `rowCount() === 0` nghĩa là không có dòng nào thoả điều kiện, tức không đủ tiền.
- Chuyển 300: An 700, Bình 800. Lần chuyển 5000 thất bại, `rollBack` đảm bảo số dư vẫn `An=700, Bình=800` .
- Bẫy kinh điển của Laravel (Part 13): job hoặc event bắn **bên trong** transaction có thể chạy trước khi dữ liệu được commit. Hãy dùng `afterCommit` .

Liên hệ Laravel

```
DB::transaction(function () use ($from, $to, $amount) { ... });
```

Closure bên trong tự `commit` khi chạy xong và tự `rollBack` rồi ném lại exception khi có lỗi: đúng mẫu ở trên, gói gọn trong một hàm (và đây là lý do closure xuất hiện nhiều ở Laravel).

← Về đề bài 12.10

Đáp án 12.11 SQL injection: ghép chuỗi và tham số hoá

CODE ĐÁP ÁN

```
<?php

declare(strict_types=1);

$pdo = makeDb();
$input = "' OR '1'='1";

echo "Ghép chuôĩ:\n";
show($pdo, "SELECT name FROM users WHERE email = '$input'");

echo "Tham sô´hoá:\n";
show($pdo, 'SELECT name FROM users WHERE email = ?', [$input]);

function safeSortColumn(string $input): string
{
    return in_array($input, ['id', 'title', 'views'], true) ? $input : 'id';
}

echo safeSortColumn('views'), "\n";
echo safeSortColumn('views; DROP TABLE posts'), "\n";
```

Đoạn code trên chạy kèm phần chuẩn bị chung (hàm `makeDb()` và `show()` ở bài 12.1). Hãy chép phần đó vào đầu file của bạn.

KẾT QUẢ KHI CHẠY

```
Ghép chuôĩ:
An
Bình
Chi
```

```
Tham số hoá:
views
id
```

Giải thích

- **Cách nguy hiểm**: câu SQL thực tế thành `... WHERE email = '' OR '1'='1'`. Điều kiện `'1'='1'` luôn đúng, nên trả về **toàn bộ user**. Kẻ tấn công đăng nhập được mà không cần biết email. Biến thể nặng hơn là chèn `; DROP TABLE ...` hoặc đọc dữ liệu bảng khác bằng `UNION`.
- **Cách an toàn (prepared statement)**: câu SQL được gửi cho database **trước**, với `?` là chỗ trống; giá trị gửi **riêng sau**. Database coi giá trị chỉ là **dữ liệu**, không bao giờ là một phần của câu lệnh, nên cả chuỗi `' OR '1'='1` chỉ được tìm như một địa chỉ email lạ. Kết quả: không có dòng nào.
- Tham số hoá **chỉ áp dụng cho giá trị**, không áp dụng cho **tên cột, tên bảng, hướng sắp xếp** (`ORDER BY ?` không hoạt động như bạn muốn). Với những thứ này phải dùng **danh sách cho phép (allowlist)**: `in_array($input, [...], true)`. Đây là cảnh báo chính thức của docs Laravel: "PDO does not support binding column names. Therefore, you should never allow user input to dictate the column names referenced by your queries, including order by columns".
- Query Builder và Eloquent **tự tham số hoá** mọi giá trị bạn truyền, nên bạn không cần làm sạch chuỗi. Nguy cơ quay lại khi bạn dùng `DB::raw`, `whereRaw`, `selectRaw`, `orderByRaw` và **ghép biến người dùng vào chuỗi**. Khi buộc phải dùng `Raw`, truyền giá trị qua tham số thứ hai: `whereRaw('views > ?', [$min])`.

Quy tắc ngắn gọn

1. Giá trị từ người dùng luôn đi qua tham số `?` hoặc Query Builder.
2. Tên cột, bảng, hướng sắp xếp từ người dùng luôn đi qua allowlist.
3. Không bao giờ nội suy `$biến` hay nối `.` vào câu SQL.

← Về đề bài 12.11

PHỤ LỤC

Phụ lục A: Bảng hàm array cần nhớ

Hàm	Tác dụng	Chú ý
`array_map(fn, $a)`	Biến đổi từng phần tử	Callback đứng **trước**; giữ key
`array_filter($a, fn)`	Giữ phần tử thoả điều kiện	Mảng đứng **trước**; **giữ key**, bọc `array_values`
`array_reduce($a, fn, $init)`	Gộp thành một giá trị	Nhớ giá trị khởi đầu
`array_values($a)`	Bỏ key, đánh lại 0,1,2	Dùng sau `array_filter`, `array_unique`
`array_keys($a)`	Lấy các key	
`array_column($rows, 'c', 'k')`	Lấy một cột từ mảng 2 chiều	Đánh lại chỉ số khi không có tham số thứ 3
`array_slice($a, $off, $len)`	Lấy một đoạn	Không sửa mảng gốc
`array_merge($a, $b)`	Nối mảng	Key chuỗi trùng thì giá trị sau ghi đè
`array_unique($a)`	Loại trùng	Giữ key đầu tiên, cần `array_values`
`array_search($v, $a, true)`	Tìm key của giá trị	Trả `false` nếu không có, so bằng `=== false`
`in_array($v, $a, true)`	Có chứa không	Luôn dùng tham số `true`
`array_sum`, `max`, `min`, `count`	Tổng, lớn nhất, nhỏ nhất, đếm	
`array_key_first`, `array_key_last`	Key đầu / cuối	An toàn hơn `end()`
`array_combine($keys, $vals)`	Ghép hai mảng thành mảng kết hợp	Hai mảng phải cùng độ dài
`array_is_list($a)`	Key có liên tục từ 0 không	PHP 8.1+
`usort($a, fn)`	Sắp xếp tuỳ chỉnh tại chỗ	Trả `true`, không trả mảng
`implode($sep, $a)` / `explode($sep, $s)`	Mảng thành chuỗi / chuỗi thành mảng	

Phụ lục B: Hàm file, chuỗi, JSON hay dùng

Hàm	Tác dụng
`file_exists($p)`	File/thư mục có tồn tại không
`file_get_contents($p)` / `file_put_contents($p, $d, FILE_APPEND)`	Đọc cả file / ghi (nối nếu có cờ)
`fopen`, `fgets`, `fclose`	Mở, đọc từng dòng, đóng
`fputcsv`, `fgetcsv`	Ghi, đọc một dòng CSV
`file($p, FILE_IGNORE_NEW_LINES)`	Đọc file thành mảng dòng

`unlink($p)`	Xoá file
`__DIR__`, `basename($p)`	Thư mục hiện tại, tên file
`json_encode($d, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE)`	Mảng thành JSON
`json_decode($s, true, 512, JSON_THROW_ON_ERROR)`	JSON thành mảng, ném lỗi nếu hỏng
`sprintf`, `printf`, `number_format`	Định dạng chuỗi và số
`str_split`, `strrev`, `strtolower`, `rtrim`	Thao tác chuỗi cơ bản

Phụ lục C: Các lỗi hay gặp và cách nhận ra

Triệu chứng	Nguyên nhân thường gặp	Cách sửa
`Undefined variable $x`	Closure `function` quên `use ($x)`	Thêm `use`, hoặc dùng `fn`
`Undefined array key "x"`	Đọc key không tồn tại	Dùng `??`, hoặc kiểm tra `isset`
JSON ra `{...}` thay vì `[...]`	Key không liên tục sau `array_filter`	Bọc `array_values()`
`UnhandledMatchError`	`match` không có nhánh khớp, thiếu `default`	Thêm `default` hoặc bắt lỗi
`match (x)` với điều kiện khoảng luôn lỗi	Dùng `match ($x)` thay vì `match (true)`	Đổi thành `match (true)`
`TypeError`	Truyền sai kiểu dưới `strict_types`	Ép kiểu rõ ràng, ví dụ `(int)`
Dữ liệu CSV bị nhân đôi	`fopen` chế độ `'a'` chạy nhiều lần	Xoá file trước khi thử, hoặc kiểm tra tồn tại
`$x = usort(...)` ra `true`	`usort` sửa tại chỗ	Gọi `usort($mảng, ...)` rồi dùng `$mảng`
Counter không tăng trong closure	Quên `&` ở `use (&$n)` hoặc dùng `fn`	Dùng `function` với `use (&$n)`
Tiếng Việt bị `\u...` trong JSON	Thiếu `JSON_UNESCAPED_UNICODE`	Thêm cờ
Sort sai khi bằng nhau	Thứ tự `?:` hoặc so sánh một tiêu chí	Thêm tiêu chí phụ bằng `?:`